

in28minutes.com · Interview Preparation Series

JAVA

Interview Guide

17 Chapters · 215 Questions & Answers · June 2026

Table of Contents

1	Java Platform	<i>19 Q&A</i>
2	Java Landscape	<i>12 Q&A</i>
3	Java Memory Management	<i>13 Q&A</i>
4	Java Object Oriented Programming Concepts	<i>24 Q&A</i>
5	Java Object Oriented Programming Details	<i>14 Q&A</i>
6	Core Java in Depth	<i>17 Q&A</i>
7	Functional Programming in Java	<i>16 Q&A</i>
8	Threads and Concurrency in Java	<i>21 Q&A</i>
9	Collections in Java	<i>22 Q&A</i>
10	Generics in Java	<i>5 Q&A</i>
11	Modifiers in Java	<i>5 Q&A</i>
12	Exception Handling in Java	<i>10 Q&A</i>
13	Serialization in Java	<i>7 Q&A</i>
14	Built-in Classes in Java	<i>9 Q&A</i>
15	Miscellaneous Topics in Java	<i>11 Q&A</i>
16	New Features in Java	<i>3 Q&A</i>
17	Java Interview - Diagrams	<i>7 Q&A</i>

CHAPTER 1

Java Platform

19 Questions & Answers

Q1

What is Platform Independence?

- What?: Ability of a software to run on different operating systems without changes.
- How Java Achieves It: JVM (Java Virtual Machine) runs the same bytecode on any OS.
- Example: A Java program compiled on Windows runs on Linux or macOS without recompilation.
- C/C++ are NOT platform independent:
Need separate compilation for each OS.

Java

```
public class Example {  
    public static void main(String[] args) {  
        System.out.println("Java is platform-independent!");  
    }  
}
```

Q2

Platform Independence vs Portability**Platform Independence**

- What?: Ability of software to run on different operating systems without modification.
- Example:
Java compiles to bytecode, which runs on any JVM.
A Java program compiled on Windows runs on Linux or macOS without recompilation.
Java follows "Write Once, Run Anywhere".

Portability

- What?: Ability of software to be adapted to different platforms with minor changes.
- Example:
A C/C++ program can be run on different OSs after separate compilations for each OS.
C/C++ are portable but not platform-independent.

Remember:

- Java is both portable and platform-independent
- C/C++ are portable but not platform-independent
- Assembly Language is neither portable nor platform-independent

Q3

Code vs Machine Code vs Bytecode

Code (Source Code)

- What?: Human-readable instructions written in a programming language.
- Execution: Needs to be converted to machine code before running on a machine.
- Example (Java Source Code):

Bytecode

- What?: Intermediate, platform-independent code executed by a virtual machine (JVM).
- Execution: Runs on the Java Virtual Machine (JVM), which translates it into machine code at runtime.
- Example (Java Bytecode - Generated from `javac Hello.java`):

Machine Code

- What?: Low-level binary instructions that a CPU can execute directly.
- Execution: Runs directly on hardware without further translation.
- Example (x86 Machine Code):

Java

```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

Code

```
0: ldc "Hello, World!"  
2: invokevirtual #2  
5: return
```

Code

```
10110000 00000101 ; (Move value 5 into register AL)
00000100 00000011 ; (Add 3 to AL)
```

Q4

What are the major events in programming history that led to Java?

Key Events

- Writing Machine Code
- Using Assembly Language
- Structured Programming with C Language
- OOPS with C++
- OOPS with Java

1: Writing Machine Code

- Machine Code: Consists of binary instructions (0s and 1s) that the CPU directly executes
Performs basic operations like moving data, adding numbers, or jumping to another instruction
- (Advantage) Fastest execution since it runs directly on hardware
- (Disadvantage) Extremely difficult to write and debug manually
- (Disadvantage) Different for each CPU architecture
- Example:

Register: A small, high-speed storage location inside the CPU that temporarily holds data while processing instructions

2: Using Assembly Language

- Assembly Language: Human-readable version of machine code, using mnemonics like MOV, ADD, and JMP
- Each instruction maps directly to a machine code equivalent
- (Advantage) Easier than writing raw machine code
- (Disadvantage) BUT still verbose and difficult to write
- (Disadvantage) Different for each CPU architecture
- Example 1: Simple Addition:
- Example 2: Average of 4 numbers:

3: Structured Programming with C Language

- Created by Dennis Ritchie at Bell Labs in the 1970s
- Used structured programming
- (Advantage) Easier than using assembly language
- (Advantage) More easier to adapt to different CPU architectures
- (Disadvantage) Building large apps with hundreds of thousands of lines of code is challenging
- Example:

4: OOPS with C++

- Developed by Bjarne Stroustrup as an extension of C in the 1980s
- Supports both procedural and object-oriented paradigms
 - Object Oriented Features: classes, inheritance, and polymorphism
- Example:

5: Java

- Developed by Sun Microsystems in the 1990s
- Introduced platform independence with the JVM (Java Virtual Machine)
- Provides automatic memory management (garbage collection)
- Commonly used for web applications, enterprise software, and Android development
- Example:

Code

```
10110000 00000101 ; (Move value 5 into register AL)
00000100 00000011 ; (Add 3 to AL)
```

Code

```
MOV AX, 5 ; Load 5 into register AX
ADD AX, 3 ; Add 3 to AX
```

Code

```

; Load first number into AX
MOV AX, 10      ; AX = 10

; Load second number into BX
MOV BX, 20      ; BX = 20
ADD AX, BX      ; AX = AX + BX (10 + 20)

; Load third number into CX
MOV CX, 30      ; CX = 30
ADD AX, CX      ; AX = AX + CX (30)

; Load fourth number into DX
MOV DX, 40      ; DX = 40
ADD AX, DX      ; AX = AX + DX (40)

; Divide by 4 to get the average
; for word-sized division
; the quotient is always stored in AX
; the remainder is stored in DX
MOV DX, 0       ; Clear DX before division
MOV CX, 4       ; Divide by 4
DIV CX          ; AX = AX / 4, remainder in DX

```

Code

```

#include <stdio.h>

int main()
{
    int num1 = 10, num2 = 20, num3 = 30, num4 = 40;
    float average;

    // Calculate the average
    average = (num1 + num2 + num3 + num4) / 4.0;

    // Print the result
    printf("Average is: %.2f\n", average);

    return 0;
}

```

Code


```

#include <iostream>

class AverageCalculator {
private:
    int num1, num2, num3, num4;
public:
    // Constructor to initialize numbers
    AverageCalculator(int a, int b, int c, int d) {
        num1 = a;
        num2 = b;
        num3 = c;
        num4 = d;
    }

    // Function to calculate average
    float calculateAverage() {
        return (num1 + num2 + num3 + num4) / 4.0;
    }

    // Function to print the average
    void printAverage() {
        std::cout << calculateAverage() << std::endl;
    }

    // Destructor (optional for debugging)
    ~AverageCalculator() {
        std::cout << "Object deleted." << std::endl;
    }
};

int main() {
    // Creating an object dynamically
    AverageCalculator* avg
        = new AverageCalculator(10, 20, 30, 40);

    // Printing the average
    avg->printAverage();

    // Deallocating memory to prevent memory leaks
    delete avg;

    return 0;
}

```

Java

```

class AverageCalculator {

    private int num1, num2, num3, num4;

    // Constructor to initialize numbers
    public AverageCalculator(int a, int b, int c, int d) {
        this.num1 = a;
        this.num2 = b;
        this.num3 = c;
        this.num4 = d;
    }

    // Method to calculate average
    public double calculateAverage() {
        return (num1 + num2 + num3 + num4) / 4.0;
    }

    // Method to print the average
    public void printAverage() {
        System.out.println(calculateAverage());
    }
}

public class Main {

    public static void main(String[] args) {

        // Creating an object dynamically
        AverageCalculator avg
            = new AverageCalculator(10, 20, 30, 40);

        // Printing the average
        avg.printAverage();

        //Do NOT worry about managing memory! Automatic GC.
    }
}

```

Q5

JDK vs JVM vs JRE

JDK (Java Development Kit)

- What?: A full Java development package that includes everything needed to write, compile, and run Java programs.
- Includes:
 - JRE (Java Runtime Environment) – Needed to run Java applications.
 - JVM (Java Virtual Machine) – Executes Java bytecode.
 - Development Tools – Compiler (javac), debugger, and other utilities.

JRE (Java Runtime Environment)

- What?: A package that provides everything needed to run Java applications but does not include development tools.

- Includes:

- JVM (Java Virtual Machine) – Converts bytecode into machine code.

- Core Java Libraries – Essential classes (java.lang, java.util, etc.).

JVM (Java Virtual Machine)

- What?: A virtual machine that interprets or compiles Java bytecode into machine code for execution.

- Includes:

- Class Loader – Loads .class files into memory.

- Bytecode Interpreter – Converts bytecode into native machine code.

- JIT (Just in Time) Compiler – Optimizes performance by compiling frequently used bytecode into native machine code.

Comparison Table: JDK vs JRE vs JVM

Feature	JDK (Development Kit)	JRE (Runtime Environment)
Purpose	Develop & run Java applications	Run Java applications
Includes JRE?	Yes	Yes
Includes JVM?	Yes	Yes
Includes Compiler (javac)?	Yes	No
Best For	Java developers	End users running Java apps

Q6

Static vs Dynamic Programming Languages

What?

- How variables are checked in a language.

- Static: Checked at compile-time.

- Dynamic: Checked at runtime.

Comparison

Feature	Static Languages (Java, C)
Type Checking	Compile-time
Performance	Faster
Flexibility For Programmers	Less flexible
Error Detection	Early (Compile-time)
Code Safety	More strict

When? and How?

- When to use Static? Large, performance-critical apps.
- When to use Dynamic? Quick prototyping, scripting.
- How to choose?
 - Need speed and safety? Static
 - Need flexibility? Dynamic

Code

```
// Static Language (Java)
int number = "Hello"; // Error at compile-time
```

Code

```
# Dynamic Language (Python)
number = "Hello" # No error at assignment
number = 10      # Type can change
```

Code

```
// Static Language (Java)
int number = "Hello"; // Error at compile-time
```

Code

```
# Dynamic Language (Python)
number = "Hello" # No error at assignment
number = 10      # Type can change
```

Q7

How JVM is Reused by Other Programming Languages

- The JVM is NOT just for Java!
- Many languages compile to JVM bytecode:
Allows them to benefit from Java's ecosystem, portability, and performance optimizations
- Examples:
Kotlin : Designed as a better Java with more concise syntax.
Example:
Scala: Scala blends object-oriented and functional programming, while Java is primarily object-oriented (functional programming features were added later).
Example:
Groovy: Dynamic scripting language with Java-like syntax.
Example:
Others: Jython (Python on JVM), Clojure, ..

Scala, Kotlin and Groovy code can be compiled into JVM bytecode and executed by the JVM.

Code

```
fun main() {  
    val name = "Ranga"  
    println("Hello, $name!")  
}
```

Python

```
object HelloWorld {  
    def main(args: Array[String]): Unit = {  
        println("Hello from Scala!")  
    }  
}
```

Code

```
println "Hello from Groovy!"
```

Code

```
fun main() {  
    val name = "Ranga"  
    println("Hello, $name!")  
}
```

Code

```
fun main() {
    val name = "Ranga"
    println("Hello, $name!")
}
```

Python

```
object HelloWorld {
    def main(args: Array[String]): Unit = {
        println("Hello from Scala!")
    }
}
```

Python

```
object HelloWorld {
    def main(args: Array[String]): Unit = {
        println("Hello from Scala!")
    }
}
```

Code

```
println "Hello from Groovy!"
```

Code

```
println "Hello from Groovy!"
```

Q8

Advantages of Reusing JVM

Feature

Platform Independence

Memory Management

Performance

Java Interoperability

Q9

What is CLASSPATH?

- What?: CLASSPATH tells the JVM where to find .class files
- Why?: Helps Java locate user-defined classes and external dependencies.
- Default Behavior: If not set, Java looks in the current directory (.) and the standard Java libraries.
- Setting CLASSPATH:

Code

```
# Command Line
java -cp mylibrary.jar MyClass

#Windows
set CLASSPATH=C:\myapp\lib;

#Linux, Mac
export CLASSPATH=/home/user/myapp/lib
```

Q10

What is the role for a classloader in Java?

- What?: Finds .class files and loads them as and when they are needed
- Why?:
 - Efficient Memory Use: Loads classes only when needed, saving memory.
 - Security: Prevents unauthorized access by controlling class loading.
 - Custom Class Loading: Supports loading classes from databases, networks, or encrypted files.

Q11

Types of Java Class Loaders

1: Bootstrap ClassLoader

- Loads: Core Java classes from the java.base module.
- Example Packages:
 - java.lang (e.g., java.lang.Object, java.lang.String)
 - java.util (e.g., java.util.List, java.util.Map)
 - java.io (e.g., java.io.InputStream, java.io.OutputStream)
 - java.nio (e.g., java.nio.file.Path)
 - java.math (e.g., java.math.BigDecimal)
- Hierarchy: Root of the class loader hierarchy (has no parent).

2: Platform ClassLoader

- Loads: JDK-defined platform classes from specific modules (excluding java.base).
- Example Packages:
 - java.sql (e.g., java.sql.Driver, java.sql.Connection)
 - java.logging (e.g., java.util.logging.Logger)
 - java.xml (e.g., javax.xml.parsers.DocumentBuilderFactory)
 - java.net (e.g., java.net.HttpURLConnection)
- Hierarchy: Child of the Bootstrap ClassLoader.

3: System or Application ClassLoader

- Loads: Classes from the application's classpath.
- Location: Defined by the CLASSPATH environment variable or -cp/--class-path command-line argument.
- Example Packages:
 - Custom user-defined packages (com.in28minutes, org.example)
 - Libraries included in the classpath (springframework.*, hibernate.*, jackson.*)
- Hierarchy: Child of the Platform ClassLoader.

4: Custom ClassLoader (User-Defined)

- What?: Used to load classes from custom sources (network, databases, encrypted storage, etc.).
- How?: Extends ClassLoader and overrides findClass().
- Example:
- Example Use Case:
 - Dynamic class loading from .class files stored in a database.
 - Loading encrypted classes from a secured source.

Java

```
class MyClassLoader extends ClassLoader {
    @Override
    protected Class<?> findClass(String name) throws ClassNotFoundException {
        // Custom logic to load the class
        return super.findClass(name);
    }
}
```

Q12

ClassLoader Delegation Model (Parent-Child Hierarchy)

- Java uses the Parent Delegation Model to maintain class uniqueness and security.

1: Application ClassLoader receives a class request and first asks its parent (Platform ClassLoader).

2: Platform ClassLoader then asks the Bootstrap ClassLoader to load the class.

3: Bootstrap ClassLoader loads core Java classes (java.lang.String, java.util.List).

(REMEMBER) If the class is not found at any level, the request is passed back down, allowing the child ClassLoader to load it.

- (WHY?) This prevents duplicate class loading and ensures that system classes are not overridden.

Q13

How does the ClassLoader Delegation Model prevent system classes from getting overridden?

- If none of the parent class loaders find the class, only then does the Application ClassLoader attempt to load it from the classpath (.jar or .class files).
- System classes (java.lang.String, java.util.List, etc.) are always loaded by the Bootstrap ClassLoader.
- Even if a user defines java.lang.String in their code, it will not be loaded because the Bootstrap ClassLoader has already loaded the real system version.
- Example: The JVM does not allow defining classes in java.lang to prevent conflicts.

Result:

Java

```
// Illegal: System classes cannot be overridden
package java.lang;

public class String {
    public String customMethod() {
        return "This should not work!";
    }
}
```

Code

Error: The package java.lang is system-restricted.

Q14

Compilation vs Interpretation

Compilation

- What?: Translates the entire source code into machine code before execution.
- Execution Speed: Faster, as the program runs directly as machine code.
- Example Languages: C, C++, Java (compiles to bytecode).
- How?: Source Code → Compiler → Executable → Run

Interpretation

- What?: Translates and executes code line by line at runtime.
- Execution Speed: Slower, since it processes each instruction while running.
- Example Languages: Python, JavaScript, Ruby, Java (JVM interprets bytecode).
- How?: Source Code → Interpreter → Execute (line by line)

Q15

Is Java Compiled or Interpreted?

- Java is compiled into bytecode and then interpreted by the JVM
 Compiled: Java source code (.java) is compiled into bytecode (.class) using the javac compiler.
 Interpreted: The JVM interprets bytecode and executes it line by line.
- Process Flow
 Java Source Code (.java) → Compilation (javac) → Bytecode (.class)
 Bytecode (.class) → JVM Interprets & Executes

Q16

How does Java achieve Performance Optimization in spite of Interpretation?

- What? Java needs to balance portability with good performance.
- How? Provide good performance in spite of being interpreted!
 - Compile time optimizations: Small optimizations at compile time
 - Dead Code Elimination
 - Constant Folding
 - Runtime Optimizations: Java tries to optimize frequently executed code (hotspots) at runtime:
 - 90-10 Rule: 90% of execution time is spent on 10% of code. How about trying to identify hot spots and optimize them?
 - Just-In-Time (JIT) Compilation: Identify hot spots (frequently executed code), convert bytecode into native machine code and reuse it for future executions (avoid re-interpretation)
 - Adaptive Optimization: Dynamically optimize based on execution patterns (Method Inlining, ...).

Q17

What is Just-In-Time (JIT) Compilation?

- What?: JVM monitors method execution and compiles frequently used code paths into efficient machine code.

- Why?: Improves performance by avoiding repeated interpretation of "hot" spots

- Example:

Initially, compute() is interpreted by the JVM.

As it is called millions of times, the JIT compiler identifies it as a "hot spot".

The JVM compiles it into native machine code, improving performance.

Java

```
public class JITExample {
    public static void main(String[] args) {
        long start = System.nanoTime();
        for (int i = 0; i < 10_000_000; i++) {
            compute();
        }
        long end = System.nanoTime();
        System.out.println("Execution Time: "
            + (end - start) + " ns");
    }

    static void compute() {
        int x = 5 * 10;
    }
}
```

Java

```
public class JITExample {
    public static void main(String[] args) {
        long start = System.nanoTime();
        for (int i = 0; i < 10_000_000; i++) {
            compute();
        }
        long end = System.nanoTime();
        System.out.println("Execution Time: "
            + (end - start) + " ns");
    }

    static void compute() {
        int x = 5 * 10;
    }
}
```

Q18

Give a few examples of optimizations performed by Java?

1: Method Inlining

- What?: Replaces a method call with its actual body to avoid function call overhead.
- Why?: Reduces method call latency and improves execution speed.
- Example

Optimization in Action

Detects `add()` as small and frequently used.

Instead of making method calls, it inserts the logic directly into the loop.

This eliminates function call overhead and speeds up execution.

2: Loop Unrolling

- What?: Expands loop iterations to reduce loop control overhead (e.g., branching and index updates).

- Why?: Fewer iterations mean less CPU time wasted on loop conditions.

- Example

Optimization in Action

Loop may be re-written as:

This removes the loop condition check and improves performance.

3: Escape Analysis

- What?: Determines whether an object escapes the current method and allocates it on the stack instead of the heap if possible.

- Why?: Reduces garbage collection overhead and improves memory efficiency.

- Example

Optimization in Action

Since `numbers` is not used outside the method, the JVM allocates it on the stack instead of the heap.

Stack allocation is faster and avoids unnecessary garbage collection.

4: Dead Code Elimination (Compile time)

- What?: Remove code that has no effect on the final output.
- Why?: Reduces unnecessary computations, improving performance.
- Example

5: Constant Folding (Compile time)

- What?: Evaluate constant expressions at compile time instead of runtime.
- Why?: Reduces redundant computations during execution.
- Example

Optimization in Action

50 * 2 is replaced with 100, avoiding multiplication at runtime.

Java

```
public class MethodInliningExample {
    public static void main(String[] args) {
        long start = System.nanoTime();
        for (int i = 0; i < 10_000_000; i++) {
            // JVM may inline this method
            System.out.println(add(i, i+1));
        }
        long end = System.nanoTime();
        System.out.println("Execution Time: "
                           + (end - start) + " ns");
    }

    static int add(int a, int b) {
        // Small method, likely to be inlined
        return a + b;
    }
}
```

Java

```
public class LoopUnrollingExample {
    public static void main(String[] args) {
        int sum = 0;
        // Instead of iterating 4 times
        // JVM may unroll this into direct additions
        for (int i = 0; i < 4; i++) {
            sum += i;
        }
        System.out.println(sum);
    }
}
```

Code

```
sum += 0;
sum += 1;
sum += 2;
sum += 3;
```

Code

```
sum += 0;
sum += 1;
sum += 2;
sum += 3;
```

Code

```
sum += 0;
sum += 1;
sum += 2;
sum += 3;
```

Java

```
import java.util.ArrayList;
import java.util.List;

public class EscapeAnalysisExample {
    public static void main(String[] args) {
        for (int i = 0; i < 1_000_000; i++) {
            // Likely optimized away by Escape Analysis
            List<Integer> numbers = createList();
        }

        static List<Integer> createList() {
            List<Integer> list = new ArrayList<>();
            list.add(1);
            list.add(2);
            list.add(3);
            return list;
        }
    }
}
```

Java

```
public class DeadCodeExample {
    public static void main(String[] args) {
        int x = 10;
        int y = 20;

        // This result is never used,
        // Remove it
        int z = x + y;

        System.out.println(x * y);
    }
}
```

Java

```
public class ConstantFoldingExample {
    public static void main(String[] args) {
        // Replaces this with '100'
        int x = 50 * 2;
        System.out.println(x);
    }
}
```

Q19

Why does Java NOT do all optimizations at compile time?

- What?: Java delays optimizations to make smart decisions at runtime!
- Some optimizations happen at compile time: Dead Code Elimination, Constant Folding, ..
- Most optimizations happen at runtime (JVM JIT Compiler): Escape Analysis, Loop

Unrolling, Method Inlining..

- Why?

90-10 Rule: Java optimizes only frequently used parts to avoid unnecessary overhead

Cross-Platform Execution: Compile time optimizations might not be efficient on all platforms.

Preserving Runtime Information: Optimizing too early may remove valuable runtime insights.

CHAPTER 2

Java Landscape

12 Questions & Answers

Q1

Who owns Java?

- 1995: Sun Microsystems created Java.
- 2009: Oracle Corporation acquired Sun Microsystems, gaining control over Java.
Oracle continues to develop and release new Java versions.

Q2

Java SE vs Java EE vs Java ME

Java SE (Standard Edition)

- Core Java platform with fundamental libraries, JDK, JRE and JVM
- Core Libraries: Includes java.lang, java.util, java.io, and java.net
- Database Connectivity: Uses JDBC for interacting with databases
- Earlier called J2SE

Java EE (Enterprise Edition) Now Jakarta EE

- Built on Java SE to support enterprise-grade applications
- Common Use Cases: Large-scale applications, APIs, and cloud services
 - Servlets & JSP: Enables dynamic web applications
 - JPA (Java Persistence API): Supports database ORM (Object-Relational Mapping)
 - Web Services: Provides REST (JAX-RS) and SOAP (JAX-WS) support
- Renamed: J2EE > Java EE > Jakarta EE

Java EE (Enterprise Edition) was rebranded as Jakarta EE after being transferred from Oracle to the Eclipse Foundation in 2017

Java ME (Micro Edition)

- Lightweight Java version for mobile, embedded systems, and IoT devices
- Optimized for Low-Memory Devices: Used in feature phones, smart cards etc.
- Limited API Support: Subset of Java SE, designed for resource-constrained environments

Q3

How has Java evolved over the last few decades?

Embedded Systems (1990s)

- Early Purpose: Developed as a platform-independent language for embedded systems
- Oak to Java: Originally designed for interactive televisions, rebranded as Java in 1995 for general computing

Web Applications (Early 2000s)

- Adoption in Web Development: Became widely used with Java EE (now Jakarta EE)
- Dynamic Web Pages: Servlets and JSP enabled interactive and data-driven web applications
- Framework Evolution: Spring and Hibernate simplified development and improved scalability

Mobile Development (Mid-2000s)

- Java in Mobile: Became the dominant language for mobile applications
- Android Boom: In 2008, Google adopted Java as the primary Android language
- Rise of Kotlin: Introduced in 2017 as an official Android language, but Java remains widely used

Microservices & Cloud (2010s–Present)

- Shift to Cloud-Native: Java adapted to cloud-based and microservices architectures
- Spring Boot Revolution: Improved modularity and scalability for microservices
- Containerization: Java applications now run efficiently on Docker, Kubernetes, and cloud platforms

Q4

What is a JAR file?

Overview

- What?: Single package containing .class files, images, configuration files, and libraries in one archive.
- Why?: Makes Java applications easy to package, distribute, and run

Why Use JAR Files?

Feature
Easy Distribution
Compression
Executable
Create Sharable Libraries

Creating and running a JAR file

Manifest File for an Executable JAR: META-INF/MANIFEST.MF

Code

```
jar cf MyApp.jar *.class #Create a JAR File
java -jar MyApp.jar #Run it (Needs `MANIFEST.MF`)
```

Code

```
Main-Class: com.example.MainApp
```

Q5

Oracle JDK vs Amazon Corretto vs OpenJDK vs GraalVM

OpenJDK

- Purpose: Open-source reference implementation of Java
- Maintenance: Managed by the Java community and major vendors (IBM, Apple..)
- Support: Regular updates but no built-in commercial support
- Best For: Developers and organizations preferring open-source solutions

Oracle JDK

- Purpose: Official Java distribution from Oracle
- Features: Premium support and security updates
- Best For: Enterprises needing long-term support

Amazon Corretto

- Purpose: Free, OpenJDK-based distribution from Amazon
- Optimization: Designed for AWS and cloud environments
- Support: Includes long-term support and performance enhancements
- Best For: Businesses on AWS seeking cost-effective Java solutions

GraalVM

- Purpose: An advanced JDK with ahead-of-time (AOT) Native Image compilation
- Low Resource Usage: Native executables use only a fraction of memory and CPU resources required by a JVM, which improves utilization and reduces costs.
- Fast Startup: Requires no warmup to run at peak performance
- Compact Packaging: Native executables are small and easy to deploy in minimal container images
- Support:
 - Community Edition – Free, open-source, maintained by Oracle
 - Enterprise Edition – Paid support with advanced optimizations and security updates from Oracle
- Best For:
 - Microservices and serverless computing where fast startup time is critical
 - Cloud-native applications with low memory footprint

Q6

What is JCP?

JCP ensures Java evolves through an open, community-driven process, with JSRs defining major new features.

- JCP (Java Community Process)
 - Definition: The official process for defining and evolving Java standards
 - Participants: Oracle, corporations, developers, and Java experts
- How It Works
 - Proposals: New Java platform features are proposed as JSRs (Java Specification Requests)
 - Review & Voting: The Executive Committee reviews, provides feedback, and votes
 - Reference Implementation: A working implementation is developed and tested
- Example: JSR 330 (Dependency Injection for Java)
 - Purpose: Standardized annotations like `@Inject` for dependency injection
 - Approval: Passed through the JCP and became part of Java EE 6
 - Impact: Unified dependency injection across different frameworks like Spring and Jakarta EE

Q7

JSR vs JEP

JEPs handle JDK enhancements, while JSRs define major Java standards that require JCP approval.

JEP (JDK Enhancement Proposal)

- Purpose: Proposes and tracks changes to the JDK (Java Development Kit)
- Scope: Used for incremental improvements, like language enhancements and performance optimizations
- Process: Proposed by developers, reviewed internally by the OpenJDK team
- Approval: Requires review but does not need JCP approval
- Example:
 - JEP 378: Text Blocks
 - JEP 359: Records
 - JEP 436: Virtual Threads (Project Loom)

JSR (Java Specification Request)

- Purpose: Defines new Java standards, including APIs, libraries, and frameworks
- Scope: Used for major changes that require community consensus, such as Java EE (Jakarta EE) specifications
- Process: Goes through the JCP (Java Community Process) for approval
- Approval: Voted on by the JCP Executive Committee
- Example:
 - JSR 330: Standardized Dependency Injection annotations (@Inject)
 - JSR 370: JAX-RS 2.1 (RESTful Web Services)

Q8 Java Release Numbering

Currently - a sequential numbering system like Java 8, Java 9, Java 10,..., Java 21,..., Java 27, ...

Q9 Did Java Release Numbering follow a different standard earlier?

Pre-2017 Versioning (Major.Minor Format)

- Java versions followed a Major.Minor format, such as Java 1.2, Java 1.4, Java 1.5.
- Confusing Naming: Java 1.5 was also called Java 5, creating inconsistency.

Post-2017 Versioning (Simplified Integer Format)

- **Format Change:** Versioning switched to a simple integer format (Java 9, Java 10, Java 11, etc.).

Why the Change?

- **Simplifies Naming:** No more confusion between "Java 1.8" and "Java 8"—just Java 8, Java 9, Java 10, etc.

Q10

When are new Java versions released?

Pre-2017 Release Schedule

- **Irregular Releases:** Java versions were released every few years, with long gaps between major versions
- **Notable Versions:** Java 5 (2004), Java 6 (2006), Java 7 (2011), Java 8 (2014)
- **Unpredictable Upgrades:** Developers had to wait several years for new features and improvements

Post-2017 Release Schedule

- **Six-Month Cycle:** Starting with Java 9 (2017), new versions were released every six months (March & September).
- **Predictable Updates:** Developers now get regular feature releases without long delays.
- **LTS Every Two (or Three) Years:** Java 11 (2018), Java 17 (2021), Java 21 (2023), Java 25 (2025)

Why the Change?

- **Faster Innovation:** Shorter cycles allow Java to keep up with modern programming needs.
- **Less Migration Pain:** Regular, incremental updates reduce the burden of upgrading from older versions.
- **Clear Roadmap:** Developers and businesses can plan ahead for new features and LTS versions.

Q11

Long-Term Support (LTS) vs Non-LTS Releases

Long-Term Support (LTS) Releases

- Definition: LTS versions receive extended security updates and commercial support for a number of years
- Major LTS Versions: Java 8 (2014), Java 11 (2018), Java 17 (2021), Java 21 (2023), Java 25 (2025),
- Support Duration: Oracle and other vendors provide LTS support for at least 8 years.
- Best For: Enterprises and production environments needing stability and long-term maintenance.

Non-LTS Releases

- Definition: Short-term versions released every six months with new features.
- Examples: Java 22, 23, 24, 26, 27,
- Support Duration: Typically six months, until the next feature release.
- Best For: Developers who want early access to new features.

Q12

Why Java is Still Popular?

Java remains a top choice because it is stable, scalable, secure, and works everywhere—making it ideal for enterprise, web, and cloud applications.

Feature	Why It Matters?
Platform Independence	No OS-specific code needed
Strong Ecosystem	Frameworks for every use case
Performance	Faster than most interpreted languages
Memory Management	No manual malloc/free
Security	No direct memory access, strong encryption
Community Support	Millions of developers and active support
Scalability	Powers large-scale cloud apps
Backward Compatibility	Avoids costly rewrites

CHAPTER 3

Java Memory Management

13 Questions & Answers

Q1

How is Java different from C & C++ in terms of memory management?

Java handles memory automatically, while C (malloc/free) and C++ (new/delete) require the programmer to allocate and release memory.

1: Manual Memory Management in C

- What?: The programmer allocates and deallocates memory.
- Risk: Memory leaks occur if free() is not called.
- Example (C: Manual Allocation and Deallocation)

2: Manual Memory Management in C++

- What?: Uses new and delete for allocation/deallocation.
- Risk: If delete is forgotten, memory leaks occur.
- Example (C++: Using new/delete)
- Later Versions of C++: Introduced smart pointers (std::unique_ptr, std::shared_ptr) to manage memory automatically.

3: Automatic Garbage Collection in Java

- What?: Java manages memory automatically, reclaiming unused objects.
- How?: The Garbage Collector (GC) runs periodically to free up memory.
- Example (Java: No Manual Freeing Needed)

4: Summary

Feature	C (Manual)	C++ (Manual)
Allocation	malloc()	new()
Deallocation	free()	delete
Risk	Memory leaks if free() is forgotten	Memory leaks if delete is forgotten
Performance	Fast	Fast

Code

```
// Allocate memory
int *ptr = (int*) malloc(sizeof(int));

*ptr = 10;

printf("Value: %d\n", *ptr);

// Free memory to prevent leaks
free(ptr);
```

Code

```
int* ptr = new int(10); // Allocate memory

std::cout << "Value: " << *ptr << std::endl;

delete ptr; // Manually free memory
```

Java

```
// Memory allocated
Demo demo = new Demo(10);

System.out.println("Demo: " + demo);

// No need to manually free memory
// Once there are no references to an object
// GC will free the memory
```

Q2

What is a Memory Leak?

Overview

- Memory leak: A program fails to release unused memory
- Causes increased memory usage over time
- Can eventually lead to OutOfMemoryError and slow performance

Why Are Memory Leaks Rare in Java?

- Garbage Collection (GC): The JVM automatically reclaims unused objects, reducing leaks.
- (Remember) Memory leaks still occur if objects are unintentionally kept in memory

Examples of Memory Leaks in Java

1: Static Variables Holding References

- What?: Static variables persist throughout the program, preventing garbage collection.

2: Un-closed Resources (File, Database, Network)

- What?: Resources like file streams stay open, causing memory leaks.
- Fix: Use try-with-resources to auto-close resources.

Code

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Allocate memory
    int *ptr = (int*) malloc(sizeof(int));

    *ptr = 10;

    printf("Value: %d\n", *ptr);

    //Programmer forgot to free memory
    //free(ptr);
    return 0;
}
```

Code

```
private static List<Something> millionObjects
    = new ArrayList<>();

//Code to Populate millionObjects with objects

//Objects in millionObjects persist until end of program
//unless you write code to remove objects from the list!
```

Java

```
import java.io.*;

public class FileLeakExample {
    public static void main(String[] args)
        throws IOException {
        FileInputStream file
            = new FileInputStream("data.txt");

        // No file.close() Memory leak
    }
}
```

Code

```
try (FileInputStream file = new FileInputStream("data.txt")) {
    // Auto-closed at end of block
}
```

Q3**How does Garbage Collection work?**

What?

- Garbage Collection: Process where Java automatically removes un-referenced objects from memory.

How?

- If an object has no references, it is unreachable => Object eligible for Garbage Collection (GC).
- Example:

When?

- The JVM decides when to run GC based on memory usage. (There are many algorithms!)
- You can request GC using `System.gc()`
But it is not guaranteed to run immediately

Java

```
String str = new String("Hello");  
System.out.println(str);  
  
str = null;  
// Object is now unreachable Eligible for GC  
  
//Other code
```

Q4**Memory Spaces in Java**

Java efficiently manages memory by using

- Stack for method calls & local variables
- Heap for objects
- Metaspace for class metadata and
- Code Cache for JIT-compiled machine code

1: Stack Memory

- What? Stores method calls & local variables.
- When? Exists only during method execution.
- Where? Each thread has its own stack.
- Code Example

New stack frame is created when add(a, b) is called.

Stack frame is removed when add() finishes, freeing memory.

2: Heap Memory

- What? Stores objects.
- Why? Needed for objects where object lifespan is unknown.
- Code Example

p1 and p2 references are in Stack, but objects exist in Heap.

Objects persist even after the method call ends, until GC removes them.

3: Metaspace (Replaces PermGen)

- What? Stores class metadata
 - Stores the class structure (including the constant pool, field, and method data).
 - Stores method and constructor code, including static methods.
- When? Holds data until the class is unloaded from JVM.

4: Code Cache

- What? Stores JIT-compiled machine code for faster execution.
- Why? Reduces repeated bytecode interpretation, improving performance.
- How? Managed by the JVM's Just-In-Time (JIT) compiler.

Summary

- Stack Stores local variables & method calls, cleared when methods exit.
- Heap Stores objects, cleaned by Garbage Collector.
- Metaspace Holds class metadata.
- Code Cache Stores JIT-compiled machine code. Reduces repeated bytecode interpretation.

```

public class StackExample {
    public static void main(String[] args) {
        int a = 10; // Stored in Stack
        int b = 20; // Stored in Stack
        add(a, b); // Creates new Stack Frame
    }

    static int add(int x, int y) {
        // Parameters, Local variables stored in Stack
        int sum = x + y;
        return sum;
    }
}

```

Code

```

// `p1` in Stack, object in Heap
Person p1 = new Person("Alice");

// Another object in Heap
Person p2 = new Person("Bob");

// A very long program follows!
// A very long program follows!
// A very long program follows!

```

Q5

Comparison: Stack vs Heap Memory

Feature	Stack
Stores	Local variables, method calls
Lifespan	Cleared after method exits
Memory Management	Automatic
Thread Scope	Each thread has its own stack

Q6

Why is GC Complex?

Steps in Garbage Collection

- Step 1: Marking Finds objects still in use.
- Step 2: Sweeping Removes unused objects from memory.
- Step 3: Compacting Rearranges memory to avoid fragmentation.

Why is it Complex?

Task

Finding Unused Objects

Minimizing Application Pauses

Avoiding Memory Fragmentation

Providing Flexibility

Q7

Reference Counting vs Mark & Sweep

1: Reference Counting

What?

- Each object has a reference counter: Tracks how many references point to it
- When the count reaches zero, the object is eligible for garbage collection.

Why?

- Simple and fast
- Memory can be freed immediately when no references exist

Issue?

- Fails with circular references Two objects referencing each other will never be collected.

Example:

- Problem: Even though objA and objB are unreachable, their reference count is still 1, preventing GC.
- Solution: Java does not use Reference Counting

2: Mark & Sweep

What?

- JVM scans the heap and marks objects that are still reachable.
- Unmarked objects are deleted.

Why?

- Avoids circular reference issues.

How?

- STEP 1: Mark Phase – JVM starts from root references and marks all reachable objects
 - Active Thread Stack Local variables in method calls.
 - Static Variables Class-level variables (static int x).
 - JNI References Objects referenced from native C/C++ code.
- STEP 2: Sweep Phase – JVM removes unmarked (unused) objects, reclaiming memory.

Comparison: Reference Counting vs Mark & Sweep

Feature	Reference Counting
Concept	Tracks the number of references to each object
Performance	Fast, immediate cleanup
Handles Circular References?	Fails
Used in Java?	No

Code

```
class A {
    B b;
}

class B {
    A a;
}

//Assume this code in a method

A a = new A(); // objA on Heap refCount = 1
B b = new B(); // objB on Heap refCount = 1

a.b = b; // objB on Heap refCount increases to 2
b.a = a; // objA on Heap refCount increases to 2

a = null; // objA on Heap refCount decreases to 1
b = null; // objB on Heap refCount decreases to 1

// Even though objA and objB are unreachable,
//their refCount is still 1.

//Lot of other code follows
```


Overview

- What? Java's heap memory is divided into generations – Young and Old.
- Approach: Frequent garbage collection for short-lived objects (Young Generation) and less frequent collection for long-lived objects (Old Generation).
- Why? To improve Garbage Collection (GC) efficiency.

Generations in Java

- Young Generation: Stores new objects.
- Old Generation: Stores long-lived objects.

Garbage Collection Cycles

- Minor GC Cycle: Cleans Young Generation (runs frequently).
- Major GC (Full GC) Cycle: Cleans Old Generation (runs less often).

Object Lifecycle in Generations

- Objects are created in the Young Generation.
- Minor GC cycles frequently clean up short-lived objects.
- Surviving objects move to the Old Generation after multiple Minor GC cycles.
- Major GC cycle runs when the Old Generation fills up, cleaning long-lived objects.

Why Is This Efficient?

- Separates objects based on their likelihood of removal, optimizing GC effort.
- Most objects are short-lived, so frequent Young Generation GC quickly reclaims memory.
- Old Generation objects persist longer, making frequent GC on them inefficient.

NOTE

- The Permanent Generation (PermGen) was a memory area in the Java Virtual Machine (JVM) used to store class metadata.
- Replaced by Metaspace in Java 8, which does NOT use heap memory.

- STW GC pauses all application threads for garbage collection.
- No user code runs during this time.
- Ensures memory safety but affects performance and response time.
- When Does Stop-the-World GC Occur?

Minor GC: Brief pause for Young Generation cleanup.

Major GC (Full GC): Longer pause for Old Generation cleanup.

New GC methods: Aim to reduce or eliminate STW pauses.

Q10

Types of Garbage Collectors in Java

1: Serial GC

- What? A simple, single-threaded garbage collector.
- Why? Best for small applications with low memory requirements.
- When? Typically used when JVM detects a single-core CPU
- How? Performs Stop-the-World (STW) GC
- Where? Best for small desktop applications.
- Example: Enable Serial GC

2: Parallel GC (Throughput Collector)

- What? Uses multiple threads for garbage collection.
- When? Suitable for multi-threaded applications.
- How? Runs Minor GC (Young Gen) in parallel, reducing collection time.
- Where? Ideal for medium-sized applications with multi-core processors.
- Example: Enable Parallel GC

3: G1 GC (Garbage First GC)

- Why? Reduces long GC pauses by prioritizing regions with most garbage.
- When? Best for apps needing low-latency performance.
- How? Divides heap into fixed-size regions and collects garbage concurrently.

Choose when response time is important

More granular memory management – Instead of treating the heap as only "Young vs. Old," G1 GC manages memory in smaller, more flexible regions

Better efficiency – Prioritizes garbage collection based on garbage density

- Example: Enable G1 GC

4: ZGC (Z Garbage Collector)

- What? Fully concurrent GC avoids full application freezes, improving response time.
- Why? Best for applications needing near real-time performance.
- When? If response time is a high priority and the heap size is large (multi-terabyte scale)
- How? Performs almost all GC work concurrently, avoiding long pauses.
- Example: Enable ZGC

Comparison : Java Garbage Collectors

GC Type	Best For	Pause Time	Threads
Serial GC	Small applications	High (Stop-the-World)	Single-threaded
Parallel GC	Medium-sized apps	Medium	Multi-threaded
G1 GC	Large apps needing low latency	Low	Multi-threaded
ZGC	Large-scale, real-time apps	Very Low	Multi-threaded

Key Difference in One Line

- Serial GC: "I'll stop everything and clean up memory in one go"
- Parallel GC: "I'll use multiple threads to clean up faster, but I still need to pause everything a number of times"
- G1 GC: "I'll clean up memory little by little, but I still need to stop you a few times."
- ZGC: "I'll clean up memory silently in the background, and you won't even notice!"

Code

```
java -XX:+UseSerialGC MyApp
```

Code

```
java -XX:+UseParallelGC MyApp
```

Code

```
java -XX:+UseG1GC MyApp
```

Code

```
java -XX:+UseZGC MyApp
```

Q11**Customization Options for Garbage Collection**

Option	Description
-XX:+UseSerialGC	Enables Serial GC (single-threaded, best for small apps)
-XX:+UseParallelGC	Enables Parallel GC (multi-threaded, good for high throughput)
-XX:+UseG1GC	Enables G1 GC (low-latency, region-based GC)
-XX:+UseZGC	Enables ZGC (fully concurrent, ultra-low pause time)
-Xms<size>	Sets initial heap size
-Xmx<size>	Sets maximum heap size
-XX:+PrintGCDetails	Prints detailed GC logs

Q12**Why is finalize() Method Deprecated in Java 9?**

What?

- `finalize()`: A method called before an object is garbage collected to perform cleanup tasks
- Why Deprecated?: Unreliable, slow, and can cause security risks

Problems with `finalize()`:

- Unpredictable Execution – The JVM does not guarantee when or if `finalize()` will run.
- Performance Overhead – Objects with `finalize()` require extra GC cycles, delaying memory cleanup.
- Security Risks – Bad code can cause security risks.
For example: Objects can be revived inside `finalize()`, preventing garbage collection.
If an object assigns this to a static field or another reachable reference inside `finalize()`, it becomes reachable again and won't be garbage collected.

Java is always "Call by Value" – for primitives, it passes values, and for objects, it passes reference copies

Call by Value

- What? A copy of the value is passed to the method.
- The original variable remains unchanged after the method call.
- Example:

Passing Objects in Java: Also Call by Value

- What? When an object is passed, the reference is copied, not the actual object.
- Why? The reference itself cannot be changed, but the object's internal values can be modified.
- Example:

Java

```
public class CallByValueExample {
    static void modify(int num) {
        num = num + 10;
    }

    public static void main(String[] args) {
        int x = 20;
        modify(x);
        System.out.println(x); // Output: 20 (unchanged)
    }
}
```

Java

```
class Person {
    String name;
}

public class CallByReferenceExample {
    static void modify(Person p) {
        p.name = "Alice"; // Modifies the object

        //p = new Person();
        //Does
    }

    public static void main(String[] args) {
        Person person = new Person();
        person.name = "Bob";
        modify(person);
        System.out.println(person.name);
        // Output: Alice (object modified)
    }
}
```

CHAPTER 4

Java Object Oriented Programming Concepts

24 Questions & Answers

Q1

Structured vs Object Oriented Programming

1: Structured Programming

- A procedural approach that focuses on functions (procedures) to break down a problem into steps.
- Control flow is managed using function calls.
- Example: Taking a Flight (Structured Programming)

2: Object-Oriented Programming (OOP)

- Focuses on objects rather than procedures.
- Objects combine data (state) and methods (behavior).
- Encapsulation, inheritance, and polymorphism make code reusable and modular.
- Example: Taking a Flight (OOP Approach)

3: Key Differences

Feature	Structured Programming
Focus	Functions (procedures)
Data Handling	Stored separately, functions operate on it
Reusability	Code is reused via functions
Scalability	Harder to scale for large apps
Example Languages	C, Pascal, ..

[Code](#)


```
#include <stdio.h>

void travelToAirport() {
    printf("Traveling to the airport...\n");
}

void checkIn() {
    printf("Checking in...\n");
}

void boardFlight() {
    printf("Boarding the flight...\n");
}

int main() {
    travelToAirport();
    checkIn();
    boardFlight();
    return 0;
}
```

Java

```

// Main class to execute the program
public class FlightProcess {
    public static void main(String[] args) {
        Passenger passenger1 = new Passenger("Alice");

        Car taxiCar = new Car("Toyota Camry");

        TaxiService taxiService = new TaxiService();
        taxiService.driveToAirport(passenger1, taxiCar);

        AirportCheckInDesk checkInDesk
            = new AirportCheckInDesk();

        Flight flight = new Flight("AI-202");
        checkInDesk.checkIn(passenger1, flight);
        passenger1.boardFlight(flight);
    }
}

// Represents a Car that a taxi service uses
class Car {
    private String model;

    Car(String model) {
        this.model = model;
    }

    public String getModel() {
        return model;
    }
}

// Represents a Flight
class Flight {
    private String flightNumber;

    Flight(String flightNumber) {
        this.flightNumber = flightNumber;
    }

    public String getFlightNumber() {
        return flightNumber;
    }
}

// Represents a Passenger
class Passenger {
    private String name;

    Passenger(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    // Passenger now initiates boarding
    void boardFlight(Flight flight) {
        System.out.println(name +
            " is boarding flight " +
            flight.getFlightNumber());
    }
}

// Handles taxi services
class TaxiService {

```

```

        void driveToAirport(Passenger passenger, Car car) {
            System.out.println(passenger.getName()
                + " is traveling to the airport in a "
                + car.getModel());
        }
    }

    // Handles check-in at the airport
    class AirportCheckInDesk {
        void checkIn(Passenger passenger, Flight flight) {
            System.out.println(passenger.getName() +
                " is checking in to the flight" +
                flight.getFlightNumber());
        }
    }
}

```

Q2

Class vs Object vs State vs Behavior

What is a Class?

- A blueprint or template that defines what an object should be.
- Specifies attributes (state) and methods (behavior).

What is an Object?

- An instance of a class, created using the new keyword.

What is State?

- The data stored inside an object (attributes or instance variables).
- Each object can have different values for its state.
- State of an object changes with time.

What is Behavior?

- The actions an object can perform, defined as methods in a class.

Summary: Class vs Object vs State vs Behavior

Concept	Definition
Class	A blueprint defining state & behavior
Object	An instance of a class
State	Data inside an object
Behavior	Actions an object performs

Java

```
public class CricketScorer {
    // State (instance variable)
    private int score;

    // Behavior (methods)
    public void addFour() {
        score += 4; // Increment score by 4
    }

    public void addSix() {
        score += 6; // Increment score by 6
    }

    public int getScore() {
        return score; // Return current score
    }
}
```

Java

```
public class Match {
    public static void main(String[] args) {
        CricketScorer scorer1
            = new CricketScorer(); // Object 1
        CricketScorer scorer2
            = new CricketScorer(); // Object 2

        //Objects `scorer1` and `scorer2` are instances
        //with their own scores.
    }
}
```

Java

```

public class Match {
    public static void main(String[] args) {
        CricketScorer scorer1 = new CricketScorer();
        CricketScorer scorer2 = new CricketScorer();

        scorer1.addFour(); // scorer1's score = 4
        scorer2.addSix(); // scorer2's score = 6

        System.out.println("Scorer1 Score: "
            + scorer1.getScore()); // 4
        System.out.println("Scorer2 Score: "
            + scorer2.getScore()); // 6

        //Each object maintains its own `score`,
        //demonstrating unique state.
    }
}

```

Java

```

public class Match {
    public static void main(String[] args) {
        CricketScorer scorer = new CricketScorer();

        // Behavior: Increase score by 4
        scorer.addFour();

        // Behavior: Increase score by 6
        scorer.addSix();

        System.out.println("Total Score: "
            + scorer.getScore()); // 10
    }
}

```

Q3

Object Lifecycle

Stage	Description
1: Object Creation	An object is created using new, calling the constructor
2: Object in Use	Methods and fields are accessed
3: Object Becomes Unreachable	No references point to the object, making it eligible for GC
4: Garbage Collection	JVM removes the object from memory when needed
5: Finalization (Deprecated in Java 9)	finalize() runs before the object is destroyed

Q4**Why is Java NOT 100% Object-Oriented?**

100% Object-Oriented Language

- A 100% object-oriented language is one where everything is an object:
Including primitive data types and operations.
- Example: Smalltalk, Ruby

Why Java is NOT Fully Object-Oriented?

Reason	Explanation
Primitive Data Types	Java has primitive types (int, char, double, etc.), which are not objects.
Operators Are Not Methods	Operators like +, -, *, and / do not invoke methods, unlike pure OOP languages.
Static Methods	Methods declared as static belong to the class instead of an object.
static Variables	static variables exist independently of objects.

Q5**Inheritance vs Abstract Class vs Interface**

Overview

- Inheritance: Allows a subclass to inherit properties and methods from a parent class.
- Abstract Class: Defines a common template with both implemented and unimplemented (abstract) methods.
- Interface: Defines a contract (an interface) that multiple classes can implement.

Inheritance

- What? Allows a subclass to reuse code from a parent class.
- Why? Reduces duplication and models real-world relationships.
- How? Uses the extends keyword to inherit from a class.
- Limitations: Tight coupling – changes in the parent class affect all subclasses.

Abstract Class

- What? A partially implemented class that contains both abstract and concrete methods.
- Why? Provides a common template for related classes while allowing specific implementations.
- How? Uses the abstract keyword and defines abstract methods for subclasses to implement.

Interface

- What? A contract that multiple classes can implement without sharing implementation details.
- Why? Allows multiple classes (even unrelated) to share common behavior.
- How? Uses the interface keyword and requires implementing classes to define all methods.

Comparison

Feature	Inheritance (extends)	Abstract Class
Definition	A subclass inherits methods from a parent class	A class with abstract and concrete methods
Supports Multiple Inheritance?	No	No
Contains Abstract Methods?	No	Yes
Contains Concrete Methods?	Yes	Yes
Can Have Instance Variables?	Yes	Yes
Can Have Constructors?	Yes	Yes
Best Use Case	When a class is a specialized version (IS-A) of another class	When multiple subclasses share common behavior but need some customization in specific implementations

When to Use What?

- Use Inheritance When there is a parent-child relationship (e.g., Animal Dog).
- Use Abstract Class When multiple sub-classes share common behavior but need some customization in specific implementations (e.g., AbstractRecipe Recipe1).
- Use Interface When multiple classes (even unrelated) must follow the same contract (e.g., Flyable Bird, Aeroplane).

Java

```
public class Animal {
    public void eat() {
        System.out.println("This animal eats food.");
    }
}

public class Dog extends Animal {
    public void bark() {
        System.out.println("Dog barks.");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.eat(); // Inherited from Animal
        dog.bark();
    }
}
```

Java


```

public abstract class AbstractRecipe {
    public void execute() {
        getReady();
        doTheDish();
        cleanup();
    }

    public abstract void getReady();
    public abstract void doTheDish();
    public abstract void cleanup();
}

public class Recipe1 extends AbstractRecipe {
    @Override
    public void getReady() {
        System.out.println("Get the raw materials");
        System.out.println("Get the utensils");
    }

    @Override
    public void doTheDish() {
        System.out.println("Cook the dish");
    }

    @Override
    public void cleanup() {
        System.out.println("Cleanup the utensils");
    }
}

public class Main {
    public static void main(String[] args) {
        AbstractRecipe recipe = new Recipe1();
        recipe.execute();
    }
}

```

Java

```

interface Flyable {
    void fly();
}

class Aeroplane implements Flyable {
    @Override
    public void fly() {
        System.out.println(
            "Aeroplane flies using engines.");
    }
}

class Bird implements Flyable {
    @Override
    public void fly() {
        System.out.println(
            "Bird flies by flapping wings.");
    }
}

public class Main {
    public static void main(String[] args) {
        Flyable plane = new Aeroplane();
        plane.fly();

        Flyable sparrow = new Bird();
        sparrow.fly();
    }
}

```

Design Choices in Java Collections: Interface, Inheritance, and Abstract Classes

Overview

- Java Collections Framework is designed to be flexible, reusable, and extendable.
- Follows a hierarchical structure using interfaces, abstract classes, and concrete implementations.

Interfaces Define common behavior (List, Set, Queue, ..).

Abstract Classes Provide initial implementations (AbstractList, AbstractSet, ..).

Concrete Classes Implement specific data structures (ArrayList, HashSet, ...).

Example: List Interface in JDK

Example: AbstractList in JDK

Example: ArrayList Extending AbstractList

Code

```
public interface List<E> extends Collection<E> {
    boolean add(E e);
    E get(int index);
    boolean remove(Object o);
    //Others
}
```

Code

```
//Provides partial implementation, allowing customization
public abstract class AbstractList<E>
    extends AbstractCollection<E>
    implements List<E> {

    public boolean add(E e) {
        add(size(), e); // Default add() method
        return true;
    }

    public E remove(int index) {
        throw new UnsupportedOperationException();
        // Must be overridden
    }
    //Others
}
```

Java

```

public class ArrayList<E> extends AbstractList<E>
    implements List<E>, RandomAccess, Cloneable {

    private transient Object[] elementData;

    public E get(int index) {
        return (E) elementData[index];
    }

    public boolean add(E e) {
        ensureCapacityInternal(size + 1);
        elementData[size++] = e;
        return true;
    }

}

```

Q7

What are Design Patterns?

What are Design Patterns?

- What? Reusable solutions for common coding problems.
- Why? Makes code easier to understand, change, and reuse.
- Where? Used in big projects, frameworks, and everyday coding.
- How? By applying proven patterns like Singleton, Strategy, Factory, and Observer.

Example 1: Singleton Pattern

- What? One object shared across the app.
- When? Need only one instance (e.g., Logger, Config).
- How? Private constructor + static instance + lazy creation.
- Key Benefit: Saves memory, consistent behavior.
- Implementation

Example 2: Strategy Pattern

- What? Swap different algorithms at runtime.
- When? Want to choose behavior dynamically (Bubble vs Quick sort).
- How? Interface for strategy + pass concrete implementation.
- Implementation:

Example 3: Factory Method Pattern

- What? Central place to create objects.
- When? You want control over object creation!
- How? Factory method decides what object to return.
- Key Benefit: Decouples object creation from usage.
- Implementation

Example 4: Observer Pattern

- What? One-to-many notification system.
- When? Need listeners (e.g., UI buttons, events).
- How? Register listeners, notify them when something happens.
- Implementation:

Java

```
Logger logger1 = Logger.getInstance();
Logger logger2 = Logger.getInstance();
System.out.println(logger1 == logger2); // true
```

Java

```
class Logger {
    private static Logger instance;

    private Logger() {
        // private constructor to
        //prevent external instantiation
    }

    public static Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }

    public void log(String message) {
        System.out.println("LOG: " + message);
    }
}

public class SingletonPattern {
    public static void main(String[] args) {
        Logger logger1 = Logger.getInstance();
        Logger logger2 = Logger.getInstance();

        logger1.log("First message");
        logger2.log("Second message");

        System.out.println(logger1 == logger2); // true
    }
}
```

Code

```
Sortable sorter = new BubbleSort(); // or QuickSort
ComplexClass task = new ComplexClass(sorter);
task.doAComplexThing();
```

Java

```
interface Sortable {
    public int[] sort(int[] numbers);
}

class BubbleSort implements Sortable {
    public int[] sort(int[] numbers) {
        return numbers; //LOGIC GOES HERE
    }
}

class QuickSort implements Sortable {
    public int[] sort(int[] numbers) {
        return numbers; //LOGIC GOES HERE
    }
}

class ComplexClass {
    private Sortable sorter;

    ComplexClass(Sortable sorter) {
        this.sorter = sorter;
    }

    void doAComplexThing() {
        int[] values = null;

        // logic...

        sorter.sort(values);

        // logic...
    }
}

public class StrategyPattern {
    public static void main(String[] args) {
        ComplexClass complexClassInstance
            = new ComplexClass(new BubbleSort());
        complexClassInstance.doAComplexThing();
    }
}
```

Code

```
Shape shape = ShapeFactory.getShape("CIRCLE"); // returns Circle
```

Java

```

class ShapeFactory {
    public static Shape getShape(String type) {
        if (type.equalsIgnoreCase("CIRCLE")) {
            return new Circle();
        } else if (type.equalsIgnoreCase("SQUARE")) {
            return new Square();
        }
        return null;
    }
}

class Shape {
}

class Circle extends Shape {
}

class Square extends Shape {
}

public class FactoryMethodPattern {
    public static void main(String[] args) {
        Shape circle
            = ShapeFactory
                .getShape("CIRCLE");

        Shape square
            = ShapeFactory
                .getShape("SQUARE");
    }
}

```

Java

```

Listener listener
    = () -> System.out.println(
        "Button clicked!");

button.setListener(listener);

button.click();

```

Java

```

interface Listener {
    void onClick();
}

class Button {
    private Listener listener;

    public void setListener(Listener listener) {
        this.listener = listener;
    }

    public void click() {
        if (listener != null) {
            listener.onClick();
        }
    }
}

public class ObserverPattern {
    public static void main(String[] args) {
        Button button = new Button();

        Listener listener
            = () -> System.out.println(
                "Button clicked!");

        button.setListener(listener);

        button.click();
    }
}

```

Q8

What is Dependency Injection?

What is Dependency Injection (DI)?

- What? Passing dependencies from outside, not creating inside.
- Why? Makes code flexible, testable, and less dependent.
- When? Use when you want to swap implementations easily.
- Example without DI

Direct creation of MySQLDatabase

Can't change database easily

- Example with DI

Inject Database through constructor of Application

Decoupled: Application depends on interface, not specific class.

Bad Example

Good Example

[Code](#)

```
class Application {
    private Database db = new MySQLDatabase(); // Yikes!
}
```

Code

```
class Application {
    private Database db = new MySQLDatabase(); // Yikes!
}
```

Code

```
Application app1 = new Application(
    new MySQLDatabase());
Application app2 = new Application(
    new MongoDBDatabase());
```

Code

```
Application app1 = new Application(
    new MySQLDatabase());
Application app2 = new Application(
    new MongoDBDatabase());
```

Java

```
class MySQLDatabase {
    public void connect() {
        System.out.println("Connecting to MySQL Database");
    }
}

class Application {
    //Database is a dependency of Application!
    private MySQLDatabase db = new MySQLDatabase();

    public void start() {
        db.connect();
        //Do whatever you want!
    }
}

public class DependencyInjectionExample {
    public static void main(String[] args) {
        Application app = new Application();
        app.start();
    }
}
```

Java


```

interface Database {
    void connect();
}

class MySQLDatabase implements Database {
    public void connect() {
        System.out.println("Connected to MySQL.");
    }
}

class MongoDBDatabase implements Database {
    public void connect() {
        System.out.println("Connected to MongoDB.");
    }
}

class Application {
    private Database db;

    public Application(Database db) {
        this.db = db;
    }

    public void start() {
        db.connect();
    }
}

public class Main {
    public static void main(String[] args) {
        Database mySQL = new MySQLDatabase();
        Application app1 = new Application(mySQL);
        app1.start();

        Database mongo = new MongoDBDatabase();
        Application app2 = new Application(mongo);
        app2.start();
    }
}

```

Q9

Explain a few different scenarios of using Interfaces in Java

1. Defining Common Behavior for Unrelated Classes

- What? Different classes that should share the same behavior.
- Example: Flyable interface for birds and airplanes.

2. Achieving Multiple Inheritance

- What? Java does not support multiple inheritance with classes, but it allows multiple interfaces.
- Example: A SmartDevice that acts as both Camera and Phone.

3. Implementing Dependency Injection

- What? Allows flexibility by programming to an interface instead of a concrete class.
- Example: Database interface for switching between MySQLDatabase and MongoDBDatabase.

4. Creating Callbacks (Observer Pattern)

- What? Enables event-driven programming where one class notifies another.
- Example: Listener interface for handling button clicks.

5. Defining Strategy Pattern for Flexible Algorithms

- What? Allows switching between different behaviors at runtime.
- Example: Different payment methods implementing PaymentStrategy.

Java

```
interface Flyable {
    void fly();
}

class Bird implements Flyable {
    public void fly() {
        System.out.println(
            "Bird flaps its wings to fly.");
    }
}

class Airplane implements Flyable {
    public void fly() {
        System.out.println(
            "Airplane flies using engines.");
    }
}

public class Main {
    public static void main(String[] args) {
        Flyable bird = new Bird();
        bird.fly();

        Flyable plane = new Airplane();
        plane.fly();
    }
}
```

Java

```

interface Camera {
    void takePhoto();
}

interface Phone {
    void makeCall(String number);
}

class SmartPhone implements Camera, Phone {
    public void takePhoto() {
        System.out.println("Photo taken.");
    }

    public void makeCall(String number) {
        System.out.println("Calling " + number);
    }
}

public class Main {
    public static void main(String[] args) {
        SmartPhone phone = new SmartPhone();
        phone.takePhoto();
        phone.makeCall("123-456-7890");
    }
}

```

Java

```

interface Database {
    void connect();
}

class MySQLDatabase implements Database {
    public void connect() {
        System.out.println("Connected to MySQL.");
    }
}

class MongoDBDatabase implements Database {
    public void connect() {
        System.out.println("Connected to MongoDB.");
    }
}

class Application {
    private Database db;

    public Application(Database db) {
        this.db = db;
    }

    public void start() {
        db.connect();
    }
}

public class Main {
    public static void main(String[] args) {
        Database mySQL = new MySQLDatabase();
        Application app1 = new Application(mySQL);
        app1.start();

        Database mongo = new MongoDBDatabase();
        Application app2 = new Application(mongo);
        app2.start();
    }
}

```

Java

```
interface OnClickListener {
    void onClick();
}

class Button {
    private OnClickListener listener;

    public void setOnClickListener(
        OnClickListener listener) {
        this.listener = listener;
    }

    public void click() {
        if (listener != null) {
            listener.onClick();
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Button button = new Button();

        button.setOnClickListener(new OnClickListener() {
            public void onClick() {
                System.out.println("Button clicked!");
            }
        });

        button.click();
    }
}
```

Java

```

interface PaymentStrategy {
    void pay(int amount);
}

class CreditCardPayment implements PaymentStrategy {
    public void pay(int amount) {
        System.out.println("Paid $"
            + amount + " using Credit Card.");
    }
}

class PayPalPayment implements PaymentStrategy {
    public void pay(int amount) {
        System.out.println("Paid $"
            + amount + " using PayPal.");
    }
}

class ShoppingCart {
    private PaymentStrategy paymentStrategy;

    public ShoppingCart(PaymentStrategy paymentStrategy) {
        this.paymentStrategy = paymentStrategy;
    }

    public void checkout(int amount) {
        paymentStrategy.pay(amount);
    }
}

public class Main {
    public static void main(String[] args) {
        ShoppingCart cart1
            = new ShoppingCart(new CreditCardPayment());
        cart1.checkout(100);

        ShoppingCart cart2
            = new ShoppingCart(new PayPalPayment());
        cart2.checkout(200);
    }
}

```

Q10

Give different usecases for inheritance in Java

Here are a few usecases for using inheritance in Java:

1. Code Reusability

- What? Allows a class to reuse properties and methods of another class.
- Why? Avoids duplicate code and promotes efficient programming.
- Example: Car and Bike inherit common behavior from Vehicle.

2. Method Overriding (Polymorphism)

- What? Allows a subclass to provide a different implementation of a method.
- Why? Enables dynamic method behavior based on object type.
- Example: Dog and Cat override makeSound() from Animal.

3. Extending Functionality

- What? Allows adding new behavior to an existing class.
- Why? Enhances the base class without modifying it.
- Example: ElectricCar extends Car to add battery functionality.

Summary Table

Need	Example
1. Code Reusability	Vehicle Car, Bike
2. Method Overriding	Animal Dog, Cat
3. Extending Functionality	Car ElectricCar

Java

```

class Vehicle {
    void start() {
        System.out.println("Vehicle is starting...");
    }
}

class Car extends Vehicle {
    void drive() {
        System.out.println("Car is driving...");
    }
}

class Bike extends Vehicle {
    void ride() {
        System.out.println("Bike is riding...");
    }
}

public class Main {
    public static void main(String[] args) {
        Car car = new Car();
        car.start();
        car.drive();

        Bike bike = new Bike();
        bike.start();
        bike.ride();
    }
}

```

Java

```

class Animal {
    void makeSound() {
        System.out.println("Animal makes a sound...");
    }
}

class Dog extends Animal {
    void makeSound() {
        System.out.println("Dog barks...");
    }
}

class Cat extends Animal {
    void makeSound() {
        System.out.println("Cat meows...");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal myDog = new Dog();
        myDog.makeSound();

        Animal myCat = new Cat();
        myCat.makeSound();
    }
}

```

Java

```

class Car {
    void drive() {
        System.out.println("Car is driving...");
    }
}

class ElectricCar extends Car {
    void chargeBattery() {
        System.out.println("Battery is charging...");
    }
}

public class Main {
    public static void main(String[] args) {
        ElectricCar tesla = new ElectricCar();
        tesla.drive();
        tesla.chargeBattery();
    }
}

```

Q11

What is Encapsulation?

Overview

- Bundling data (variables) and methods (functions) into a single unit
- Restricts direct access to data
 - Allows controlled modifications through methods.

Bad Code: No Encapsulation

- Problem: Direct access to balance allows invalid values.

Intermediate Code: Getters and Setters Without Validation

Why Is This Still Not Ideal?

- Uses private fields with getters and setters (better than public fields).
- (HOWEVER) No validation in `setBalance()`, allowing negative values.
- (HOWEVER) Allows setting any balance without controlled deposit/withdraw methods.

Improved Code: Encapsulation

- Use private fields
- Add getters and setters as needed
- Add methods to perform other operations with necessary validations

Java

```
class BankAccount {
    public double balance;
}

public class Main {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();
        account.balance = -1000; // Invalid state
        System.out.println("Balance: " + account.balance);
    }
}
```

Java


```

class BankAccount {
    private double balance;

    public double getBalance() {
        return balance;
    }

    public void setBalance(double balance) {
        // No validation
        this.balance = balance;
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();

        // Still allows invalid values
        account.setBalance(-1000);

        System.out.println("Balance: "
            + account.getBalance());
    }
}

```

Java

```

class BankAccount {
    private double balance;

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }

    public void withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
        }
    }

    public double getBalance() {
        return balance;
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();
        account.deposit(500);
        account.withdraw(100);
        System.out.println("Balance: "
            + account.getBalance());
    }
}

```

Q12

Advantages of Encapsulation

Encapsulation improves maintainability, and flexibility in object-oriented programming.

1. Data Hiding (Prevents Direct Modification)

- Bad Code: balance is public, allowing direct access
- Improved Code: balance is private, preventing direct modification

2. Controlled Access to Data

- Bad Code: Getters and setters allow setting an invalid balance.
- Improved Code: deposit() and withdraw() validate transactions before modifying balance.
- Advantage: Prevents accidental or malicious modifications.

3. Increased Maintainability & Flexibility

- Bad Code: Direct access to balance means changing the logic requires modifying multiple parts of the program.
- Improved Code: Encapsulated logic allows easy modifications inside deposit() or withdraw().
- Advantage: If business rules change, updates happen in one place instead of throughout the program.

Code

```
// Bad: Allows setting an invalid balance
account.balance = -1000;

// Improved: Direct modification is restricted
account.deposit(500);
account.withdraw(100);
```

Code

```
// Bad: No validation in setter
// Allows invalid values
account.setBalance(-1000);

// Improved: Only valid transactions are allowed
account.deposit(500);
// Will not withdraw more than available balance
account.withdraw(600);
```

Code

```
// Improved: We can easily add new rules
public void deposit(double amount) {
    // Added new rule: Max deposit limit
    if (amount > 0 && amount < 10000) {
        balance += amount;
    }
}
```

Q13**Encapsulation vs Security**

Concept	Encapsulation
Purpose	Prevent accidental modification of data in your code
How It Works	Uses private variables and controlled access via methods
Example	Prevents direct modification of score in a CricketScorer class

Q14**Explain Encapsulation in Java Using ArrayList as an example****1: How Data is Stored in ArrayList**

- ArrayList internally uses a dynamic array (elementData) to store elements.
- This array is private, meaning it cannot be accessed or modified directly from outside the class.
- The size of the list is managed separately to track the number of elements.
- ArrayList prevents direct access to elementData, ensuring safety and consistency.
- JDK Implementation (ArrayList.java):

2: Operations Defined Without Direct Access to Data

- Since elementData is private, Java provides public methods to safely modify and access elements.
- These methods validate inputs, manage memory, and prevent illegal operations.
- Example Operation: Adding Elements (add())
The add() method increases the size of ArrayList and stores the element safely.
Ensures capacity before adding a new element.
Users can only add elements through add(), preventing direct modification of elementData

Accessing Elements (get())

- The get() method retrieves an element from elementData while checking for valid indexes.

3: Summary

Encapsulation Feature	Implementation in ArrayList
Private Data	private transient Object[] elementData;
Controlled Access	add(E e), get(int index), remove(int index)
Validation Methods	rangeCheck(index), ensureCapacityInternal(int minCapacity)

Java

```
public class ArrayList<E>
    extends AbstractList<E>
    implements List<E>, RandomAccess, Cloneable {

    // Internal storage array (Encapsulated - private)
    private transient Object[] elementData;

    // Current number of elements in the list
    private int size;

}
```

Code

```
public boolean add(E e) {
    // Check if array needs resizing
    ensureCapacityInternal(size + 1);

    // Store element safely
    elementData[size++] = e;

    return true;
}

// Ensures there is enough space for new elements
private void ensureCapacityInternal(int minCapacity) {
    if (minCapacity - elementData.length > 0) {
        // Increases array size if needed
        grow(minCapacity);
    }
}
```

Code

```
public E get(int index) {
    rangeCheck(index); // Validate index
    return (E) elementData[index]; // Return element
}

// Ensures index is within valid range
private void rangeCheck(int index) {
    if (index >= size)
        throw new IndexOutOfBoundsException();
}
```

Overview

- Abstraction hides complexity
Only show what is necessary
- The user interacts with the essential parts
While the implementation details remain hidden

Real-Life Examples of Abstraction

- Driving a Car: You turn the steering wheel and press the accelerator
You don't need to know how the engine, fuel injection, or transmission work.
- Printing a Document: You click "Print" on your computer.
You don't need to know how the printer processes the data or converts it into ink on paper.

Abstraction in Software Engineering

- Layered Architecture in Applications: Modern applications follow a layered approach to abstract complexity.

The Web Layer doesn't need to know how the Database Layer retrieves data. Layer Purpose Web Layer Displays the webpage to the user Business Logic Layer Has the Business Logic Data Layer Retrieves and stores data in the database

Abstraction in Programming

- High Level Languages: Computers only understand binary (0s and 1s).
Instead of writing machine code, we write Java Code
The complexity of converting Java code and running it is abstracted away from us
- Writing SQL Queries: SQL abstracts database operations.
You write a simple query like: `SELECT * FROM users WHERE age > 18`
The database handles how the query is executed behind the scenes
- Calling Built-In Methods: Use library methods
When calling `Math.sqrt(9)`, you don't see how Java calculates the square root.
The internal implementation is abstracted away.

Summary

- Abstraction simplifies usage by hiding low-level details.
- Allows focus on essential functionality without worrying about implementation.
- Used in everyday life, software architecture, and programming languages.

Q16

What is Cohesion?

Overview

- What?: Cohesion measures how closely related the responsibilities of a class are.
- High Cohesion: A class has a single, well-defined responsibility.
- Low Cohesion: A class has multiple unrelated responsibilities.

Example of Low Cohesion

- Multiple Responsibilities: This class downloads data, parses it, and stores it.
- Tightly Coupled: Changes in downloading logic may affect parsing and storage.
- Difficult to Reuse: If another part of the program needs only specific feature, it might be difficult to reuse.

Example of High Cohesion

- Single Responsibility: Each class performs a specific task.
- Easier to Test: Each class can be tested separately.
- Improved Reusability: The `InternetDownloader` class can be used to download other data as well.

How ArrayList Maintains High Cohesion?

Cohesion Principle

Single Responsibility

Encapsulated Data

No Unrelated Methods

Java

```

public class DownloadAndStore {
    public void downloadFromInternet() {
        // Code to download data from the internet
    }

    public void parseData() {
        // Code to parse the downloaded data
    }

    public void storeInDatabase() {
        // Code to store the parsed data in the database
    }

    public void doEverything() {
        downloadFromInternet();
        parseData();
        storeInDatabase();
    }
}

```

Java

```

public class InternetDownloader {
    public Data downloadFromInternet() {
        // Code to download data from the internet
        return new Data();
    }
}

public class DataParser {
    public ParsedData parseData(Data data) {
        // Code to parse the downloaded data
        return new ParsedData();
    }
}

public class DatabaseStorer {
    public void storeInDatabase(ParsedData parsedData) {
        // Code to store the parsed data in the database
    }
}

public class DownloadAndStore {
    private InternetDownloader downloader
        = new InternetDownloader();
    private DataParser parser = new DataParser();
    private DatabaseStorer storer = new DatabaseStorer();

    public void doEverything() {
        Data data = downloader.downloadFromInternet();
        ParsedData parsedData = parser.parseData(data);
        storer.storeInDatabase(parsedData);
    }
}

```

Q17

What is Coupling?

Overview

- What?: Coupling measures how much a class depends on another class.
- Tight Coupling: A class is tightly linked to another. Changes in one impacts other.
- Loose Coupling: A class has minimal dependencies. Changes are easy to make.

Simple Example of Tight Coupling

- Problem: OrderProcessor is tightly coupled to PayPalPayment. If we need another payment method, we must modify OrderProcessor.

Improved - Loose Coupling

- Solution: Use an interface (PaymentMethod) to allow multiple implementations.

Complex Example of Tight Coupling

- Direct Dependency: Order directly accesses ShoppingCartEntry fields.
- Tightly Coupled: Changes in ShoppingCartEntry might affect Order.

Solution: Reducing Coupling

- Encapsulation: ShoppingCartEntry handles its own pricing logic.
- Improved Maintainability: Order no longer depends on ShoppingCartEntry.

Java

```
class PayPalPayment {
    void processPayment(int amount) {
        System.out.println(
            "Processing PayPal payment of $" + amount);
    }
}

class OrderProcessor {
    private PayPalPayment payment = new PayPalPayment();

    void checkout(int amount) {
        payment.processPayment(amount);
    }
}
```

Java


```

interface PaymentMethod {
    void processPayment(int amount);
}

class PayPalPayment implements PaymentMethod {
    public void processPayment(int amount) {
        System.out.println(
            "Processing PayPal payment of $"
            + amount);
    }
}

class CreditCardPayment implements PaymentMethod {
    public void processPayment(int amount) {
        System.out.println(
            "Processing Credit Card payment of $"
            + amount);
    }
}

class OrderProcessor {
    private PaymentMethod payment;

    OrderProcessor(PaymentMethod payment) {
        this.payment = payment;
    }

    void checkout(int amount) {
        payment.processPayment(amount);
    }
}

public class Main {
    public static void main(String[] args) {
        OrderProcessor order1
            = new OrderProcessor(new PayPalPayment());
        order1.checkout(100);

        OrderProcessor order2
            = new OrderProcessor(new CreditCardPayment());
        order2.checkout(200);
    }
}

```

Java

```

public class ShoppingCartEntry {
    public double price;
    public int quantity;
}

public class Order {
    private ShoppingCartEntry[] items;
    private double salesTax;

    public double calculateTotalPrice() {
        double totalPrice = 0;
        for (ShoppingCartEntry item : items) {
            totalPrice += item.price * item.quantity;
        }
        return totalPrice + salesTax;
    }
}

```

Java

```

public class ShoppingCartEntry {
    private double price;
    private int quantity;

    public double getTotalPrice() {
        return price * quantity;
    }
}

public class ShoppingCart {
    private List<ShoppingCartEntry> items;

    public double getTotalPrice() {
        double totalPrice = 0;
        for (ShoppingCartEntry item : items) {
            totalPrice += item.getTotalPrice();
        }
        return totalPrice;
    }
}

public class Order {
    private ShoppingCart cart;
    private double salesTax;

    public double calculateTotalPrice() {
        return cart.getTotalPrice() + salesTax;
    }
}

```

Q18

How to Reduce Coupling in Java?

1: Use Interfaces and Dependency Injection

- **Bad Example (Tight Coupling)**
Car is tightly coupled to Engine.
If Engine changes or a new engine type is needed, Car must also change.
- **Good Example (Loose Coupling - Using Interfaces and DI)**
Car depends on Engine interface, not a specific implementation.
Easily switch engines without modifying Car.

2: Use Factory Pattern Instead of Hardcoding Dependencies

- **Bad Example (Tight Coupling - Hardcoded Object Creation)**
Application is tightly coupled to MySQLDatabase.
Cannot easily switch to PostgreSQL or other databases.
- **Good Example (Loose Coupling - Using Factory Pattern)**
Application now depends on Database, not a specific implementation.
Can easily switch databases without modifying Application class.
Factory Pattern manages object creation.

Java

```
class Engine {
    void start() {
        System.out.println("Engine starting...");
    }
}

class Car {
    // Direct dependency
    private Engine engine = new Engine();

    void drive() {
        engine.start();
        System.out.println("Car is driving...");
    }
}

public class Main {
    public static void main(String[] args) {
        Car car = new Car();
        car.drive();
    }
}
```

Java

```
interface Engine {
    void start();
}

class PetrolEngine implements Engine {
    public void start() {
        System.out.println("Petrol engine starting...");
    }
}

class DieselEngine implements Engine {
    public void start() {
        System.out.println("Diesel engine starting...");
    }
}

class Car {
    private Engine engine;

    // Constructor injection (Dependency Injection)
    public Car(Engine engine) {
        this.engine = engine;
    }

    void drive() {
        engine.start();
        System.out.println("Car is driving...");
    }
}

public class Main {
    public static void main(String[] args) {
        Engine petrol = new PetrolEngine();
        Car car1 = new Car(petrol);
        car1.drive();

        Engine diesel = new DieselEngine();
        Car car2 = new Car(diesel);
        car2.drive();
    }
}
```

Java

```
class MySQLDatabase {
    void connect() {
        System.out.println("Connected to MySQL.");
    }
}

class Application {
    private MySQLDatabase database
        = new MySQLDatabase(); // Hardcoded dependency

    void start() {
        database.connect();
        System.out.println("Application started.");
    }
}
```

Java

```

interface Database {
    void connect();
}

class MySQLDatabase implements Database {
    public void connect() {
        System.out.println("Connected to MySQL.");
    }
}

class PostgreSQLDatabase implements Database {
    public void connect() {
        System.out.println("Connected to PostgreSQL.");
    }
}

// Factory to create Database instances
class DatabaseFactory {
    public static Database getDatabase(String type) {
        if (type.equalsIgnoreCase("MySQL")) {
            return new MySQLDatabase();
        } else if (type.equalsIgnoreCase("PostgreSQL")) {
            return new PostgreSQLDatabase();
        }
        throw new IllegalArgumentException(
            "Unknown database type");
    }
}

class Application {
    private Database database;

    // Constructor Injection
    public Application(Database database) {
        this.database = database;
    }

    void start() {
        database.connect();
        System.out.println("Application started.");
    }
}

public class Main {
    public static void main(String[] args) {
        Database db = DatabaseFactory.getDatabase("MySQL");
        Application app = new Application(db);
        app.start();
    }
}

```

Q19

Abstraction vs Coupling vs Cohesion vs Encapsulation

Remember: There is a great deal of intersection between these concepts!

Concept	Definition	Goal	How to Achieve
Encapsulation	Bundling data (variables) and methods (functions) into a single unit	Prevent unintended changes	Private variables, well designed instance methods, getters, setters
Abstraction	Hides implementation details and exposes only what's necessary	Reduce complexity	Encapsulation, Interface, ..
Coupling	Measures how dependent a class is on another	Loose Coupling - Improve maintainability	Use well-defined interfaces
Cohesion	Measures how well a class focuses on a single responsibility	High Cohesion - Increase maintainability	Split unrelated responsibilities into separate classes

Q20

Favor Composition Over Inheritance

Overview

- Inheritance and composition are two options to reuse code
- However, composition is generally preferred unless there is a clear "IS-A" relationship.
- Why?
 - Provides greater flexibility
 - Reduces tight coupling between classes

What is Composition?

- Composition: "has-a" relationship where an object contains another object
- Allows objects to delegate behavior to other objects rather than inheriting behavior from a parent class.

Bad Example For Inheritance

Problems with the Above Approach

- **Incorrect Relationship:** A Car is not a type of Engine, but it has an engine.

Using inheritance (extends Engine) creates an incorrect "IS-A" relationship instead of a "HAS-A" relationship.

- **Unnecessary Method Inheritance:** Every method in Engine is automatically inherited by Car, even if it doesn't logically belong to a car.

Example: If Engine has a method `changeOil()`, Car will inherit it, even though a car itself does not change oil—the engine does.

- **Limited Flexibility & Maintainability:** What if we need to support multiple engine types like `PetrolEngine` or `ElectricEngine`?

With inheritance, we'd have to create multiple Car subclasses (`PetrolCar`, `ElectricCar`), making the system harder to extend and maintain.

A better approach is to use composition, where a Car contains an Engine object and delegates operations to it.

Better Approach: Composition

When to Use Inheritance vs. Composition?

Criteria	Use Inheritance
Relationship Type	When there is a true "is-a" relationship (e.g., Dog extends Animal)
Code Reusability	Inherits behavior from the parent class
Flexibility	Creates a rigid relationship – subclasses inherit everything from the parent class

Final Recommendations

- Use inheritance only when there is a true "is-a" relationship (e.g., Dog extends Animal).
- Use composition for "has-a" relationships (e.g., Car has an Engine).
- Avoid deep inheritance trees – They make code hard to maintain and understand.

Java

```

class Engine {
    public void start() {
        System.out.println("Engine is starting...");
    }
}

class Car extends Engine {
    public void drive() {
        System.out.println("Car is driving...");
    }
}

public class Main {
    public static void main(String[] args) {
        Car myCar = new Car();
        myCar.start(); // Inherited from Engine
        myCar.drive();
    }
}

```

Java

```

class Engine {
    public void start() {
        System.out.println("Engine is starting...");
    }
}

class Car {
    // Composition: Car has an Engine
    private Engine engine;

    public Car(Engine engine) {
        this.engine = engine;
    }

    public void drive() {
        // Delegating behavior to Engine
        engine.start();
        System.out.println("Car is driving...");
    }
}

public class Main {
    public static void main(String[] args) {
        Engine myEngine = new Engine();
        Car myCar = new Car(myEngine);
        myCar.drive();
    }
}

```

Q21

What is Polymorphism?

Overview

- What?: Polymorphism allows the same method or function to behave differently
- Meaning: Comes from the Greek words "poly" (many) and "morph" (forms).

Types of Polymorphism

- Compile-Time Polymorphism (Method Overloading) Decided at compile time.
- Run-Time Polymorphism (Method Overriding) Decided at runtime.

Example: Compile-Time Polymorphism (Method Overloading)

Example: Run-Time Polymorphism (Method Overriding)

Polymorphism works with interfaces as well

When a reference of an interface is used to hold objects of different classes implementing the interface, the method executed depends on the actual object:

This will print:

Even though `object.fly()` is called on the same reference type, different implementations are executed.

Java

```
public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }

    public double add(double a, double b) {
        return a + b;
    }
}

public class Main {
    public static void main(String[] args) {
        Calculator calc = new Calculator();
        System.out.println(calc.add(2, 3));
        System.out.println(calc.add(2.5, 3.5));
    }
}
```

Java

```

class Animal {
    public void sound() {
        System.out.println("Some generic animal sound");
    }
}

class Dog extends Animal {
    @Override
    public void sound() {
        System.out.println("Woof Woof");
    }
}

class Cat extends Animal {
    @Override
    public void sound() {
        System.out.println("Meow Meow");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal[] animals = {new Dog(), new Cat()};

        for (Animal animal : animals) {
            animal.sound();
        }
    }
}

```

Code

```

Flyable[] flyingObjects = {new Bird(), new Aeroplane()};

for (Flyable object : flyingObjects) {
    object.fly();
}

```

Code

Bird flies by flapping wings
Aeroplane flies using engines.

Q22

What is Dynamic Method Dispatch?

Overview

- What?: Mechanism by which a call to an overridden method is resolved at runtime instead of compile time.
- How?: Uses method overriding and a parent class reference that points to a child class object.

Example: Dynamic Method Dispatch in Action

- The method execution is determined at runtime, based on the actual object (Child), not the reference type (Parent).
- This enables loose coupling, where the parent reference can point to any subclass object.

Java

```
class Parent {
    void show() {
        System.out.println("Parent class method");
    }
}

class Child extends Parent {
    @Override
    void show() {
        System.out.println("Child class method");
    }
}

public class Main {
    public static void main(String[] args) {
        // Parent reference, Child object
        Parent obj = new Child();

        obj.show(); // Output: Child class method
    }
}
```

Q23

Static vs Dynamic Binding

Feature	Static Binding
Definition	Method call is resolved at compile time
Type of Polymorphism	Compile-time polymorphism (Method Overloading)
Method Execution	Based on reference type
Performance	Faster (resolved at compile time)
Example	Static Method Calls

Example: Static Binding (Compile-Time Resolution)

Example: Dynamic Binding (Runtime Resolution)

Key Takeaways

- Polymorphism enables flexibility by allowing the same method call to perform different actions.
- Dynamic Method Dispatch is the mechanism for achieving runtime polymorphism in Java.
- Static Binding happens at compile time, while Dynamic Binding happens at runtime.

Java

```
class MathOperations {
    static void multiply(int a, int b) {
        System.out.println("Multiplication: " + (a * b));
    }
}

public class Main {
    public static void main(String[] args) {
        // Output: Multiplication: 50
        MathOperations.multiply(5, 10);
    }
}
```

Java

```
class Parent {
    void display() {
        System.out.println("Parent display");
    }
}

class Child extends Parent {
    @Override
    void display() {
        System.out.println("Child display");
    }
}

public class Main {
    public static void main(String[] args) {
        Parent obj = new Child();
        obj.display(); // Output: Child display
    }
}
```

Q24

What is the need for Sealed Interfaces and Classes?

Example: Sealed Class

- Only allowed subclasses (Car, Truck, Bike) can extend Vehicle.
- Car is final, so it cannot be extended further.
- Truck is non-sealed, so any class can extend it.
- Bike is sealed again, allowing only ElectricBike to extend it.

Example: Sealed Interface

- Controls which classes can implement Animal.
- Dog is final, so no other class can extend it.
- Bird is non-sealed, so any class can implement it.
- Cat is sealed, so only PersianCat can extend it.

Summary:

- Java New Feature: Introduced in Java 17
- Restricts Inheritance: Only specified classes can extend or implement.
- Prevents Unwanted Subclassing: No accidental extensions.
- Improves Maintainability: Clearer type hierarchy.

Code

```
sealed class Vehicle permits Car, Truck, Bike {}

// No further subclassing
final class Car extends Vehicle {}

// Can be extended freely
non-sealed class Truck extends Vehicle {}

sealed class Bike extends Vehicle
    permits ElectricBike {}

// No further subclassing
final class ElectricBike extends Bike {}
```

Code

```
sealed interface Animal permits Dog, Cat, Bird {}

// Cannot be extended
final class Dog implements Animal {}

// Restricted to PersianCat
sealed class Cat implements Animal permits PersianCat {}

// Can be freely extended
non-sealed class Bird implements Animal {}

// No further subclassing
final class PersianCat extends Cat {}
```

CHAPTER 5

Java Object Oriented Programming Details

14 Questions & Answers

Q1

What is the superclass of every class in Java?

Overview

- In Java, the superclass of every class is the Object class.
- All Java classes implicitly inherit from Object - even if not explicitly specified.
- The Object class provides common methods that all Java objects inherit.
 - toString() Returns a string representation of the object.
 - equals() Compares two objects for equality.
 - hashCode() Returns a unique integer (hash code) for the object.
 - Few other methods related to synchronization that we will discuss later!

Example: Using Object Methods

Java

```
public class Demo {  
    public static void main(String[] args) {  
        Object obj = new Object();  
  
        // Outputs: java.lang.Object@<hashcode>  
        System.out.println(obj.toString());  
    }  
}
```

Q2

What are the best practices in implementing the toString() method?

Overview

- The toString() method provides a string representation of an object.
- By default, it prints ClassName@hashcode.
- Best Practice: Override toString() to return meaningful information about the object.

Example: Default toString() Behavior

Default Implementation in Object Class:

Example: Overriding toString() for Better Readability

toString() Implementation in String Class:

toString() Implementation in ArrayList Class:

Best Practices for Overriding toString():

- Provide Meaningful Information: Include relevant details about the object's state to aid in debugging and logging.
- Ensure Consistency: The string representation should be consistent with the object's state and behavior.
- Avoid Sensitive Information: Do NOT include confidential data in the toString() output.

Java

```
public class Animal {
    private String name;

    public Animal(String name) {
        this.name = name;
    }
}

public class Main {
    public static void main(String[] args) {
        Animal dog = new Animal("Tommy");

        // Output: Animal@<hashcode>
        System.out.println(dog);
    }
}
```

Code

```
public String toString() {
    return getClass().getName()
        + "@" + Integer.toHexString(hashCode());
}
```

Java


```

class Animal {
    private String name;

    public Animal(String name) {
        this.name = name;
    }

    @Override
    public String toString() {
        return "Animal{name='" + name + "'}";
    }
}

public class Main {
    public static void main(String[] args) {
        Animal dog = new Animal("Tommy");

        // Output: Animal{name='Tommy'}
        System.out.println(dog);
    }
}

```

Code

```

public String toString() {
    return this; //Returns the string itself
}

```

Code

```

//Returns a string representation of the list's elements.
//Empty - []
//Single Element - [10]
//Multiple Values - [10, 20, 30]
public String toString() {
    Iterator<E> it = iterator();
    if (!it.hasNext())
        return "[]";

    StringBuilder sb = new StringBuilder();
    sb.append('[');
    for (;;) {
        E e = it.next();
        sb.append(e == this ? "(this Collection)" : e);
        if (!it.hasNext())
            return sb.append(']').toString();
        sb.append(',').append(' ');
    }
}

```

Q3

What are the best practices in implementing the equals() method?

Overview

- Purpose: Used to compare two objects for equality.
- Default Behavior: Compares memory addresses of the objects (same as ==).

Object Class:

Returns true only if both references point to the same object

String class equals method:

Explanation:

- Reference Check: If both references point to the same object (`this == anObject`), it returns true.
- Type Check: It verifies that the passed object is an instance of String.
- Content Comparison: It compares the lengths and corresponding characters of the two strings.

AbstractList class

Explanation:

- Reference Check: If the object `o` is the same instance as this, it returns true.
- Type Check: It verifies that `o` is an instance of List.
- Element Comparison: It uses `ListIterator` to traverse both lists simultaneously, comparing corresponding elements using their `equals()` method. If any pair of elements is not equal, it returns false.
- Size Check: After the loop, if one list has more elements than the other, it returns false. Otherwise, it returns true.

Best Practices for Implementing `equals()`

- Reflexive `x.equals(x)` must return true.
- Symmetric If `x.equals(y)`, then `y.equals(x)`.
- Transitive If `x.equals(y)` and `y.equals(z)`, then `x.equals(z)`.
- Consistent Multiple calls to `x.equals(y)` should return the same result unless any of the objects are modified.
- Non-null `x.equals(null)` must always return false.

Important: Always override `hashCode()` when overriding `equals()` to ensure correct behavior in collections like `HashSet` and `HashMap`.

Objects.equals() method

- Static method in Objects class
- Provides a null-safe way to compare two objects for equality
- Commonly used when overriding the equals() method

Example Implementation

Code

```
public boolean equals(Object obj) {
    return (this == obj);
}
```

Java

```
@Override
public boolean equals(Object anObject) {
    if (this == anObject) {
        return true;
    }
    if (anObject instanceof String) {
        String aString = (String) anObject;
        int n = value.length;
        if (n == aString.value.length) {
            char[] v1 = value;
            char[] v2 = aString.value;
            int i = 0;
            while (n-- != 0) {
                if (v1[i] != v2[i])
                    return false;
                i++;
            }
            return true;
        }
    }
    return false;
}
```

Code

```
public boolean equals(Object o) {
    if (o == this)
        return true;
    if (!(o instanceof List))
        return false;

    ListIterator<E> e1 = listIterator();
    ListIterator<?> e2 = ((List<?>) o).listIterator();
    while (e1.hasNext() && e2.hasNext()) {
        E o1 = e1.next();
        Object o2 = e2.next();
        if (!(o1 == null ? o2 == null : o1.equals(o2)))
            return false;
    }
    return !(e1.hasNext() || e2.hasNext());
}
```

Java

```
System.out.println(Objects.equals("Hello", "Hello")); // true
System.out.println(Objects.equals("Hello", "World")); // false
System.out.println(Objects.equals(null, null)); // true
System.out.println(Objects.equals(null, "Hello")); // false
```

Java

```
@Override
public boolean equals(Object obj) {
    if (this == obj) return true;

    if (obj == null
        || getClass() != obj.getClass()) return false;

    Animal animal = (Animal) obj;

    return age == animal.age &&
        Objects.equals(name, animal.name) &&
        Objects.equals(species, animal.species);
}
```

Q4

What are the best practices in implementing the hashCode() method?

Overview

- Purpose: Generates a unique integer (hash code) for an object.
- Why?: Used in hashing-based collections like HashMap, HashSet, and Hashtable.
- Key Rule: If two objects are equal (equals() returns true), they must have the same hashCode().

Objects.hash() Method

- Static method in Objects class
- Provides a simple way to compute a hash code based on multiple fields of an object
- Commonly used when overriding the hashCode() method

Example: Overridden hashCode()

Example with both hashCode and equals

Code

```
public static int hash(Object... values)
//return Objects.hash(name, age);
```

Java

```
import java.util.Objects;

class Animal {
    private String name;

    public Animal(String name) {
        this.name = name;
    }

    @Override
    public int hashCode() {
        return Objects.hash(name);
    }
}
```

Java

```
import java.util.Objects;

class Animal {
    private String name;
    private int age;
    private String species;

    public Animal(String name, int age, String species) {
        this.name = name;
        this.age = age;
        this.species = species;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;

        if (obj == null
            || getClass() != obj.getClass()) return false;

        Animal animal = (Animal) obj;

        return age == animal.age &&
            Objects.equals(name, animal.name) &&
            Objects.equals(species, animal.species);
    }

    @Override
    public int hashCode() {
        return Objects.hash(name, age, species);
    }

    public static void main(String[] args) {
        Animal a1 = new Animal("Leo", 5, "Lion");
        Animal a2 = new Animal("Leo", 5, "Lion");
        Animal a3 = new Animal("Simba", 3, "Lion");

        System.out.println(a1.hashCode()); // Same as a2
        System.out.println(a2.hashCode()); // Same as a1
        System.out.println(a3.hashCode()); // Different
        System.out.println(a1.equals(a2)); // true
        System.out.println(a1.equals(a3)); // false
    }
}
```

Q5

Discuss Some of the Important Ways to Initialize Objects in Java

1. Using a Constructor

- What? Assigns values when the object is created.
- Why? Ensures the object starts with valid data.
- How? Define a constructor and pass values.

2. Using Setter Methods

- What? Assigns values after creating the object.
- Why? Allows changing values later.
- How? Create setter methods and call them.

3. Using Object Initialization Block

- What? Initializes variables inside an instance block.
- Why? Ensures some default values before constructors run.
- Usecase? Avoids code duplication in multiple constructors.
- How? Use { } blocks inside the class.

4. Using Static Block (For Static Initialization)

- What? Initializes static variables before object creation.
- Why? Useful for loading configuration data, database records, or external resources before objects are created.

Executes once when the class is loaded.

Ensures static data is ready before any instance is accessed.

- How? Use static { } inside the class.

5. Using a Factory Method

- What?: A static method creates and returns objects based on specific conditions.
- Why?

Controls object creation by centralizing logic.

Allows customization before returning instances.

Helps manage different object types or caching.

- How?

Use static methods to create and return instances.

Implement validation, default values, or object reuse inside the method.

6. Using Reflection API

- What? Creates an object using reflection.
- Why? Used in frameworks and dynamic object creation.
- How? Use `Class.forName()` and `newInstance()`.

7. Using Cloning

- What? Copies an existing object.
- Why? Useful for creating duplicate objects.

Cloning preserves the exact structure and properties of an object.

Useful when an object has multiple attributes, and manually copying each field is cumbersome.

- How? Implement `Cloneable` and override `clone()`.

8. Using Singleton

- What? Ensures only one instance of a class exists and provides a global access point to it.
- Why?

Prevents multiple object creation, reducing memory usage.

Ensures controlled access to shared resources.

- How?

Declare a private static instance inside the class.

Provide a public static method to return the instance.

Make the constructor private to restrict direct instantiation.

Example: Singleton for Database Connection

- Ensures only one database connection exists – `getInstance()` always returns the same reference.
- Prevents multiple connections – Avoids unnecessary resource usage.
- Global Access – Any class can get a database connection using `DatabaseConnection.getInstance()`.

Comparison Table

Method	Purpose
Constructor	When all values are known at creation
Setter Methods	When values may change later
Initialization Block	Default values before constructor runs (Common logic for multiple constructors)
Static Block	Useful for loading configuration data, database records, or external resources before objects are created.
Factory Method	Helps manage different object types or caching.
Reflection API	Frameworks and dependency injection
Cloning	Quickly and easily create an exact copy of an object
Singleton	Ensures only one instance exists in the JVM

Java

```

public class Fan {
    private String make;
    private double radius;
    private String color;

    public Fan(String make,
               double radius, String color) {
        this.make = make;
        this.radius = radius;
        this.color = color;
    }

    public void display() {
        System.out.println(make
            + " " + radius + " " + color);
    }

    public static void main(String[] args) {
        Fan fan = new Fan("Bajaj", 12.5, "white");
        fan.display();
    }
}

```

Java


```
public class Fan {
    private String make;
    private double radius;
    private String color;

    public void setMake(String make) {
        this.make = make;
    }

    public void setRadius(double radius) {
        this.radius = radius;
    }

    public void setColor(String color) {
        this.color = color;
    }

    public void display() {
        System.out.println(make
            + " " + radius + " " + color);
    }

    public static void main(String[] args) {
        Fan fan = new Fan();
        fan.setMake("Havells");
        fan.setRadius(14.0);
        fan.setColor("Black");
        fan.display();
    }
}
```

Java

```

public class Fan {
    private String make;
    private double radius;
    private String color;
    private String serialNumber;

    // Instance Initialization Block
    // Runs before any constructor
    // Common to multiple constructors!
    {
        //Let's assume that this is unique!
        serialNumber = "FAN-"
            + System.currentTimeMillis();
    }

    // Default Constructor
    public Fan() {
        System.out.println("Default Constructor: ");
    }

    // Constructor with make and radius
    public Fan(String make, double radius) {
        this.make = make;
        this.radius = radius;
        System.out.println(
            "Constructor: make and radius");
    }

    // Fully Parameterized Constructor
    public Fan(String make, double radius, String color) {
        this.make = make;
        this.radius = radius;
        this.color = color;
        System.out.println(
            "Constructor: make, radius, and color.");
    }

    public void display() {
        System.out.println(
            serialNumber + " " + make +
            " " + radius + " " + color);
    }
}

```

Java

```
import java.util.ArrayList;
import java.util.List;

public class Fan {
    private static List<String> availableModels;

    // Static block for initialization
    static {
        availableModels = new ArrayList<>();
        loadModelsFromDatabase();
    }

    // Simulated database loading method
    private static void loadModelsFromDatabase() {
        availableModels.add("Orient Aeroquiet");
        availableModels.add("Havells Stealth");
        availableModels.add("Crompton Energion");
        availableModels.add("Usha Bloom");
    }

    public static void main(String[] args) {
        System.out.println("Available Fan Models: "
                           + availableModels);
    }
}
```

Java

```

enum FanType {
    CEILING, TABLE, EXHAUST
}

public class Fan {
    private String make;
    private double radius;
    private String color;
    private FanType type;
    private int speedLevels;

    // Private constructor to restrict direct instantiation
    private Fan(String make, double radius,
        String color, FanType type, int speedLevels) {
        this.make = make;
        this.radius = radius;
        this.color = color;
        this.type = type;
        this.speedLevels = speedLevels;
    }

    // Factory method
    public static Fan createFan(FanType type, String make,
        double radius, String color) {
        int defaultSpeedLevels;

        switch (type) {
            case CEILING:
                // Ceiling fans typically have more speeds
                defaultSpeedLevels = 5;
                break;
            case TABLE:
                // Table fans usually have 3 speeds
                defaultSpeedLevels = 3;
                break;
            case EXHAUST:
                // Exhaust fans generally have a single speed
                defaultSpeedLevels = 1;
                break;
            default:
                throw new IllegalArgumentException(
                    "Unknown fan type");
        }

        return new Fan(make, radius,
            color, type, defaultSpeedLevels);
    }

    public void display() {
        StringBuilder sb = new StringBuilder();
        sb.append(type != null ? type + " Fan - " : "Fan - ");
        sb.append(make != null ? make : "Unknown Make");
        sb.append(", ").append(
            radius > 0 ? radius + " inches" : "Unknown Size");
        sb.append(", ").append(
            color != null ? color : "Unknown Color");
        sb.append(", Speed Levels: ")
            .append(speedLevels > 0 ? speedLevels : "N/A");

        System.out.println(sb.toString());
    }

    public static void main(String[] args) {
        Fan ceilingFan = Fan.createFan(FanType.CEILING,
            "Havells", 48.0, "White");
        Fan tableFan = Fan.createFan(FanType.TABLE,
            "Crompton", 16.0, "Blue");
        Fan exhaustFan = Fan.createFan(FanType.EXHAUST,
            "Bajaj", 12.0, "Black");

        ceilingFan.display();
        tableFan.display();
        exhaustFan.display();
    }
}

```

```
}
```

Java

```
public class Fan {
    private String make;
    private double radius;
    private String color;

    public Fan() {
        System.out.println(
            "Fan object created using Reflection!");
    }

    public static void main(String[] args)
        throws Exception {
        Fan fan = (Fan) Class.forName("Fan")
            .getDeclaredConstructor().newInstance();
    }
}
```

Java

```
public class Fan implements Cloneable {
    private String make;
    private double radius;
    private String color;

    public Fan(String make,
        double radius, String color) {
        this.make = make;
        this.radius = radius;
        this.color = color;
    }

    @Override
    protected Object clone()
        throws CloneNotSupportedException {
        return super.clone();
    }

    public void display() {
        System.out.println(make +
            " " + radius + " " + color);
    }

    public static void main(String[] args)
        throws CloneNotSupportedException {
        Fan original = new Fan("Kenstar", 12.0, "Green");
        Fan cloned = (Fan) original.clone();

        original.display();
        cloned.display();
    }
}
```

Java

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DatabaseConnection {
    private static DatabaseConnection instance;
    private Connection connection;

    private DatabaseConnection() {
        try {
            // Simulated database connection
            String url = "jdbc:mysql://localhost:3306/mydatabase";
            String user = "root";
            String password = "password";
            connection = DriverManager.
                getConnection(url, user, password);
            System.out.println("Database connected.");
        } catch (SQLException e) {
            throw new RuntimeException(
                "Error connecting to database!", e);
        }
    }

    // Public method to provide access to the single instance
    // TODO: Synchronize if you want to make this thread safe
    public static DatabaseConnection getInstance() {
        if (instance == null) {
            instance = new DatabaseConnection();
        }
        return instance;
    }

    public Connection getConnection() {
        return connection;
    }

    public static void main(String[] args) {
        // Getting the singleton instance
        DatabaseConnection db1
            = DatabaseConnection.getInstance();

        DatabaseConnection db2
            = DatabaseConnection.getInstance();

        // Verifying both references point to the same object
        System.out.println("db1 == db2: " + (db1 == db2));
    }
}

```

Q6

How does the default constructor work in Java?

What?

- A default constructor is a constructor with no parameters.
- If a class has no constructor, Java automatically provides a default constructor.

Why?

- It initializes an object with default values.

Example:

Will a default constructor be provided if you define a custom constructor?

- No: Java does NOT provide a default constructor if a custom constructor is defined.

Example Without Custom Constructor

Example With Custom Constructor (No Default Constructor)

How to Fix?

- If both default and parameterized constructors are needed, explicitly define the default constructor.

Java

```
public class Fan {  
    public static void main(String[] args) {  
        Fan fan = new Fan(); // Calls default constructor  
        fan.display();  
    }  
}
```

Java

```
public class Fan {  
    private String make;  
    // Java automatically provides:  
    // public Fan() { }  
}  
  
//`new Fan();`  
// Java adds a default constructor.
```

Java

```

public class Fan {
    private String make;

    // Custom Constructor
    public Fan(String make) {
        this.make = make;
    }

    public static void main(String[] args) {
        Fan fan1 = new Fan("Havells"); // Works
        Fan fan2 = new Fan(); // Compilation Error
    }
}

```

Java

```

public class Fan {
    private String make;

    // Default Constructor
    public Fan() {
    }

    // Custom Constructor
    public Fan(String make) {
        this.make = make;
    }
}

```

Q7

Why is a copy constructor needed?

What?

- A copy constructor creates a new object by copying another object.

Why?

- clone() copies everything. Copy constructor provides a way to customize what you want to copy!

How?

- Define a constructor that takes an object of the same class as a parameter.

Example:

Java


```

public class Fan {
    private String make;
    private double radius;
    private String color;

    // Constructor
    public Fan(String make, double radius, String color) {
        this.make = make;
        this.radius = radius;
        this.color = color;
    }

    // Copy Constructor
    public Fan(Fan other) {
        this.make = other.make;
        this.radius = other.radius;
        this.color = other.color;
    }

    public void display() {
        System.out.println(make + " " + radius + " " + color);
    }

    public static void main(String[] args) {
        Fan fan1 = new Fan("Havells", 12.5, "Black");
        Fan fan2 = new Fan(fan1); // Using copy constructor

        fan1.display();
        fan2.display();
    }
}

```

Q8

What is the need for a private constructor?

What?

- A private constructor restricts object creation from outside the class.

Why?

- Singleton Pattern (Only one instance of a class).
- Factory Methods (Control object creation).
- Utility Classes (Prevent object creation).

Example (Singleton Pattern):

Example (Factory Method)

JDK Example: java.lang.Math

The Math class consists solely of static methods and constants. To prevent instantiation, it employs a private constructor:

JDK Example: java.lang.Runtime

The Runtime class follows the Singleton pattern, ensuring a single instance throughout the application's lifecycle. It achieves this by using a private constructor and a static method to provide access to the instance:

Java

```
public class Fan {
    private static Fan instance;

    // Private Constructor
    private Fan() {
        System.out.println("Fan instance created!");
    }

    public static Fan getInstance() {
        if (instance == null) {
            instance = new Fan();
        }
        return instance;
    }

    public static void main(String[] args) {
        Fan fan1 = Fan.getInstance();
        Fan fan2 = Fan.getInstance();
    }
}
```

Java

```

enum FanType {
    CEILING, TABLE, EXHAUST
}

public class Fan {
    private String make;
    private double radius;
    private String color;
    private FanType type;
    private int speedLevels;

    // Private constructor to restrict direct instantiation
    private Fan(String make, double radius,
        String color, FanType type, int speedLevels) {
        this.make = make;
        this.radius = radius;
        this.color = color;
        this.type = type;
        this.speedLevels = speedLevels;
    }

    // Factory method
    public static Fan createFan(FanType type, String make,
        double radius, String color) {
        int defaultSpeedLevels;

        switch (type) {
            case CEILING:
                // Ceiling fans typically have more speeds
                defaultSpeedLevels = 5;
                break;
            case TABLE:
                // Table fans usually have 3 speeds
                defaultSpeedLevels = 3;
                break;
            case EXHAUST:
                // Exhaust fans generally have a single speed
                defaultSpeedLevels = 1;
                break;
            default:
                throw new IllegalArgumentException(
                    "Unknown fan type");
        }

        return new Fan(make, radius,
            color, type, defaultSpeedLevels);
    }
}

```

Code

```

public final class Math {
    // Private constructor prevents instantiation
    private Math() {}
    // Static methods and constants follow
}

```

Java

```

public class Runtime {
    private static final Runtime currentRuntime
        = new Runtime();

    // Private constructor prevents instantiation
    private Runtime() {}

    public static Runtime getRuntime() {
        return currentRuntime;
    }
}

```

Q9

super() vs this() vs super. vs this.

Keyword	What It Does	Where Used?
super()	Calls parent class constructor	Inside a subclass constructor
this()	Calls another constructor in the same class	Inside a constructor
super.	Accesses parent class variables/methods	Inside a subclass
this.	Refers to current class variables/methods	Inside the same class

1. Using super() (Calling Parent Constructor)
2. Using this() (Calling Another Constructor)
3. Using super. and this. (Accessing Variables)

Using this. is optional except when there is shadowing

Java

```

class Appliance {
    Appliance() {
        System.out.println("Appliance Constructor");
    }
}

class Fan extends Appliance {
    Fan() {
        super(); // Calls Appliance constructor
        System.out.println("Fan Constructor");
    }

    public static void main(String[] args) {
        Fan fan = new Fan();
    }
}

```

Java

```

public class Fan {
    private String make;
    private double radius;
    private String color;

    // Constructor 1
    public Fan() {
        this("Havells", 12.0, "Black"); // Calls Constructor 2
    }

    // Constructor 2
    public Fan(String make, double radius, String color) {
        this.make = make;
        this.radius = radius;
        this.color = color;
    }

    public void display() {
        System.out.println(make + " " + radius + " " + color);
    }

    public static void main(String[] args) {
        // Calls first constructor
        // which calls the second constructor
        Fan fan = new Fan();
        fan.display();
    }
}

```

Java

```

class Appliance {
    String brand = "Generic";
}

class Fan extends Appliance {
    String brand = "Havells";

    public void showBrands() {
        System.out.println("Fan Brand: "
            + this.brand); // Refers to Fan's brand
        System.out.println("Appliance Brand: "
            + super.brand); // Refers to Appliance's brand
    }

    public static void main(String[] args) {
        Fan fan = new Fan();
        fan.showBrands();
    }
}

```

Java

```

class Animal {
    private String name;
    private int age;
    private String species;

    public Animal(String name, int age, String species) {
        this.name = name;
        this.age = age;
        this.species = species;
    }

    //Equals

    @Override
    public int hashCode() {
        return Objects.hash(name, age, species);
    }
}

```

Q10

Why are default methods in an interface useful?

What?

- Default methods in an interface are methods with a body (implementation).
- Introduced in Java 8 using the default keyword.

Why?

- Helps in achieving backward compatibility:
Allows adding new methods to interfaces without breaking existing code.
- Provides a default implementation that classes can override.

Example:

Java

```
interface Fan {
    void turnOn();

    // Default Method
    default void showStatus() {
        System.out.println("Fan is running");
    }
}

class CeilingFan implements Fan {
    public void turnOn() {
        System.out.println("Ceiling Fan is ON");
    }
}

public class Main {
    public static void main(String[] args) {
        CeilingFan fan = new CeilingFan();
        fan.turnOn();
        fan.showStatus(); // Calls default method
    }
}
```

Q11

How Do Default Methods Help in Achieving Backward Compatibility?

What

- Default methods in interfaces (introduced in Java 8) allow adding new methods without breaking existing implementations.
- This avoids the need to modify all implementing classes, making library upgrades smoother.

Problem: Adding a New Method to an Interface

Consider an interface with an existing method `methodA()` and four implementations:

Option 1: Adding `methodB()` to Interface (Without Default Method)

- All four classes must implement `methodB()`, or the code won't compile.
- If this interface is part of a library and the four implementations are in different components, every component must be updated, which is not practical for backward compatibility.

Option 2: Using a Default Method for Backward Compatibility

- Instead of breaking all implementations, we provide a default implementation in the interface.
- Existing classes continue to work without modification, but they can override methodB() if needed.

Java

```
//Defined in a Common Library
interface ExampleInterface {
    void methodA(); // Existing method
}

// Four different classes implementing the interface
// In Four Different Projects!

class Class1 implements ExampleInterface {
    public void methodA() {
        System.out.println("Class1 - methodA");
    }
}

class Class2 implements ExampleInterface {
    public void methodA() {
        System.out.println("Class2 - methodA");
    }
}

class Class3 implements ExampleInterface {
    public void methodA() {
        System.out.println("Class3 - methodA");
    }
}

class Class4 implements ExampleInterface {
    public void methodA() {
        System.out.println("Class4 - methodA");
    }
}
```

Code

```
interface ExampleInterface {
    void methodA();

    // New method added
    // Breaks existing implementations
    void methodB();
}
```

Java


```
interface ExampleInterface {
    void methodA();

    default void methodB() {
        System.out.println("Default methodB implementation");
    }
}
```

Q12

When do you use a Marker Interface in Java?

What?

- A marker interface is an interface with no methods or fields.

Why?

- Tells the Java compiler or runtime that a class has a special property.
- Used by Java libraries to enable specific behavior.
- Example: Serializable tells Java that an object can be serialized.

Examples in Java:

Marker Interface

Serializable

Cloneable

Remote

Example (Creating Marker Interface)

- SecureResource is an empty interface used to mark classes needing extra security.
- BankAccount implements SecureResource, while PublicDocument does not require extra security.
- verifyAccess() uses instanceof to determine if an object needs additional verification.

Java

```

// Marker Interface (Empty)
interface SecureResource {}

// Class implementing the marker interface
class BankAccount implements SecureResource {
    private String accountNumber;
    private double balance;

    public BankAccount(
        String accountNumber, double balance) {
        this.accountNumber = accountNumber;
        this.balance = balance;
    }

    public void display() {
        System.out.println("Bank Account: " + accountNumber +
            ", Balance: $" + balance);
    }
}

// Class without marker interface
class PublicDocument {
    private String title;

    public PublicDocument(String title) {
        this.title = title;
    }

    public void display() {
        System.out.println("Public Document: " + title);
    }
}

// Utility to check security access
class SecurityCheck {
    public static void verifyAccess(Object obj) {
        if (obj instanceof SecureResource) {
            System.out.println(
                "Additional security check needed.");
        } else {
            System.out.println("Access granted.");
        }
    }
}

public class MarkerInterfaceDemo {
    public static void main(String[] args) {
        BankAccount account =
            new BankAccount("123456789", 5000.00);

        PublicDocument doc =
            new PublicDocument("Public Guidelines");

        account.display();
        doc.display();

        SecurityCheck.verifyAccess(account);
        SecurityCheck.verifyAccess(doc);
    }
}

```

Q13

Different Types of Inner Classes in Java

What?

- An inner class is a class defined inside another class.

Why?

- Improves encapsulation (restricts access).
- Helps group related logic together.
- Can access private members of the outer class.

Types of Inner Classes in Java

Type	Definition
Member Inner Class	A class inside another class (without static).
Static Nested Class	A static class inside another class.
Local Inner Class	A class inside a method.
Anonymous Inner Class	A class without a name, used once.

1. Member Inner Class

What?

- A non-static class inside another class.
- Can access all members (including private) of the outer class.
- Use when an inner class needs access to instance members of the outer class.

When you need to encapsulate logic that is tightly coupled with the outer class, like handling employee salary calculations inside an Employee class.

Example: Member Inner Class

2. Static Nested Class

What?

- A static inner class does not require an instance of the outer class.
- Can only access static members of the outer class.
- Use when an inner class does not need access to instance variables of the outer class.
- When you need a helper class that groups related functionality, like a

Database.Connection class managing database connections without needing an outer class instance.

3. Local Inner Class

What?

- A class inside a method.
- Only accessible within the method where it is defined.
- Use a local inner class when you need temporary, method-specific logic, like generating invoices inside an order processing method without exposing the class globally.

Example: Defining a Class Inside a Method

4. Anonymous Inner Class

What?

- A class without a name.
- Defined and instantiated in one step.
- Use when only a single-use implementation is needed.

Example: Implementing an Interface Without Creating a Separate Class

When to Use What?

- Member Inner Class Use when an employee's salary calculation logic is tightly coupled with the Employee class (needs full access to outer class members).
- Static Nested Class Use when a Database.Connection class manages connections independently of the Database instance (does not need instance variables of the outer class).
- Local Inner Class Use when an Order class needs an Invoice generator inside its processOrder() method (used only within a specific method).
- Anonymous Inner Class Use when handling a button click event in a GUI application (needed only once, for immediate use).

```

class Employee {
    private String name;
    private double baseSalary;

    Employee(String name, double baseSalary) {
        this.name = name;
        this.baseSalary = baseSalary;
    }

    // Member Inner Class
    class SalaryCalculator {
        double calculateBonus(double percentage) {
            return baseSalary * percentage / 100;
        }

        void displaySalaryDetails() {
            System.out.println("Employee: " + name);
            System.out.println("Base Salary: $" + baseSalary);
            System.out.println("Bonus: $" + calculateBonus(10));
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Employee emp = new Employee("John Doe", 5000);

        // Creating Inner Class Object
        Employee.SalaryCalculator calculator = emp.new SalaryCalculator();
        calculator.displaySalaryDetails();
    }
}

```

Java

```

class Database {
    private static String url = "jdbc:mysql://localhost:3306/mydb";

    // Static Nested Class
    static class Connection {
        void connect() {
            // Can access only static members
            System.out.println("Connecting to: " + url);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        // No outer class object needed
        Database.Connection connection = new Database.Connection();

        connection.connect();
    }
}

```

Java

```

class Order {
    private String orderId = "ORD123";

    void processOrder() {
        // Local Inner Class
        class Invoice {
            void generateInvoice() {
                System.out.println("Generating invoice for Order ID: " + orderId);
            }
        }

        // Local Inner Class object
        Invoice invoice = new Invoice();
        invoice.generateInvoice();
    }
}

public class Main {
    public static void main(String[] args) {
        Order order = new Order();
        order.processOrder();
    }
}

```

Java

```

interface Fan {
    void turnOn();
}

public class Main {
    public static void main(String[] args) {
        // Anonymous Inner Class
        Fan fan = new Fan() {
            public void turnOn() {
                System.out.println("Fan is turning on...");
            }
        };

        fan.turnOn();
    }
}

```

Q14

How is a reference variable different from a primitive variable?

A reference variable stores the memory location of an object, while a primitive variable directly stores a value.

- Primitive variables are stored in the stack memory.
- Reference variables are stored in the stack, but the objects they refer to are stored in the heap memory.

What happens when you assign a reference variable to another?

When a reference variable is assigned to another, only the memory location (reference) is copied, not the actual object.

Example:

Both cat and nothing now point to the same object in memory.

Since reference variables point to the same object, modifying the object through one reference affects all references pointing to it.

Example:

Since nothing and cat refer to the same object, changing nothing.id also updates cat.id.

Code

```
// Primitive variable stores the actual value 5
int i = 5;

// Reference variable stores the memory location of the object
Animal dog = new Animal(12);
```

Code

```
Animal cat = new Animal(15);
Animal nothing = cat; // Copies reference, not the object
```

Java

```
nothing.id = 10;

// Output: 10
System.out.println(cat.id);
```

CHAPTER 6

Core Java in Depth

17 Questions & Answers

Q1 Why are Wrapper Classes Needed?

What?

- Represents wrappers around primitive data types (int, double, etc.)
- Implemented in java.lang package (e.g., Integer, Double)

Why?

- Allows primitives to be used as objects (e.g., in collections like ArrayList).
Works with collections and other generic classes (List<Integer>, Optional<Double>).
- Provides utility methods (e.g., Integer.parseInt(), Double.valueOf()).
- Supports autoboxing and unboxing, making conversions easier.

Wrapper Classes in Java

Primitive Type

byte

short

int

long

float

double

char

boolean

Example Methods in Wrapper Classes

Wrapper Class	Method	Description
Integer	valueOf (String s)	Converts a string to an Integer object
Integer	parseInt (String s)	Converts a string to an int primitive
Integer	compare (int x, int y)	Compares two int values
Integer	toBinaryString (int n)	Converts an int to a binary string
Double	valueOf (String s)	Converts a string to a Double object
Double	parseDouble (String s)	Converts a string to a double primitive
Double	isNaN (double d)	Checks if the value is NaN (Not a Number)
Boolean	valueOf (String s)	Converts a string to a Boolean object
Boolean	parseBoolean (String s)	Converts a string to a boolean primitive
Character	isDigit (char ch)	Checks if a character is a digit
Character	toUpperCase (char ch)	Converts a character to uppercase
Character	toLowerCase (char ch)	Converts a character to lowercase

Things to Remember

- Autoboxing: Converts a primitive into a wrapper object automatically.
- Unboxing: Converts a wrapper object back into a primitive automatically.
- Immutable: Wrapper class objects cannot be modified after creation.

Java

```
import java.util.ArrayList;
import java.util.List;

public class WrapperExample {
    public static void main(String[] args) {
        // Cannot use List<int>
        List<Integer> numbers = new ArrayList<>();

        numbers.add(10); // Autoboxing (int to Integer)
        numbers.add(20);

        // Unboxing (Integer to int)
        int sum = numbers.get(0) + numbers.get(1);
        System.out.println("Sum: " + sum);
    }
}
```

Code

```
Integer num = 10; // int to Integer
```

Code

```
int value = num; // Integer to int
```

Q2

Best Practices in Using Wrapper Classes

1. Creating Wrapper Objects

Use `valueOf()` instead of `new` (Efficient Memory Usage)

Preferred:

Avoid:

2. Avoid Autoboxing in Loops (Performance Issue)

Autoboxing creates unnecessary objects, leading to performance issues.

Preferred:

Avoid:

3. Use `parseXxx()` for String to Primitive Conversion

Avoids unnecessary object creation

Preferred:

Avoid:

Code

```
// Uses cached objects
Integer num = Integer.valueOf(10);
```

Code

```
// Unnecessary object creation
Integer num = new Integer(10);
```

Code

```
int sum = 0;

for (int i = 0; i < 1000; i++) {
    sum += i; // Uses int (fast)
}

//Even Faster
int sumFunctional = IntStream.range(0, 1000).sum();
```

Code

```
Integer sum = 0;
for (int i = 0; i < 1000; i++) {
    sum += i; // Autoboxing occurs (slow)
}
```

Code

```
int num = Integer.parseInt("123");
```

Code

```
// Deprecated and inefficient
Integer num = new Integer("123");
```

Q3

How does Java optimize memory usage with Integer.valueOf()?

What?

- `Integer.valueOf(int)` caches commonly used integer values.
- Instead of creating a new object, it reuses an existing one from the cache when possible.

Why?

- Reduces memory usage by avoiding unnecessary object creation.
- Improves performance since cached objects are returned instead of allocating new memory.

How?

- Java maintains a cache of Integer objects for values from -128 to 127.
- When `Integer.valueOf(n)` is called within this range, it returns a cached object instead of creating a new one.
- If the number is outside the range, a new object is created.

Example: Cached vs. Non-Cached Integers

Where is this cache defined?

- Java uses an internal cache inside the Integer class.
- The cache is implemented in `IntegerCache` (an inner class of Integer).

Source Code (from Integer class)

This ensures efficient memory usage for frequently used integers.

Cached Values for Wrapper Classes

Wrapper Class

Byte

Short

Integer

Long

Character

Boolean

Float

Double

Java

```
public class IntegerCacheExample {  
    public static void main(String[] args) {  
        Integer a = Integer.valueOf(100); // Cached  
        Integer b = Integer.valueOf(100); // Cached  
  
        System.out.println(a == b); // true (Same cached object)  
  
        Integer x = Integer.valueOf(200); // Not Cached  
        Integer y = Integer.valueOf(200); // Not Cached  
  
        System.out.println(x == y); // false (Different objects)  
    }  
}
```

Code

```
private static class IntegerCache {  
    static final Integer cache[];  
  
    static {  
        // Cache range: -128 to 127  
        cache = new Integer[-(-128) + 127 + 1];  
  
        for (int i = 0; i < cache.length; i++)  
            cache[i] = new Integer(i - 128);  
    }  
}
```

Q4

Why Are Wrapper Classes in Java Immutable?

What is Immutability?

- What? Immutability means an object cannot be modified after it is created.
- Why? Helps with thread safety, caching, and predictable behavior.
- How? Any modification creates a new object instead of changing the existing one.

Example of an Immutable Class in Java:

- Final class prevents subclassing.
- Private final fields prevent changes after initialization.
- No setter methods ensure immutability.

Wrapper Classes are Immutable

- What? Wrapper classes (Integer, Double, Boolean, etc.) are immutable in Java.
- Why?

Caching Mechanism – Frequently used values (Integer.valueOf(10)) are reused, reducing memory usage.

Thread Safety – Threads cannot modify shared values.

Example of Wrapper Class Immutability:

Code

```
final class ImmutableExample {
    private final int value;

    ImmutableExample(int value) {
        this.value = value;
    }

    public int getValue() {
        return value;
    }
}
```

Java

```
public class WrapperImmutableExample {
    public static void main(String[] args) {
        Integer x = 10;

        // Creates a new Integer object,
        //old one is discarded
        x = x + 1;

        System.out.println(x); // Output: 11
    }
}
```

Using String for Immutable Data

- What? Used for storing fixed values like constants, configuration keys, or error messages.
- Example:
- When to Use String class?
 - Fixed values that do not change
 - Avoid for frequent modifications (creates multiple objects)

Using StringBuffer for Thread-Safe Modifications

- What? A mutable string alternative that is thread-safe (synchronized).
- Why? Ensures safe string modification in multi-threaded environments.
- Example:
- When to Use?
 - Multi-threaded modifications
 - Avoid in single-threaded cases (slower than StringBuilder)

Using StringBuilder for Fast String Modifications

- What? A mutable string alternative optimized for performance.
- Why? Faster than StringBuffer because it is not synchronized.
- Example:
- When to Use?
 - Frequent modifications in single-threaded programs
 - Avoid in multi-threaded environments (not thread-safe)

Avoiding Performance Issues with String Concatenation in Loops

Inefficient Approach Using String: Creates 1000+ unnecessary String objects, impacting performance.

Optimized Approach Using StringBuilder: Uses a single StringBuilder object, significantly improving performance.

Comparison

Factor	String	StringBuffer
Immutable Data	YES	NO
Thread Safety	Thread-Safe	Thread-Safe
Performance (Loops)	Slowest	Slower (due to synchronization)
Frequent Changes	Creates new objects	Thread-Safe

Java

```
public class StringExample {
    public static void main(String[] args) {
        String greeting = "Hello";
        greeting += " World"; // Creates a new String object

        System.out.println(greeting); // Output: Hello World
    }
}
```

Java

```
public class StringBufferExample {
    public static void main(String[] args) {

        StringBuffer buffer
            = new StringBuffer("Thread-safe");

        buffer.append(" operations");
        buffer.insert(0, "Ensuring ");
        buffer.reverse();

        System.out.println(buffer);
    }
}
```

Java

```
public class StringBuilderExample {
    public static void main(String[] args) {
        StringBuilder builder
            = new StringBuilder("Efficient");

        builder.append(" string");
        builder.append(" operations");
        builder.delete(0, 4); // Removes "Effe"

        System.out.println(builder);
    }
}
```

Java

```

public class StringLoopExample {
    public static void main(String[] args) {
        String result = "";

        for (int i = 0; i < 1000; i++) {
            // Creates a new String object in every iteration
            result += i;
        }

        System.out.println(result.length());
    }
}

```

Java

```

public class StringBuilderLoopExample {
    public static void main(String[] args) {
        StringBuilder result = new StringBuilder();

        for (int i = 0; i < 1000; i++) {
            result.append(i);
        }

        System.out.println(result.length());
    }
}

```

Q6

Why Are String Classes in Java Immutable?

- What? The String class in Java is immutable, meaning its value cannot be changed after creation.

- Why?

Security – Strings are used in class loading, networking, and security keys. If mutable, an attacker could change "password123" to "password456".

Thread Safety – Since a String object cannot change, it can be shared safely across threads.

String Pooling Optimization – Multiple references to the same string reuse the same object, saving memory.

Example of String Immutability:

- Why does "Hello" remain unchanged? `s.concat(" World")` creates a new String, but `s` still points to "Hello".

Java

```

public class StringImmutableExample {
    public static void main(String[] args) {
        String s = "Hello";

        s.concat(" World"); // Creates a new object
        //does NOT modify `s`

        System.out.println(s); // Output: Hello
    }
}

```

Q7

How do Text Blocks Help?

Example Without Text Blocks:

Same Example With Text Blocks:

Summary:

- Java New Feature: Introduced in JDK 15
- Less Escape Sequences: No more `\n` or `\` for multi-line strings.
- More Readable: Code looks like actual text formatting.
- Auto Alignment: Handles indentation automatically.
- Better for JSON, HTML, SQL: Keeps structure intact.

Code

```

String json = "{\n" +
    "  \"name\": \"John\", \n" +
    "  \"age\": 30\n" +
    "}";

```

JavaScript

```

String json = ""
{
  "name": "John",
  "age": 30
}
"";
//json ==> "{\n  \"name\": \"John\", \n  \"age\": 30\n}"

```

Q8

What is a String Pool?

- What? A special memory area inside the heap where string literals are stored only once.
- Why? Saves memory by reusing the same String object instead of creating duplicates.
- How? When a new string is created using double quotes (""), it is automatically added to the pool.

If the same string already exists, Java reuses the existing reference instead of creating a new object.

Example of String Pool Optimization:

Both s1 and s2 refer to the same "Java" object in the pool, improving memory efficiency.

Heap Memory

- When using `new String("Hello")`, a new object is always created in the heap memory, even if the same value exists in the pool.
- This results in two objects: one in the heap and one in the pool (if not already present).

Example:

Best Practice: Always prefer string literals (`String str = "Hello";`) to save memory and improve performance.

Java

```
public class StringPoolExample {  
    public static void main(String[] args) {  
        String s1 = "Java"; // Stored in String Pool  
        String s2 = "Java"; // Reuses the same object  
        System.out.println(s1 == s2);  
        // Output: true (same reference)  
    }  
}
```

Java

```

public class StringMemoryExample {
    public static void main(String[] args) {

        // Stored in String Constant Pool
        String s1 = "Hello";

        // Reuses the same object
        String s2 = "Hello";

        // Creates a new object in Heap
        String s3 = new String("Hello");

        // true (same reference)
        System.out.println(s1 == s2);

        // false (different objects)
        System.out.println(s1 == s3);
    }
}

```

Q9

How Does intern() Work?

- What? Forces a string to be stored in the String Pool, even if it was created using new.
- Why? Helps reduce duplicate objects and optimize memory.

Example of intern() Method:

- s1 is created outside the pool, but s1.intern() moves it into the pool.
- s2 and s3 now share the same reference, saving memory.

Java

```

public class StringInternExample {
    public static void main(String[] args) {
        String s1 = new String("Hello");
        String s2 = s1.intern(); // Moves to String Pool
        String s3 = "Hello";
        System.out.println(s2 == s3); // Output: true
    }
}

```

Q10

What are the things to be careful about when comparing Strings?

- Using == instead of .equals()
 == compares references, not values.
 Always use .equals() to check content equality.
- Case Sensitivity
 equals() is case-sensitive. Use equalsIgnoreCase() for case-insensitive checks.
- String Pool Behavior
 String literals are stored in the String Constant Pool and can be shared.
 Using new String() creates a new object, even if the same value exists.

Java

```
String s1 = "Hello";
String s2 = new String("Hello");
System.out.println(s1 == s2); // false (different objects)
System.out.println(s1.equals(s2)); // true (same content)
```

Java

```
System.out.println(
    "hello".equalsIgnoreCase("HELLO")); // true
```

Java

```
String a = "Java";
String b = "Java";

// true (same object in pool)
System.out.println(a == b);

String c = new String("Java");

// false (different objects)
System.out.println(a == c);
```

Q11

What Are the Best Practices with Conditionals?

Use else if Instead of Multiple if Statements

Avoid Complex Conditions

Hard to Read (Complex Condition in if)

Improved Readability (Using Meaningful Variables)

Prefer Ternary Operator for Simple Cases

Use switch Instead of Multiple if-else When Applicable

Java

```
if (age < 18) {
    System.out.println("Minor");
} else if (age < 60) {
    System.out.println("Adult");
} else {
    System.out.println("Senior");
}
```

Java

```
if ((age > 18 && hasLicense)
    || (age > 16 && hasPermit && !hasViolations)) {
    System.out.println("Allowed to drive");
}
```

Java

```
boolean isAdultWithLicense = age > 18 && hasLicense;
boolean isTeenWithPermit = age > 16
                        && hasPermit && !hasViolations;

if (isAdultWithLicense || isTeenWithPermit) {
    System.out.println("Allowed to drive");
}
```

Code

```
// Verbose
String result;
if (x > 10) result = "High"; else result = "Low";

// Use ternary
String result = (x > 10) ? "High" : "Low";
```

Java

```
String result = switch (day) {
    //Add other days :)
    case "Monday", "Tuesday" -> "Workday";

    default -> "Weekend";
};

System.out.println(result);
```

Use Enhanced for Loop or Functional Programming for Collections (Faster & Readable)

Avoid Expensive Operations Inside Loops

Use StringBuilder Instead of String for Concatenation

Use break and continue Wisely

Use Parallel Streams for Large Data Sets (Java 8+)

Choose the Right Loop Type

Scenario

Fixed number of iterations

Loop until condition is met

Iterating over a collection

Processing large datasets

Java

```
// Slower: Uses index-based access repeatedly
for (int i = 0; i < list.size(); i++) {
    System.out.println(list.get(i));
}

// Faster: Uses iterator internally
for (String item : list) {
    System.out.println(item);
}

// ALTERNATIVE: Functional Programming
list.forEach(System.out::println);
```

Code


```
// Inefficient: Calls `list.size()` in each iteration
for (int i = 0; i < list.size(); i++) {
    process(list.get(i));
}

// Efficient: Store size in a variable
int size = list.size();
for (int i = 0; i < size; i++) {
    process(list.get(i));
}

// ALTERNATIVE: Functional Programming
list.forEach(this::process);
```

Code

```
// Inefficient: Creates new String objects in each iteration
String result = "";
for (String word : words) {
    result += word; // New object each time
}

// Efficient: Uses a single `StringBuilder` object
StringBuilder result = new StringBuilder();
for (String word : words) {
    result.append(word);
}
```

Java

```
// Inefficient: Loops through all elements
// even after finding the match
for (int num : numbers) {
    if (num == target) {
        System.out.println("Found!");
    }
}

// Efficient: Stops loop once found
for (int num : numbers) {
    if (num == target) {
        System.out.println("Found!");
        break; // Exits loop early
    }
}
```

Java

```
// Parallel processing for large lists
list.parallelStream().forEach(System.out::println);
```

Q13

What Are the Best Practices with Using Arrays?

Use `Arrays.toString()` to Print Arrays

Use Enhanced for Loop or Functional Programming instead of Traditional for Loop

Use Arrays.equals() to Compare Arrays

Use Arrays.deepEquals() for Multidimensional Arrays

Use Arrays.copyOf() for Cloning

Use System.arraycopy() for Fast Copying of values (Faster than looping for large arrays)

Use List Instead of Arrays When Possible

Use Streams for Advanced Array Operations

Java

```
int[] myArray = {1, 2, 3, 4};

// Prints something like [I@1b6d3586
System.out.println(myArray);

// Prints: [1, 2, 3, 4]
System.out.println(Arrays.toString(myArray));
```

Java

```
for (int num : myArray) {
    System.out.println(num);
}
```

Java

```
Arrays.stream(myArray).forEach(System.out::println);
```

Java

```
if (Arrays.equals(array1, array2)) {
    System.out.println("Arrays are equal");
}
//If you compare arrays using ==,
//it checks only the memory references, NOT the actual contents.
```

Java

```
if (Arrays.deepEquals(array1, array2)) {  
    System.out.println("2D Arrays are equal");  
}
```

Code

```
int[] original = {1, 2, 3, 4, 5};  
  
int[] copy = Arrays.copyOf(original, original.length);  
  
//[1, 2, 3]  
int[] partialCopy = Arrays.copyOf(original, 3);  
  
//[1, 2, 3, 4, 5, 0, 0]  
int[] expanded = Arrays.copyOf(original, 7);  
  
//Creates a new array No modification of the original.  
//Simple way to resize arrays
```

Code

```
//dest is an existing array  
System.arraycopy(src, 0, dest, 0, src.length);  
//Can copy from any index  
//Faster (native implementation)
```

Code

```
//Allows Adding/Removing Elements  
List<String> list = new ArrayList<>(Arrays.asList(array));
```

Code

```
int sum = Arrays.stream(myArray).sum();
```

Q14

How does Local Variable Type Inference Help? (Java 10)

Example:

Advantages

- Less Code: Reduces redundancy in variable declarations.
- Improves Readability: Focus on logic, not type details.
- Flexible: Works with generics, loops, and complex types.
- Compiler Safety: Still enforces strong typing at compile-time.

Code

```
// ArrayList<String> numbers = new ArrayList<>();

// No need to repeat ArrayList<String>
var numbers = new ArrayList<String>();

// Inferred as HashMap<Integer, String>
var map = new HashMap<Integer, String>();
```

Q15

How do Records improve the conciseness of Java code?

- Records eliminate boilerplate code in Java Beans by automatically generating:
 - Constructor
 - Getters (accessors)
 - toString(), equals(), and hashCode()
- Makes code more readable and maintainable.

Before Records (Verbose Java Class)

With Records (Concise and Readable)

Java

```

public class Person {
    private final String name;
    private final String email;
    private final String phoneNumber;

    public Person(
        String name, String email,
        String phoneNumber) {
        this.name = name;
        this.email = email;
        this.phoneNumber = phoneNumber;
    }

    public String getName() {
        return name;
    }
    public String getEmail() {
        return email;
    }
    public String getPhoneNumber() {
        return phoneNumber;
    }

    @Override
    public boolean equals(Object obj) {
        /* Implementation */
    }
    @Override
    public int hashCode() {
        /* Implementation */
    }
    @Override
    public String toString() {
        /* Implementation */
    }
}

```

Java

```

public record Person(
    String name, String email,
    String phoneNumber) {}

//You Can Write Code like this!
Person person = new Person(
    "John Doe",
    "[email protected]",
    "1234567890");

// Auto-generated toString()
System.out.println(person);

// No need for getName()
System.out.println(person.name());

// No need for getEmail()
System.out.println(person.email());

```

Q16

What are the best practices with Records?

Use Records for Immutable Data

- Best suited for DTOs (Data Transfer Objects) and configuration objects .

Add Custom Validation in Compact Constructors

- Use a compact constructor to enforce field constraints. (runs automatically during object creation, validating name. If name is null or blank, it throws an exception.)

Use Methods for Additional Functionality

- Records can contain instance methods for derived values.

Use Static Fields and Methods When Needed

- Records do NOT allow instance variables, but you can use static fields.

Code

```
public record UserDTO(String username, String role) {}
```

Code

```
public record Person(String name, String email) {  
    public Person {  
        if (name == null || name.isBlank()) {  
            throw new IllegalArgumentException(  
                "Name cannot be empty");  
        }  
    }  
}
```

Code

```
public record Rectangle(int width, int height) {  
    public int area() {  
        return width * height;  
    }  
}
```

Java

```
Rectangle rect = new Rectangle(5, 10);
System.out.println(rect.area()); // Output: 50
```

Code

```
public record Car(String model, int year) {
    static String manufacturer = "Tesla";
    public static String getManufacturer() {
        return manufacturer;
    }
}
```

Q17

What are the things that you should be careful about when using Records?

Records Are Immutable

- Fields cannot be changed after object creation.
- If a class needs mutable state or complex behavior, use a regular class instead.

Cannot Add Instance Variables

- Fields must be declared in the record header.

Inheritance is Not Supported

- Records extend `java.lang.Record`.
- Records cannot extend other classes (but can implement interfaces).

When to Use Records?

- Data Models (e.g., `UserDTO`)
- Configurations (e.g., `AppSettings`)
- Lightweight Domain Objects (e.g., `Rectangle`, `Person`)

When NOT to Use Records?

- Mutable Objects (e.g., BankAccount)
- Complex Business Logic
- Inheritance-Based Hierarchies

Good to Know: Simple Record Pattern Matching

Good to Know: Nested Record Pattern Matching

Summary:

- Records introduced in JDK 16
- Advanced Pattern Matching in JDK 21

Code

```
Person p = new Person("Alice", "[email protected]");
// p.name = "Bob"; // Compilation Error
```

Java

```
public class BankAccount {
    private double balance;

    public void deposit(double amount) {
        balance += amount;
    }

    public void withdraw(double amount) {
        balance -= amount;
    }
}
```

Code

```
public record Product(String name, double price) {
    // private int discount;
    // ERROR - Cannot add instance variables
}
```

Code

```
public record Employee(
    String name, int id) /* extends Person */ {}
// ERROR
```

Java


```
record Course(int id, String name) {}

public static void processCourse(Object obj) {
    if (obj instanceof Course(int id, String name)) {
        System.out.println(
            "Course ID: " + id + ", Name: " + name);
    }
}
```

Java

```
record Person(String name, int age) {}
record Address(String street, String city) {}
record Contact(Person person, Address address) {}

public static void processContact(Object obj) {
    if (obj instanceof Contact(
        Person(var name, var age),
        Address(var street, var city))) {
        System.out.println(name + " lives in " + city);
    }
}
```

CHAPTER 7

Functional Programming in Java

16 Questions & Answers

Q1

How does Functional Programming differ from Imperative Style?

What is Imperative Style?

- Uses step-by-step logic to solve a problem.
- Focuses on how to achieve the result.

What is Functional Programming?

- Uses declarative code instead of loops.
- Focuses on what needs to be done, not how.

Example 1: Imperative Code (Using Loops)

Explicitly loops through the list and keeps a running sum.

Example 2: Functional Programming (Using Streams)

Uses `stream()` and `reduce()` to compute the sum in a single line.

Key Differences

Feature	Imperative Style
Style	Step-by-step logic
Looping	Uses for/while loops
State Management	Uses mutable variables (sum)
Code Readability	More verbose

Java

```
import java.util.List;

public class FP02Imperative {
    private static int addListImperative(List<Integer> numbers) {
        int sum = 0;
        for (int number : numbers) {
            sum += number;
        }
        return sum;
    }
}
```

Java

```
import java.util.List;

public class FP02Functional {
    private static int addListFunctional(List<Integer> numbers) {
        return numbers.stream().reduce(0, Integer::sum);
    }
}
```

Q2

What are the core concepts of Functional Programming? Provide an example.

1. Stream

- What? A sequence of elements that supports functional-style operations.
- Why? Allows processing of data in a declarative and efficient way.
- Example:

`courses.stream()` creates a stream from the list of courses.

This stream enables pipeline processing without modifying the original list.

2. Lambda Function

- What? An anonymous function that represents a short piece of code.
- Why? Makes code concise and readable.
- Example:

`course -> course.toUpperCase()` is a lambda function that converts each course name to uppercase.

It eliminates the need for writing a separate method.

3. Method Reference

- What? A shorthand for calling a method using `ClassName::methodName` or `object::methodName`.
- Why? Further simplifies lambda expressions when an existing method can be used directly.
- Example:
`System.out::println` is a method reference that replaces `x -> System.out.println(x)`.

Java

```
courses.stream()  
    .map(course -> course.toUpperCase())  
    .forEach(System.out::println);
```

Q3

What are different ways to create Streams in Java?

1. From Collections (Lists, Sets, Maps)

- What? Convert collections into streams.
- Why? Allows functional operations on lists, sets, and maps.
- How? Use `.stream()`

2. Using `Stream.of()`

- What? Create a stream from individual elements.
- Why? Useful for small, fixed data sets.
- How? Use `Stream.of()`.

3. Using `Arrays.stream()`

- What? Convert primitive arrays into streams.
- How? Use `Arrays.stream()`.

4. Using `Stream.builder()`

- What? Manually build a stream.
- Why? Allows adding elements dynamically.
- How? Use `Stream.builder()`.

5. Using `Stream.generate()`

- What? Create an stream of generated values.
- Why? Useful for lazy evaluation or dynamic data.
- How? Use `Stream.generate()`.

6. Using `Stream.iterate()`

- What? Create a stream by applying a function.
- Why? Useful for sequences (e.g., even numbers).
- How? Use `Stream.iterate()`.

7. Using Files (`Files.lines()`)

- What? Read a file as a stream of lines.
- Why? Helps process large files efficiently.
- How? Use `Files.lines()`.

8. Using `IntStream.range()`

- What? Create a stream of numbers within a range.
- Why? Useful for generating sequences.
- How? Use `IntStream.range()`

Comparison Table

Method
<code>collection.stream()</code>
<code>Stream.of()</code>
<code>Arrays.stream()</code>
<code>Stream.builder()</code>
<code>Stream.generate()</code>
<code>Stream.iterate()</code>
<code>Files.lines()</code>
<code>IntStream.range()</code>

```
List<Integer> numbers = List.of(1, 2, 3, 4, 5);
Stream<Integer> numberStream = numbers.stream();
numberStream.forEach(System.out::println);
```

Code

```
//{Java=100, Python=80}
Map<String, Integer> courses
    = Map.of("Java", 100, "Python", 80);

//[ "Java", "Python"]
Stream<String> keyStream = courses.keySet().stream();

//[100, 80]
Stream<Integer> valueStream = courses.values().stream();

//{Java=100, Python=80}
Stream<Map.Entry<String, Integer>> entryStream
    = courses.entrySet().stream();
```

Java

```
Stream<String> stream = Stream.of("Apple", "Banana", "Cherry");
stream.forEach(System.out::println);
```

Java

```
int[] numbers = {10, 20, 30, 40, 50};
IntStream intStream = Arrays.stream(numbers);
intStream.forEach(System.out::println);
```

Java

```
Stream.Builder<String> builder = Stream.builder();
builder.add("Java").add("Python").add("JavaScript");
Stream<String> stream = builder.build();
stream.forEach(System.out::println);
```

Java

```
Stream<Double> randomNumbers
    = Stream.generate(Math::random).limit(5);

randomNumbers.forEach(System.out::println);
```

Java

```
Stream<Integer> evenNumbers
    = Stream.iterate(0, n -> n + 2).limit(5);

evenNumbers.forEach(System.out::println);
```

Java

```
Stream<String> fileStream
    = Files.lines(Paths.get("file.txt"));

fileStream.forEach(System.out::println);
```

Java

```
// Output: 1234
IntStream.range(1, 5).forEach(System.out::print);

// Output: 12345
IntStream.rangeClosed(1, 5).forEach(System.out::print);
```

Q4

What operations can you apply to a Java Stream?

1. Intermediate Operations

- What? Transform or filter elements and return a new stream.
- Lazy Execution: These operations do not execute until a terminal operation is called.

Operation	Purpose
filter()	Retains elements that satisfy a condition.
map()	Transforms each element in the stream.
distinct()	Removes duplicate elements.
sorted()	Sorts elements in natural or custom order.
limit(n)	Takes only the first n elements.
skip(n)	Skips the first n elements.
takeWhile(predicate)	Takes elements while the condition is true, stops at first failure.
dropWhile(predicate)	Drops elements while the condition is true, then takes the rest.

2. Terminal Operations

- What? Consumes the stream and produces a result.
- Trigger Execution: The stream is processed when a terminal operation is called.

Operation	Purpose
forEach()	Iterates over each element and performs an action.
collect()	Gathers elements into a collection (List, Set, Map).
reduce()	Combines elements to produce a single result.
count()	Returns the number of elements in the stream.
findFirst()	Retrieves the first element of the stream.
findAny()	Retrieves any element (non-deterministic).
anyMatch()	Returns true if any element matches the condition.
allMatch()	Returns true if all elements match the condition.
noneMatch()	Returns true if no element matches the condition.

Q5

What should you keep in mind when working with Streams?

1. Streams Do Not Store Data

- What? Streams process data on the fly without storing it.
- Why? This makes them memory efficient, especially for large datasets.
- Example:

2. Streams Are Lazy

- What? Intermediate operations (filter, map, etc.) are not executed immediately.
- Why? Streams only process data when a terminal operation (e.g., forEach, collect, reduce) is called.

- Example:
- Intermediate operations (like filter() and map()) are lazy.
- NOT executed immediately when encountered

Instead, Java "chains" these operations and waits for a terminal operation (like findFirst()) to trigger their execution.

3. Streams Are Immutable

- What? Stream operations do not modify the original collection.
- Why? This ensures functional purity and avoids side effects.
- Example:

4. Streams Cannot Be Reused

- What? A stream is consumed once a terminal operation is called.
- Why? You need to create a new stream if you need to process the data again.
- Example:

Code

```
Stream<Integer> numberStream
    = List.of(1, 2, 3, 4, 5).stream();
```

Java

```
List.of(1, 2, 3, 4, 5).stream().map(n -> {
    System.out.println("Mapping: " + n);
    return n * 2;
}); // No output because there is no terminal operation
```

Code

```
List<Integer> numbers = List.of(1, 2, 3);

// Original list remains unchanged
numbers.stream().map(n -> n * 2);
```

Java

```
Stream<Integer> stream = numbers.stream();

// Works
stream.forEach(System.out::println);

// ERROR: Stream already consumed!
stream.forEach(System.out::println);
```

Q6

Can you provide some examples of Lambda Functions?

Lambda expressions are anonymous functions that simplify writing code by reducing boilerplate.

Syntax of Lambda Functions

- Parameters: The input to the function.
- Arrow (->): Separates parameters from the function body.
- Body: Contains the logic of the function.

Filtering Even Numbers Using Lambda

Mapping Course Names to Uppercase

Sorting Courses by Length of Name

Using UnaryOperator to Triple a Number

Code

```
(parameters) -> { body }
```

Java

```
numbers.stream()
    .filter(number -> number % 2 == 0)
    .forEach(System.out::println);
```

Java

```
courses.stream()
    .map(course -> course.toUpperCase())
    .forEach(System.out::println);
```

Java

```
courses.stream()  
    .sorted((c1, c2) -> c1.length() - c2.length())  
    .forEach(System.out::println);
```

Java

```
UnaryOperator<Integer> triple = x -> x * 3;  
System.out.println(triple.apply(10)); // Output: 30
```

Q7

When should you use Method References instead of Lambda Functions?

Method references are shortcuts for writing lambda expressions that call an existing method. They use the syntax `ClassName::methodName` or `object::methodName`.

1. Calling a Static Method

- When? When an existing static method can be directly referenced without modifications.
- Why? It improves readability and eliminates redundant lambda expressions.

- Example 1: Java Utility Method

Using Method Reference

Equivalent Lambda Expression

- Example 2: Custom Utility Method

Using Method Reference

Equivalent Lambda Expression

2. Calling an Instance Method

- When? When calling an instance method on each element.
- Using Method Reference
- Equivalent Lambda Expression

3. When creating new objects - Constructor Reference

- When? When creating new objects inside a stream.
- Using Constructor Reference
- Equivalent Lambda Expression

Java

```
courses.stream()  
    .forEach(System.out::println);
```

Java

```
courses.stream()  
    .forEach(course -> System.out.println(course));
```

Java

```
courses.stream()  
    .forEach(System.out::println);
```

Java

```
courses.stream()  
    .forEach(course -> System.out.println(course));
```

Java

```
numbers.stream()  
    .filter(FP01Functional::isEven)  
    .forEach(System.out::println);
```

Java

```
numbers.stream()  
    .filter(number -> FP01Functional.isEven(number))  
    .forEach(number -> System.out.println(number));
```

Java

```
numbers.stream()  
    .filter(FP01Functional::isEven)  
    .forEach(System.out::println);
```

Java

```
numbers.stream()  
    .filter(number -> FP01Functional.isEven(number))  
    .forEach(number -> System.out.println(number));
```

Java

```
courses.stream()  
    .map(String::toUpperCase)  
    .forEach(System.out::println);
```

Java

```
courses.stream()  
    .map(course -> course.toUpperCase())  
    .forEach(course -> System.out.println(course));
```

Code

```
Supplier<String> stringSupplier = String::new;
```

Code

```
Supplier<String> stringSupplier = () -> new String();
```

Q8

What are Functional Interfaces? Provide some examples.

- Core component of Java's functional programming approach
- Contains exactly one abstract method
- Allow passing behavior as an argument using lambda expressions
- Examples:

Consumer processes an input.

Function transforms data and returns the result.

Predicate tests a condition.

BiFunction combines two inputs into a result.

Java

```

@FunctionalInterface
public interface Consumer<T> {
    void accept(T t);
}

Consumer<Integer> printOdd = number -> {
    if (number % 2 != 0) {
        System.out.println(number);
    }
};

//What happens in the background?
public class PrintOdd implements Consumer<Integer> {
    @Override
    public void accept(Integer number) {
        if (number % 2 != 0) {
            System.out.println(number);
        }
    }
}

```

Java

```

@FunctionalInterface
public interface Function<T, R> {
    R apply(T t);
}

Function<Integer, Integer> squareFunction
    = number -> number * number;

//What happens in the background?
public class SquareFunction
    implements Function<Integer, Integer> {
    @Override
    public Integer apply(Integer number) {
        return number * number;
    }
}

```

Java

```

@FunctionalInterface
public interface Predicate<T> {
    boolean test(T t);
}

Predicate<Integer> isEven
    = number -> number % 2 == 0;

//What happens in the background?
public class IsEvenPredicate implements Predicate<Integer> {
    @Override
    public boolean test(Integer number) {
        return number % 2 == 0;
    }
}

```

Java

```

@FunctionalInterface
public interface BiFunction<T, U, R> {
    R apply(T t, U u);
}

BiFunction<Integer, Integer, Integer> maxFunction
    = (x, y) -> x > y ? x : y;

//What happens in the background?
public class MaxFunction implements BiFunction<Integer, Integer, Integer> {
    @Override
    public Integer apply(Integer x, Integer y) {
        return x > y ? x : y;
    }
}

```

Java

```

@FunctionalInterface
public interface Consumer<T> {
    void accept(T t);
}

Consumer<Integer> printOdd = number -> {
    if (number % 2 != 0) {
        System.out.println(number);
    }
};

//What happens in the background?
public class PrintOdd implements Consumer<Integer> {
    @Override
    public void accept(Integer number) {
        if (number % 2 != 0) {
            System.out.println(number);
        }
    }
}

```

Java

```

@FunctionalInterface
public interface Function<T, R> {
    R apply(T t);
}

Function<Integer, Integer> squareFunction
    = number -> number * number;

//What happens in the background?
public class SquareFunction
    implements Function<Integer, Integer> {
    @Override
    public Integer apply(Integer number) {
        return number * number;
    }
}

```

Java


```

@FunctionalInterface
public interface Predicate<T> {
    boolean test(T t);
}

Predicate<Integer> isEven
    = number -> number % 2 == 0;

//What happens in the background?
public class IsEvenPredicate implements Predicate<Integer> {
    @Override
    public boolean test(Integer number) {
        return number % 2 == 0;
    }
}

```

Java

```

@FunctionalInterface
public interface BiFunction<T, U, R> {
    R apply(T t, U u);
}

BiFunction<Integer, Integer, Integer> maxFunction
    = (x, y) -> x > y ? x : y;

//What happens in the background?
public class MaxFunction implements BiFunction<Integer, Integer, Integer> {
    @Override
    public Integer apply(Integer x, Integer y) {
        return x > y ? x : y;
    }
}

```

Q9

What are some key Functional Interfaces, and how do they work?

1. Predicate (Conditional Testing)

- Purpose: Tests a condition on input and returns a boolean.
- Example: Checking if a number is even.
- Using Lambda (Preferred)
- Equivalent Anonymous Class
- Usage in Streams

2. Function (Mapping Input to Output)

- Purpose: Accepts one argument and returns a transformed value.
- Example: Squaring a number.
- Using Lambda (Preferred)
- Equivalent Anonymous Class
- Usage in Streams

3. Consumer (Processing Without Return)

- Purpose: Accepts an input but does not return anything.
- Example: Printing values.
- Using Lambda (Preferred)
- Equivalent Anonymous Class
- Usage in Streams

4. BinaryOperator (Two Inputs, Same Type)

- Purpose: Takes two arguments and returns a result of the same type.
- Example: Summing two numbers.
- Using Lambda
- Using Method Reference
- Equivalent Anonymous Class
- Usage in Reduce Operation

5. Supplier (Generating Values Without Input)

- Purpose: Provides a value without any input.
- Example: Returning a random number.
- Using Lambda (Preferred)
- Equivalent Anonymous Class
- Usage

6. UnaryOperator (Single Input, Same Type)

- Purpose: Takes one argument and returns a result of the same type.
- Example: Tripling a number.
- Using Lambda (Preferred)
- Equivalent Anonymous Class
- Usage in Streams

7. BiPredicate (Conditional Testing with Two Inputs)

- Purpose: Tests two inputs and returns a boolean.
- Example: Checking if two numbers are equal.
- Using Lambda (Preferred)
- Equivalent Anonymous Class
- Usage

8. BiFunction (Two Inputs, One Output)

- Purpose: Accepts two inputs and returns one output.
- Example: Concatenating strings.
- Using Lambda (Preferred)
- Equivalent Anonymous Class
- Usage

9. BiConsumer (Processing Two Inputs Without Return)

- Purpose: Accepts two inputs but does not return anything.
- Example: Printing a key-value pair.
- Using Lambda (Preferred)
- Equivalent Anonymous Class
- Usage

Summary

Functional Interface	Purpose
Predicate	Returns true or false
Function<T, R>	Maps input T to output R
Consumer	Processes input T without return
BinaryOperator	Takes two T inputs, returns T
Supplier	Generates a T value
UnaryOperator	Takes T, returns T
BiPredicate<T, U>	Tests two inputs, returns boolean
BiFunction<T, U, R>	Takes two inputs, returns R
BiConsumer<T, U>	Processes two inputs without return

Code

```
Predicate<Integer> isEvenPredicate = x -> x % 2 == 0;
```

Java

```
Predicate<Integer> isEvenPredicate2 = new Predicate<>() {
    @Override
    public boolean test(Integer x) {
        return x % 2 == 0;
    }
};
```

Java

```
numbers.stream()
    .filter(isEvenPredicate)
    .forEach(System.out::println);
```

Code

```
Function<Integer, Integer> square = x -> x * x;
```

Java

```
Function<Integer, Integer> square = new Function<>() {
    @Override
    public Integer apply(Integer x) {
        return x * x;
    }
};
```

Java

```
numbers.stream()
    .map(squareFunction)
    .forEach(System.out::println);
```

Java

```
Consumer<Integer> sysoutConsumer = System.out::println;
```

Java

```
Consumer<Integer> sysoutConsumer2 = new Consumer<>() {
    @Override
    public void accept(Integer x) {
        System.out.println(x);
    }
};
```

Code

```
numbers.stream().forEach(sysoutConsumer);
```

Code

```
BinaryOperator<Integer> sum = (a, b) -> a + b;
```

Code

```
BinaryOperator<Integer> sum = Integer::sum;
```

Java

```
BinaryOperator<Integer> sumBinaryOperator2  
    = new BinaryOperator<>() {  
    @Override  
    public Integer apply(Integer a, Integer b) {  
        return Integer.sum(a, b);  
    }  
};
```

Java

```
int sum = numbers.stream().reduce(0, sumBinaryOperator);  
System.out.println(sum);
```

Code

```
Supplier<Integer> randomIntegerSupplier  
    = () -> new Random().nextInt(100);
```

Java

```
Supplier<Integer> randomIntegerSupplier2 = new Supplier<>() {  
    @Override  
    public Integer get() {  
        return new Random().nextInt(100);  
    }  
};
```

Java

```
System.out.println(randomIntegerSupplier.get());
```

Code

```
UnaryOperator<Integer> tripleOperator = x -> 3 * x;
```

Java

```
UnaryOperator<Integer> tripleOperator2 = new UnaryOperator<>() {
    @Override
    public Integer apply(Integer x) {
        return 3 * x;
    }
};
```

Java

```
numbers.stream()
    .map(tripleOperator)
    .forEach(System.out::println);
```

Code

```
BiPredicate<Integer, Integer> isEqual = (a, b) -> a.equals(b);
```

Java

```
BiPredicate<Integer, Integer> isEqual2 = new BiPredicate<>() {
    @Override
    public boolean test(Integer a, Integer b) {
        return a.equals(b);
    }
};
```

Java

```
System.out.println(isEqual.test(5, 5)); // Output: true
```

Code

```
BiFunction<String, String, String> concat
    = (a, b) -> a + " " + b;
```

Java

```
BiFunction<String, String, String> concat2 = new BiFunction<>() {
    @Override
    public String apply(String a, String b) {
        return a + " " + b;
    }
};
```

Java

```
// Output: Hello World
System.out.println(concat.apply("Hello", "World"));
```

Java

```
BiConsumer<String, Integer> printKeyValue =  
    (key, value) -> System.out.println(key + ": " + value);
```

Java

```
BiConsumer<String, Integer> printKeyValue2 = new BiConsumer<>() {  
    @Override  
    public void accept(String key, Integer value) {  
        System.out.println(key + ": " + value);  
    }  
};
```

Code

```
printKeyValue.accept("Java", 8); // Output: Java: 8
```

Q10

Why were Primitive Functional Interfaces introduced in Java?

Overview

- What? Special interfaces for primitive data types like int, double, long.
- Why? Avoids boxing/unboxing overhead, making code faster and memory-efficient.
- Example: Use IntBinaryOperator instead of BinaryOperator<Integer>.
- Before (Less Efficient)
- After (Better Performance)
- Key Benefits
 - Faster Execution: No need to wrap/unwrap primitive values.
 - Less Memory Usage: No Integer, just int!
 - Cleaner Code: Works directly with primitives, reducing complexity.

Other Primitive Interfaces

Interface	Purpose
IntPredicate	Checks conditions
IntFunction<R>	Maps int to result
IntConsumer	Accepts int, no return
IntBinaryOperator	Operates on two int

Other Examples: LongPredicate, LongFunction, LongConsumer, DoublePredicate, DoubleFunction, DoubleConsumer and a lot of others!

Code

```
BinaryOperator<Integer> sum = (x, y) -> x + y;
```

Code

```
IntBinaryOperator sum = (x, y) -> x + y;
```

Q11

When Do You Use Collectors.groupingBy?

- What? Groups elements of a stream into a Map based on a key.
- Why? Simplifies grouping, counting, and aggregating data in Java Streams.

Group Courses by Category

Output:

Count Courses in Each Category

Output:

Find Highest Rated Course in Each Category

Output:

Get List of Course Names per Category

Output:

Code


```

public record Course(
    String name, String category,
    int reviewScore, int students) {}

List<Course> courses = List.of(
    new Course("AWS", "Cloud", 95, 200000),
    new Course("Azure", "Cloud", 90, 150000),
    new Course("Docker", "Cloud", 85, 180000),
    new Course("Kubernetes", "Cloud", 88, 170000),
    new Course("Spring", "Framework", 97, 220000),
    new Course("Spring Boot", "Framework", 93, 210000),
    new Course("Microservices", "Microservices", 98, 250000),
    new Course("FullStack", "FullStack", 96, 230000)
);

```

Java

```

Map<String, List<Course>> coursesByCategory = courses.stream()
    .collect(Collectors.groupingBy(Course::category));

System.out.println(coursesByCategory);

```

Code

```

//Simplified only to show course name
{Cloud=[AWS, Azure, Docker, Kubernetes],
Framework=[Spring, Spring Boot],
Microservices=[Microservices],
FullStack=[FullStack]}

```

Java

```

Map<String, Long> courseCountByCategory = courses.stream()
    .collect(Collectors.groupingBy(
        Course::category, Collectors.counting()));

System.out.println(courseCountByCategory);

```

Code

```

{Cloud=4, Framework=2, Microservices=1, FullStack=1}

```

Java

```

Map<String, Optional<Course>> highestRatedCourseByCategory = courses.stream()
    .collect(Collectors.groupingBy(
        Course::category,
        Collectors.maxBy(Comparator.comparing(Course::reviewScore))
    ));

System.out.println(highestRatedCourseByCategory);

```

Code

```
{Cloud=Optional[AWS],
Framework=Optional[Spring],
Microservices=Optional[Microservices],
FullStack=Optional[FullStack]}
```

Java

```
Map<String, List<String>> courseNamesByCategory = courses.stream()
    .collect(Collectors.groupingBy(
        Course::category,
        Collectors.mapping(Course::name, Collectors.toList())
    ));

System.out.println(courseNamesByCategory);
```

Code

```
{Cloud=[AWS, Azure, Docker, Kubernetes],
Framework=[Spring, Spring Boot],
Microservices=[Microservices],
FullStack=[FullStack]}
```

Q12

How does the Optional class help handle missing values?

- What? A container that may or may not hold a value.
- Why? Avoids NullPointerException by explicitly handling missing values.
- Example:

Method	Purpose
Optional.of(value)	Creates an Optional with a non-null value
Optional.ofNullable(value)	Creates an Optional (null safe)
Optional.empty()	Creates an empty Optional
isPresent()	Checks if a value is present
ifPresent(Consumer)	Executes code if value exists
orElse(defaultValue)	Returns value or default if empty
orElseGet(Supplier)	Returns value or calls supplier function
orElseThrow()	Throws an exception if value is missing

Java

```
// Safe handling of null
Optional<String> optional
    = Optional.ofNullable(getValueFromDatabase());

optional.ifPresent(System.out::println);
```

Q13

How do Parallel Streams enhance performance in Java?

- **Modern multi-core processors:** Today's computers have multiple cores.
They can run different tasks at the same time.
- **Traditional Programming:** Old-style coding uses loops and shared variables.
Why is this a problem? Many threads need to share the same data (e.g., a sum variable).
This makes parallel execution hard.
- **Functional Programming:** Uses stateless streams.
Why is this good? No shared data Easy to run in parallel.
- **How does Java parallelize streams?**
parallel() tells Java to use multiple cores.
Java splits the stream into smaller parts.
Each part runs on a different core.
Finally, Java combines all results.

How Does It Work?

- **Creates a Stream of Longs** LongStream.range(0, 1000000000L) generates numbers from 0 to 999,999,999.
- **Enables Parallel Processing** .parallel() splits the range into chunks and processes them across multiple cores.
- **Calculates the Sum Efficiently** .sum() combines results from all threads.

parallel() vs parallelStream()

Feature	parallel()
Applied On	Any existing stream
Purpose	Converts a sequential stream to parallel
When to Use?	When you already have a stream
Example	list.stream().parallel()

Example - Using parallel()

Example - Using parallelStream()

Code

```
//Traditional
int sum = 0;
for (int number : numbers) {
    sum += number;
}
return sum;
```

Code

```
//Traditional
int sum = 0;
for (int number : numbers) {
    sum += number;
}
return sum;
```

Code

```
//Functional
return numbers.stream().reduce(0, Integer::sum);
```

Code

```
//Functional
return numbers.stream().reduce(0, Integer::sum);
```

Code

```
// Sequential
long sum = LongStream.range(0, 1000000000L)
    .sum();

// Parallel on Existing Streams
long parallelSum = LongStream.range(0, 1000000000L)
    .parallel()
    .sum();
```

Code

```
// Sequential
long sum = LongStream.range(0, 1000000000L)
    .sum();

// Parallel on Existing Streams
long parallelSum = LongStream.range(0, 1000000000L)
    .parallel()
    .sum();
```

Java

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
numbers.stream()
    .parallel()
    .forEach(System.out::println);
```

Java

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
numbers.parallelStream()
    .forEach(System.out::println);
```

Q14

In what ways does Functional Programming simplify Java code?

1. Creating a Thread

- Before FP:
- With FP (Lambda Expression): Shorter and more readable.

2. Creating a Comparator

- Before FP:
- With FP: Uses a lambda function, reducing code complexity.

3. Listing Files in a Directory

- Before FP: Uses a for-loop.
- With FP: Uses Files.list() with method reference.

4. Filtering & Transforming a List

- Before FP: Uses loops and conditionals.
- With FP: Uses stream().filter().map().

Java

```
// Traditional way (Verbose)
Thread thread1 = new Thread(new Runnable() {
    @Override
    public void run() {
        System.out.println("Thread running...");
    }
});
thread1.start();
```

Java

```
// Functional way (Lambda)
Thread thread2 = new Thread(
    () -> System.out.println("Thread running..."));
thread2.start();
```

Java

```
import java.util.*;

List<String> names = Arrays.asList("John", "Alice", "Bob");

// Traditional way
Collections.sort(names, new Comparator<String>() {
    @Override
    public int compare(String s1, String s2) {
        return s1.compareTo(s2);
    }
});
```

Code

```
// Functional way (Lambda)
names.sort((s1, s2) -> s1.compareTo(s2));
```

Java

```
import java.nio.file.*;

Path path = Paths.get(".");

// Traditional way
File[] files = new File(".").listFiles();
for (File file : files) {
    System.out.println(file.getName());
}
```

Java

```
// Functional way (Streams)
Files.list(path).forEach(System.out::println);
```

Code

```
List<String> words = Arrays.asList("apple", "banana", "cherry");
// Traditional way
List<String> ucWords = new ArrayList<>();
for (String word : words) {
    if (word.length() > 5) {
        uppercaseWords.add(word.toUpperCase());
    }
}
```

Code

```
// Functional way (Streams)
List<String> ucWordsFp = words.stream()
    .filter(word -> word.length() > 5)
    .map(String::toUpperCase)
    .toList();
```

Q15

How do Higher-Order Functions, Behavior Parameterization, and First-Class Functions differ?

To be frank, they are all very similar concepts!

Quick Comparison

Concept

Higher-Order Functions

Behavior Parameterization

Functions as First-Class Citizens

Example: Returning a Function

- This function returns a predicate based on a cutoff value.
- This allows dynamic filtering without hardcoding conditions.
- The returned predicate is then applied to a stream.

Example: Passing a Function

- This method accepts a predicate as an argument, allowing flexible filtering logic.
- Different conditions can be applied without modifying the method, making the code more reusable.

Code

```

public static Predicate<Course>
    createPredicate(int cutoff) {
    return course ->
        course.getReviewScore() > cutoff;
    }

// Usage
Predicate<Course> highScore = createPredicate(95);

List<Course> filtered = courses.stream()
    .filter(highScore)
    .collect(Collectors.toList());

```

Java

```

public static void filterAndPrint(
    List<Integer> numbers,
    Predicate<Integer> predicate) {

    numbers.stream()
        .filter(predicate)
        .forEach(System.out::println);

}

// Usage
filterAndPrint(numbers, x -> x % 2 == 0);
filterAndPrint(numbers, x -> x % 2 != 0);
filterAndPrint(numbers, x -> x % 3 == 0);

```

Q16

What are the benefits of using Functional Programming?

What?

- Computation is treated as the evaluation of mathematical functions
- Core idea: focus on "what to do" rather than "how to do it."

Key Principles:

- First-Class Functions: Functions are treated as values, meaning they can be passed as arguments, returned from other functions, or assigned to variables.
- Immutability: Avoids modifying variables or data.
- No Side Effects: Functions do NOT affect other parts of the program (e.g., modifying global variables).

Why Use Functional Programming?

- **Declarative Style** Focuses on what needs to be done, not how, making code cleaner and easier to read.
- **Immutability** Data is not modified, reducing unexpected side effects and making debugging easier.
- **Concise Code** Less boilerplate compared to traditional loops and conditionals.
- **Thread Safety** No shared state means safer parallel execution without race conditions.
- **Parallel Processing** Works well with streams and multi-core processors, improving performance.

Code

```
List<String> transformedNumbers = numbers.stream()  
    .map(x -> x * x) // Step 1: Square each number  
    .map(x -> x + 10) // Step 2: Add 10  
    .map(x -> "Value: " + x) // Step 3: Convert  
    .toList(); // Collect the result
```

CHAPTER 8

Threads and Concurrency in Java

21 Questions & Answers

Q1

What is the difference between Sequential, Concurrent, and Parallel Execution?

What is Sequential Execution?

- Example Scenario: 3 steps in a program
 - Step 1: IO-bound (e.g., API call, file read)
 - Step 2: IO-bound (e.g., database query, network request)
 - Step 3: Combines results from Step 1 & Step 2
- Assumption: Single-core CPU
- Sequential Execution: Run each task after the completion of the previous one.
- Problem: Steps 1 and 2 run one after another, wasting CPU time.

What is Concurrent Execution?

- What if we can start Step 2 while Step 1 is waiting?
 - Idea: Start Step 2 without waiting for Step 1 to complete!
 - Benefit: Reduces total execution time by utilizing waiting periods.
- Advantages of Concurrent Execution:
 - Faster Execution – No wasted time waiting for IO or other activities.
 - Efficient Resource Usage – CPU is used for other tasks while a task is waiting.
 - Better User Experience – Faster response times in applications.
 - Scalability – Handles more tasks in the same time.
- Concurrency = Smart juggling of tasks
 - When there's a lot of waiting involved: I/O operations like database calls, API requests, or file reads.
 - Single CPU can switch between multiple tasks
- Real-world analogy: A chef cooking multiple dishes. Stir the soup, then chop veggies while waiting for it to boil.

What is Parallel Execution?

- Multi-core CPUs: Modern processors have multiple cores for better performance
- Run Multiple Tasks at Same Time: They can run multiple tasks at the same time - one in each core

Execution Types

Execution Type	How It Works
Sequential	One task at a time
Concurrent	Switches from one task to another when waiting
Parallel	Multiple tasks at the same time
MIX OF BOTH	Multiple tasks run at the same time. If any of tasks is waiting, you switch to a different task immediately.

What is we make use of both Concurrency and Parallelism?

- Multiple CPU cores handle tasks simultaneously (parallelism).
- If a task is waiting, CPU switches to another task (concurrency).
- Best of both worlds!

Fast execution + Efficient resource usage.

Q2

How do you decide between extending Thread and implementing Runnable?

1. Extending Thread Class

2. Implementing Runnable Interface

Comparison

Feature	Extend Thread Class
Extend Another Class?	No (Single inheritance limit)
Recommended?	No

Java

```

class CountdownThread extends Thread {
    public void run() {
        for (int i = 500000; i >= 1; i--) {
            System.out.println("Countdown: " + i);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        new CountdownThread().start();
    }
}

```

Java

```

class CountdownThreadRunnable implements Runnable {
    public void run() {
        for (int i = 500000; i >= 1; i--) {
            System.out.println("Countdown: " + i);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        new Thread(
            new CountdownThreadRunnable()).start();
        new Thread(
            new CountdownThreadRunnable()).start();
    }
}

```

Q3

When should you use Callable instead of Runnable?

Runnable can't return values, but Callable can!

Java

```

import java.util.concurrent.*;

class FactorialTask
    implements Callable<Integer> {
    private int number;

    FactorialTask(int number) {
        this.number = number;
    }

    public Integer call() {
        int result = 1;

        for (int i = 2; i <= number; i++)
            result *= i;

        return result;
    }
}

public class Main {
    public static void main(String[] args)
        throws Exception {

        ExecutorService executor
            = Executors.newSingleThreadExecutor();

        Future<Integer> future
            = executor.submit(new FactorialTask(5));

        System.out.println("Factorial: "
            + future.get());

        executor.shutdown();
    }
}

```

Q4

Thread Methods - join() vs sleep() vs yield()

join() – Waiting for a Thread to Finish

- What? Makes one thread wait for another to complete execution before continuing.
- Why? Useful when a thread must complete before another can proceed.
- Overloaded Method? join(time) waits only for a specific time before continuing.

yield() – Suggesting CPU Time Release

- What? Hints to the scheduler that the current thread can pause execution to let other threads run.
- Why? Allows other threads with the same or higher priority to execute first.
- Does the scheduler always listen? No, it's just a hint, and the scheduler may ignore it.

sleep(ms) – Pausing Execution for Some Time

- What? Puts the current thread to sleep for a given time (milliseconds).
- Why? Useful for delays, avoiding busy-waiting, and simulating real-world delays.
- Throws? InterruptedException if another thread interrupts it while sleeping.

Java

```
public class JoinExample {
    public static void main(String[] args)
        throws InterruptedException {
        Thread t1 = new MyThread();
        Thread t2 = new MyThread();

        t1.start();
        t1.join(); // Main thread waits for t1 to finish

        t2.start();

        // Main thread waits max 1 sec for t2 to finish
        t2.join(1000);

        System.out.println(
            "Main thread finished execution.");
    }
}
```

Java

```
class YieldExample implements Runnable {
    public void run() {
        for (int i = 0; i < 5; i++) {
            System.out.println(
                Thread.currentThread().getName()
                + " is running.");

            // Giving a chance to other threads
            Thread.yield();
        }
    }
}
```

Java

```
class SleepExample implements Runnable {
    public void run() {
        System.out.println(
            Thread.currentThread().getName()
            + " is running.");

        try {
            Thread.sleep(3000); // Sleep for 3 seconds
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println(
            Thread.currentThread().getName()
            + " woke up!");
    }
}
```

Q5**What are the different states of a Thread?**

- **NEW State** Thread is created but not started
Exists before calling start()
Example: After Thread t = new Thread(); but before starting the thread
- **RUNNABLE State** Thread is ready but waiting for CPU
Competes with other threads for execution
Example: After calling t.start(); but before start of execution
- **RUNNING State** Thread is executing its run() method
Stays here until it finishes or is interrupted
- **BLOCKED/WAITING State** Thread is waiting for something!
 - 1: Trying to access a locked resource
 - 2: Waiting for another thread to finish
- **TERMINATED (DEAD) State** Thread has finished execution
Cannot be restarted once terminated
Example: Completed thread automatically enters this state

Q6**How to change Thread priority?**

Thread priority determines how important a thread is for CPU scheduling. Priority is represented by an integer value from 1 to 10. Threads with higher priority get preference over lower-priority threads.

It is a hint to the scheduler but does not guarantee execution order.

Constant	Value
Thread.MIN_PRIORITY	1
Thread.NORM_PRIORITY	5
Thread.MAX_PRIORITY	10

How to Set Thread Priority?

Use the setPriority(int priority) method to change priority. Use the getPriority() method to check the current priority.

How Does Thread Priority Work?

- Higher-priority threads are more likely to run before lower-priority threads.
- But it is not guaranteed! The CPU scheduler may still choose lower-priority threads.
- If multiple threads have the same priority, the scheduler picks any randomly.

Java

```
class MyThread extends Thread {
    public void run() {
        System.out.println(
            Thread.currentThread().getName() +
            " running with priority "
            + Thread.currentThread().getPriority());
    }
}

public class ThreadPriorityExample {
    public static void main(String[] args) {
        MyThread thread1 = new MyThread();
        MyThread thread2 = new MyThread();

        thread1.setPriority(Thread.MIN_PRIORITY); // Priority 1
        thread2.setPriority(Thread.MAX_PRIORITY); // Priority 10

        thread1.start();
        thread2.start();
    }
}
```

Q7

What is the difference between Operating System Processes and Operating System Threads?

Process: Full application (e.g., Google Chrome).

Thread: A smaller unit inside a process (e.g., Each Google Chrome Tab).

Feature	OS Process
Definition	Independent program execution
Memory	Each process has its own separate memory space (used by all of its threads)
Speed	Slow to start
Example	Google Chrome

Q8

Why did Java introduce Virtual Threads?

Traditionally, when using Java Platform Threads, each Java platform thread maps to an OS thread.

Problems with Java Platform Threads

Problem

High Memory Usage

Expensive Context Switching

Limited Scalability

IO-Bound Inefficiency

What are Virtual Threads?

Virtual threads run thousands (or millions) of threads on top of a small pool of OS threads. You can run millions of lightweight threads without high memory usage.

Advantages

Feature

Memory Usage

Context Switching

Scalability

Comparison: Traditional Java Threads vs Virtual Threads

Feature	Platform Threads
Thread Creation	Expensive
Max Threads	Few thousand
Best Used In	CPU-Bound Tasks with Low Number of Threads

When to Use Traditional Java Threads?

- CPU-Bound Tasks with Low Number of Threads – Heavy computations (e.g., image processing, data crunching) that need only a few threads, OS threads are fine.

When to Use Virtual Threads?

- Lots of IO-Bound Tasks – Database queries, API calls, file reads.
- Massive Concurrency – Thousands or millions of tasks (e.g., handling web requests).
- Efficient Resource Usage – No wasted OS threads waiting for IO.

Example: Running 100,000 tasks efficiently

Code

```
ExecutorService executor  
    = Executors.newFixedThreadPool(10);
```

Code

```
ExecutorService executor  
    = Executors.newVirtualThreadPerTaskExecutor();
```

Java

```
public class Main {  
    public static void main(String[] args) {  
        try (var executor =  
            Executors.newVirtualThreadPerTaskExecutor()) {  
            for (int i = 0; i < 100000; i++) {  
                executor.submit(  
                    () -> System.out.println(  
                        "Hello from " +  
                        Thread.currentThread());  
                }  
            }  
        }  
    }  
}
```

Q9

What is the need for pooling Threads?

Example: Without a Thread Pool (Inefficient)

- Creates a new thread for every task, which is expensive.

Example: Using a Thread Pool (Efficient)

- Uses fixed-size thread pool to reuse threads.

Problems Without Thread Pooling

- High Overhead – Creating/destroying threads frequently is expensive.
- Difficult to Manage – Tracking and stopping multiple threads manually is hard.

How Thread Pooling Solves These Issues

- Reuses Existing Threads – No need to create/destroy threads repeatedly.
- Limits Number of Threads – Prevents system overload.
- Efficient Task Execution – Threads are reused for new tasks.
- Easy to Manage – Centralized control using ExecutorService.

Key Takeaways

- ExecutorService simplifies Thread Pool management.
- Thread pooling avoids issues like excessive thread creation.

Java

```
class Task implements Runnable {
    public void run() {
        System.out.println(
            Thread.currentThread().getName() +
            " executing task.");
    }
}

public class Main {
    public static void main(String[] args) {
        for (int i = 0; i < 1000; i++) {
            new Thread(new Task()).start();
        }
    }
}
```

Java

```

import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

class Task implements Runnable {
    public void run() {
        System.out.println(
            Thread.currentThread().getName() +
            " executing task.");
    }
}

public class Main {
    public static void main(String[] args) {
        ExecutorService pool
            = Executors.newFixedThreadPool(10);

        for (int i = 0; i < 1000; i++) {
            pool.execute(new Task());
        }

        pool.shutdown();
    }
}

```

Q10

How to choose between different options of creating an ExecutorService?

1. Single Thread Executor

`Executors.newSingleThreadExecutor()` creates an executor with a single worker thread that executes submitted tasks sequentially, one at a time.

2. Fixed Thread Pool

`Executors.newFixedThreadPool(int n)` creates a thread pool with a fixed number of threads to execute tasks concurrently, reusing threads for improved performance.

3. Single Thread Scheduled Executor

`newSingleThreadScheduledExecutor()` creates a scheduler with a single background thread that executes tasks sequentially, supporting delayed and periodic task execution using scheduling methods.

4. Scheduled Thread Pool

`Executors.newScheduledThreadPool(int corePoolSize)` creates a thread pool that can schedule commands to run after a delay or periodically using a thread pool with configurable number of threads.

5. Virtual Thread Per Task Executor

Creates a new virtual thread per task, ideal for massively concurrent workloads (e.g., handling millions of requests).

Key Takeaways

- **Single Thread Executor:** Executes tasks one by one using a single thread, ensuring sequential processing. Ideal for logging etc...
- **Fixed Thread Pool:** Uses a fixed number of threads to handle multiple tasks in parallel. Great for managing concurrent workloads like file uploads.
- **Single Thread Scheduled Executor:** Schedules tasks to run after a delay or periodically with one thread. Perfect for serialized tasks like daily database cleanup or report generation.
- **Scheduled Thread Pool:** Runs tasks repeatedly at fixed intervals using multiple threads. Useful for recurring jobs like health checks or system monitoring.
- **Virtual Thread Per Task Executor:** Creates a lightweight virtual thread for each task, enabling massive concurrency.

Java

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class SingleThreadExecutorExample {
    public static void main(String[] args) {
        ExecutorService executor
            = Executors.newSingleThreadExecutor();

        executor.execute(
            () -> System.out.println(
                "Logging event: User Login"));

        executor.execute(
            () -> System.out.println(
                "Logging event: User Logout"));

        executor.shutdown();
    }
}
```

Java

```

import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class FixedThreadPoolExample {
    public static void main(String[] args) {
        ExecutorService executor
            = Executors.newFixedThreadPool(5);

        for (int i = 1; i <= 100; i++) {
            final int fileId = i;
            executor.execute(
                () -> System.out.println(
                    "Uploading file " + fileId));
        }

        executor.shutdown();
    }
}

```

Java

```

import java.util.concurrent.Executors;
import java.util.concurrent.ScheduledExecutorService;
import java.util.concurrent.TimeUnit;

public class SingleThreadScheduledExecutorExample {

    public static void main(String[] args) {

        ScheduledExecutorService scheduler =
            Executors.newSingleThreadScheduledExecutor();

        // 1. Run once after 24 hours
        scheduler.schedule(() ->
            System.out.println(
                "Running DB 1 Cleanup Once after 24 hours"),
            24, TimeUnit.HOURS);

        // 2. Run every 24 hours (fixed rate)
        scheduler.scheduleAtFixedRate(() ->
            System.out.println(
                "Running DB 2 Cleanup - Every 24 hours"),
            0, 24, TimeUnit.HOURS);

        // 3. Run 24 hours
        // after previous task ends (fixed delay)
        scheduler.scheduleWithFixedDelay(() ->
            System.out.println(
                "Running DB 3 Cleanup - Every 24 hours"),
            0, 24, TimeUnit.HOURS);
    }
}

```

Java

```

import java.util.concurrent.Executors;
import java.util.concurrent.ScheduledExecutorService;
import java.util.concurrent.TimeUnit;

public class ThreadPoolScheduledExecutorExample {
    public static void main(String[] args) {
        // Create a thread pool with 5 threads
        ScheduledExecutorService scheduler
            = Executors.newScheduledThreadPool(5);

        // 1. Run once after 10 seconds
        scheduler.schedule(() ->
            System.out.println(
                "Health Check 1 - One-time after 10 seconds"),
            10, TimeUnit.SECONDS);

        // 2. Run every 10 seconds (fixed rate)
        scheduler.scheduleAtFixedRate(() ->
            System.out.println(
                "Health Check 2 - Every 10 seconds"),
            0, 10, TimeUnit.SECONDS);

        // 3. Run 10 seconds
        // after previous task ends (fixed delay)
        scheduler.scheduleWithFixedDelay(() ->
            System.out.println(
                "Health Check 3 - Every 10 seconds"),
            0, 10, TimeUnit.SECONDS);
    }
}

```

Java

```

import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class VirtualThreadExample {

    public static void main(String[] args) {

        try (ExecutorService executor
            = Executors.
                newVirtualThreadPerTaskExecutor()) {

            executor.submit( () ->
                System.out.println(
                    "Running virtual thread task") );

        }

    }
}

```

Q11

What is Thread Safety? What are Race Conditions?

Thread safety ensures multiple threads can access shared resources without conflicts or errors.

Example: Without Thread Safety

What Does count++ Actually Do?

The statement `count++` looks simple but is actually a three-step operation at the CPU level:

- Read the value of `count` from memory (Load)
- Increment the value (Modify)
- Store the incremented value back to `count` (Write)

`count++` is NOT atomic

- Steps? Read Modify Write. Each step is separate.
- Atomic? Means "all-or-nothing" – can't be interrupted in the middle.
- Problem? Other threads can sneak in between the steps of executing `count++`.

Expected Sequence

Imagine two threads, Thread-1 and Thread-2, running `count++` at the same time.

It is possible that the operations are done one after the other in the expected way with the expected result.

Step	Thread-1 (T1)	Thread-2 (T2)
1	Reads count = 1000	
2	Increments value to 1001	
3	Stores 1001 in count	
4		Reads count = 1001
5		Increments value to 1002
6		Stores 1002 in count

Alternate Sequence

Both threads read the same initial value (1000). They increment separately, but the last write overwrites the first one.

Step	Thread-1 (T1)	Thread-2 (T2)
1	Reads count = 1000	
2		Reads count = 1000
3	Increments value to 1001	
4		Increments value to 1001
5	Stores 1001 in count	
6		Stores 1001 in count

You do not know which sequence will happen, leading to unpredictable final counts.

What is a Race Condition?

A race condition occurs when two or more threads access shared data at the same time, and the program's outcome depends on the sequence or timing of their execution.

Race Condition vs Thread Safety

- Race condition: Happens when multiple threads access shared data simultaneously, causing unpredictable results due to timing issues.
- Thread safety: Ensures code behaves correctly when accessed by multiple threads, preventing race conditions.
- Thread-safe code: Uses techniques to control access to shared data. Ensures correct behavior in multithreaded environments.

Java

```

package com.in28minutes;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class Main {

    public static void main(String[] args) {

        Counter counter = new Counter();

        ExecutorService executor
            = Executors.newVirtualThreadPerTaskExecutor();

        Runnable task = () -> {
            for (int i = 0; i < 10; i++) {
                counter.increment();
            }
        };

        for (int i = 0; i < 100000; i++)
            executor.submit(task);

        executor.shutdown();

        // Wait for tasks to finish
        while (!executor.isTerminated()) { }

        System.out.println(
            "Final Count: " + counter.getCount());

        //Final Count: 1000000 //MIGHT BE DIFFERENT
        //Final Count: 985813 //MIGHT BE DIFFERENT
    }

    class Counter {
        private int count = 0;

        public void increment() {
            count++; // NOT thread-safe
        }

        public int getCount() {
            return count;
        }
    }
}

```

Code

```
MOV EAX, [count]    ; Load count into register EAX
```

Code

```
ADD EAX, 1          ; Increment the value in EAX
```

Code

```
MOV [count], EAX    ; Store updated value back in memory
```

1. Using synchronized methods and blocks

What does synchronized do?

- synchronized locks the increment() method
- Only one thread can enter increment() at a time
- Prevents two threads from changing count together
- Ensures thread-safe update of shared variable

What are the problems with synchronized?

- OLD synchronized collections: A few old Collection classes use synchronized methods

Example: Vector – Thread-safe version of ArrayList using synchronized methods

Another Example: Hashtable – Thread-safe version of HashMap

Example Methods in Vector class:

```
public synchronized E get(int index)
public synchronized E set(int index, E element)
public synchronized boolean add(E e)
public synchronized void removeAllElements()
```

All of the above example methods are synchronized

- Problem: What is the problem with these synchronized methods?

They lock the entire object, even for small operations

Only one thread can access any method at a time

Other threads must wait, even for simple reads

Can become a bottleneck when many threads are used

2. ReentrantLock – More Flexible Locking

- What? Explicitly locks and unlocks
- Why? Allows more control

3. ReadWriteLock – Optimized Locking for Read-Heavy Operations

- What? Allows multiple readers but only one writer.
- Why? Improves performance in read-heavy applications.
- When? Best when reads are more frequent than writes.

4. Atomic Variable Classes (AtomicInteger, AtomicBoolean, AtomicLong)

- Atomic operation: Sequence of instructions that executes as a single, indivisible unit (thread-safe)
- What? Provides atomic operations to ensure thread safety.
- When? Best for counters and simple numeric operations.

Useful Atomic Operation Methods in Typical Atomic Variable Classes

- `get()` – Reads the current value safely
- `set(value)` – Updates the value directly
- `incrementAndGet()` – Adds 1 and returns new value
- `getAndIncrement()` – Returns current value, then adds 1
- `decrementAndGet()` – Subtracts 1 and returns new value
- `addAndGet(n)` – Adds n and returns result
- `compareAndSet(expected, newValue)` – Updates value only if it matches expected
- `getAndSet(newValue)` – Replaces with new value and returns old value

5. Concurrent Collections (ConcurrentHashMap,...)

- A few old Collection classes use synchronized
 - Example: Vector – Thread-safe version of ArrayList using synchronized methods
 - Another Example: Hashtable – Thread-safe version of HashMap
 - Example Methods in Vector class


```
public synchronized E get(int index)
public synchronized E set(int index, E element)
public synchronized boolean add(E e)
public synchronized void removeAllElements()
```
- What is the problem with these thread safe methods?
 - They lock the entire object, even for small operations
 - Only one thread can access any method at a time
 - Other threads must wait, even for simple reads
 - Can become a bottleneck when many threads are used
- What is the solution?
 - Java provides built-in thread-safe high performant concurrent collections.
 - These collections are called Concurrent Collections

Examples of Concurrent Collections

Concurrent Collection

ConcurrentHashMap

CopyOnWriteArrayList

ConcurrentLinkedQueue

Example with HashMap

Example with ConcurrentHashMap

Example Thread Safe Operations on ConcurrentHashMap

- putIfAbsent(key, value) – Adds only if key is not already present
- replace(key, oldValue, newValue) – Updates value only if old value matches
- compute(key, (k, v) -> ...) – Atomically updates the value using a lambda

Other Concurrent Collection Examples

Class

ConcurrentHashMap

CopyOnWriteArrayList

CopyOnWriteArraySet

ConcurrentLinkedQueue

Summary

Technique

synchronized

ReentrantLock

ReadWriteLock

Atomic Variable Classes

Concurrent Collections

[Code](#)

```

class Counter {
    private int count = 0;

    public synchronized void increment() {
        count++;
    }

    public int getCount() {
        return count;
    }
}

```

Python

```

import java.util.concurrent.locks.ReentrantLock;

class Counter {
    private int count = 0;
    private ReentrantLock lock = new ReentrantLock();

    public void increment() {
        lock.lock();
        try {
            count++;
        } finally {
            lock.unlock();
        }
    }

    public int getCount() {
        return count;
    }
}

```

Python

```

import java.util.concurrent.locks.ReentrantReadWriteLock;

class SharedResource {
    private int data = 0;
    private ReentrantReadWriteLock lock
        = new ReentrantReadWriteLock();

    public void write(int value) {
        lock.writeLock().lock();

        try {
            data = value;
        } finally {
            lock.writeLock().unlock();
        }
    }

    public int read() {
        lock.readLock().lock();
        try {
            return data;
        } finally {
            lock.readLock().unlock();
        }
    }
}

```

Python

```

import java.util.concurrent.atomic.AtomicInteger;

class Counter {
    private AtomicInteger count
        = new AtomicInteger(0);

    public void increment() {
        // Atomic operation
        count.incrementAndGet();
    }

    public void decrement() {
        // Atomic operation
        count.decrementAndGet();
    }

    public int getCount() {
        return count.get();
    }
}

```

Java


```

import java.util.*;
import java.util.concurrent.*;

public class VirtualThreadMapBroken {
    public static void main(String[] args) {
        Map<String, Integer> map = new HashMap<>();
        map.put("count", 0); // Initial count

        ExecutorService executor
            = Executors.newVirtualThreadPerTaskExecutor();

        for (int i = 0; i < 100000; i++) {
            executor.submit(() -> {
                map.put("count", map.get("count") + 1);
            });
        }

        executor.shutdown();

        while (!executor.isTerminated()) { }

        // Sometimes < 100000
        System.out.println("Final count: "
            + map.get("count"));
    }
}

```

Java

```

import java.util.*;
import java.util.concurrent.*;

public class VirtualThreadMapFixed {
    public static void main(String[] args) {
        // CHANGE 1: ConcurrentHashMap instead of HashMap
        Map<String, Integer> map = new ConcurrentHashMap<>();

        map.put("count", 0); // Initial count

        ExecutorService executor
            = Executors.newVirtualThreadPerTaskExecutor();

        for (int i = 0; i < 100000; i++) {
            executor.submit(() -> {
                // CHANGE 2: Atomic update - compute method!
                map.compute("count",
                    (key, value) -> value + 1 );
            });
        }

        executor.shutdown();

        while (!executor.isTerminated()) { }

        // ALWAYS 100000
        System.out.println("Final count: " + map.get("count"));
    }
}

```

Q13

Locks vs. Synchronized Approach

Example 1: Using synchronized

Example 2: Using synchronized with Object Lock

Example 3: Using ReentrantLock

QUICK COMPARISON

1. synchronized (Simple Version)

- Easy to write
- Blocks all other synchronized methods on the same object
- Good for low-concurrency or simple logic

2. Object Locks (synchronized (lock))

- More fine-grained control
- Allows incrementI() and incrementJ() to run at same time

3. ReentrantLock

- Full control with lock(), unlock(), tryLock(), timeouts
 - lock() – Manually lock a critical section
 - unlock() – Manually release the lock (always required!)
 - tryLock() – Try to get the lock without waiting forever
 - tryLock(timeout, unit) – Wait for a while, give up if not acquired
- Great for advanced concurrency needs

Java

```

package com.in28minutes.concurrency;

public class BiCounter {
    private int i = 0;
    private int j = 0;

    synchronized public void incrementI() {
        i++;
    }

    synchronized public void incrementJ() {
        j++;
    }

    public int getI() {
        return i;
    }

    public int getJ() {
        return j;
    }
}

```

Java

```

package com.in28minutes.concurrency;

public class BiCounter {
    private int i = 0;
    private int j = 0;

    private final Object lockForI = new Object();
    private final Object lockForJ = new Object();

    public void incrementI() {
        synchronized (lockForI) {
            i++;
        }
    }

    public void incrementJ() {
        synchronized (lockForJ) {
            j++;
        }
    }

    public int getI() {
        return i;
    }

    public int getJ() {
        return j;
    }
}

```

Java

```

package com.in28minutes.concurrency;
import java.util.concurrent.locks.ReentrantLock;

public class BiCounterWithLocks {
    private int i = 0;
    private int j = 0;
    private Lock LockForI = new ReentrantLock();
    private Lock LockForJ = new ReentrantLock();

    public void incrementI() {
        lockForI.lock();
        i++;
        lockForI.unlock();
    }

    public void incrementJ() {
        lockForJ.lock();
        j++;
        lockForJ.unlock();
    }

    public int getI() {
        return i;
    }

    public int getJ() {
        return j;
    }
}

```

Q14

When do you use ThreadLocal?

- What? Each thread gets its own separate instance of a variable.
- Why? Eliminates race conditions because there is no shared data between threads.
- When? Best for caching per-thread data (e.g., user sessions).

Java

```

class ThreadLocalExample {
    private static ThreadLocal<Integer> threadLocalCount =
        ThreadLocal.withInitial(() -> 0);

    public static void main(String[] args) {
        Runnable task = () -> {
            int count = threadLocalCount.get();
            threadLocalCount.set(count + 1);
            System.out.println(
                Thread.currentThread().getName() +
                " - " + threadLocalCount.get());
        };

        Thread t1 = new Thread(task);
        Thread t2 = new Thread(task);

        t1.start();
        t2.start();
    }
}

```

Q15

When do you use a Semaphore?

- What? Semaphore allows a limited number of threads to access a resource.
- Why? Prevents too many threads from accessing a shared resource at the same time.
- When? Best for controlling concurrent access to databases or APIs.

Java

```
import java.util.concurrent.Semaphore;

class Worker implements Runnable {
    private Semaphore semaphore;

    Worker(Semaphore semaphore) {
        this.semaphore = semaphore;
    }

    public void run() {
        try {

            semaphore.acquire(); // Acquire permit

            System.out.println(
                Thread.currentThread().getName() +
                " working...");

            Thread.sleep(1000);

            semaphore.release(); // Release permit

        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}

public class Main {
    public static void main(String[] args) {
        // Max 2 threads at a time
        Semaphore semaphore = new Semaphore(2);

        new Thread(new Worker(semaphore)).start();
        new Thread(new Worker(semaphore)).start();
        new Thread(new Worker(semaphore)).start();
    }
}
```

Q16

When do you use a CountdownLatch?

- What? A thread waits for a set number of threads to finish before proceeding.
- Why? Prevents race conditions by ensuring dependent tasks complete first.
- When? Best for waiting for multiple tasks to complete before continuing.

Java

```
import java.util.concurrent.CountDownLatch;

class Worker implements Runnable {
    private CountDownLatch latch;

    Worker(CountDownLatch latch) {
        this.latch = latch;
    }

    public void run() {
        System.out.println(
            Thread.currentThread().getName() +
            " finished work.");
        latch.countDown(); // Reduce the latch count
    }
}

public class Main {
    public static void main(String[] args)
        throws InterruptedException {

        CountDownLatch latch = new CountDownLatch(2);

        for (int i = 0; i < 10; i++) {
            Thread t = new Thread(new Worker(latch));
            t.start();
        }

        latch.await();

        System.out.println(
            "At least 2 workers finished execution.");
    }
}
```

Q17

Which Object Methods Are Used for Inter-Thread Communication?

Method	Description
wait()	Makes a thread wait until another thread calls notify() or notifyAll().
notify()	Wakes up one waiting thread that previously called wait().
notifyAll()	Wakes up all threads waiting on the same object.

Note: These methods must be called inside a synchronized block to avoid `IllegalMonitorStateException`.

Example: Inter-Thread Communication Using wait() and notify()

- Thread-1 (Main Thread) waits for Thread-2 (Calculator Thread) to finish computing the sum.

- Thread-2 calculates the sum and notifies Thread-1 using notify().

- Sequence of Execution:

Main thread acquires a lock on thread and calls thread.wait().

Calculator thread starts execution and acquires the same lock.

Once computation is done, notify() is called, waking up the main thread.

Main thread resumes execution and prints the result.

Why are these rarely used?

- Requires synchronized - Hard to use and blocks all other synchronized methods/blocks on the same object.

- Risk of Missed Signals – If notify() is called before wait(), the thread may wait forever.

- Better Alternatives Exist – ReentrantLock, Semaphore, CountDownLatch ...

Java

```
class Calculator extends Thread {
    long sumUptoMillion;

    public void run() {
        synchronized (this) { // Locking on this object
            calculateSumUptoMillion();
            notify(); // Notify waiting thread
        }
    }

    private void calculateSumUptoMillion() {
        for (int i = 0; i < 1000000; i++) {
            sumUptoMillion += i;
        }
    }
}

public class ThreadWaitNotify {
    public static void main(String[] args)
        throws InterruptedException {

        Calculator thread = new Calculator();

        synchronized (thread) { // Locking on the same object
            thread.start(); // Start calculation thread
            thread.wait(); // Wait for notification
        }

        System.out.println("Sum up to million: "
            + thread.sumUptoMillion);
    }
}
```

Q18

Why do Deadlocks occur?

What is a Deadlock?

- A deadlock occurs when two or more threads wait for each other indefinitely, preventing further execution.

Example: Deadlock Situation

- Thread-1 locks Resource A and waits for Resource B.
- Thread-2 locks Resource B and waits for Resource A.
- Since neither thread can proceed, the program freezes forever (Deadlock!).

Java


```

class Resource {
    private final String name;

    public Resource(String name) {
        this.name = name;
    }

    void use(Resource other) {
        // Locking this resource first
        synchronized (this) {
            System.out.println(
                Thread.currentThread().getName()
                + " locked " + this.name);

            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }

            // Now locking the second resource
            synchronized (other) {
                System.out.println(
                    Thread.currentThread().getName()
                    + " locked " + other.name);
            }
        }
    }
}

public class DeadlockExample {
    public static void main(String[] args) {
        Resource resourceA
            = new Resource("Resource A");

        Resource resourceB
            = new Resource("Resource B");

        // Thread-1 locks A B
        Thread t1 = new Thread(()
            -> resourceA.use(resourceB),
            "Thread-1");

        // Thread-2 locks B A (Deadlock risk!)
        Thread t2 = new Thread(()
            -> resourceB.use(resourceA),
            "Thread-2");

        t1.start();
        t2.start();
    }
}

```

Q19

Discuss an option to solve above deadlock

One Option: Lock Resources in the Same Order

- Lock resources in the same order, ensuring a consistent locking hierarchy.
- How?

Sort resources by unique identifiers (e.g., object hash, resource ID).

Always lock the lower ID first before the higher ID.

Other Options to Prevent or Handle Deadlocks

- Use Try-Lock with Timeout: Replace `synchronized` with `ReentrantLock.tryLock(timeout)` to avoid waiting forever. If a lock can't be acquired within the timeout, retry or exit gracefully.
- Coarse-Grained Locking: Use a single global lock instead of multiple fine-grained ones. This reduces concurrency but guarantees that threads won't compete for multiple resources simultaneously.
- Avoid Nested Locks: Design code so that threads acquire only one lock at a time. This simple discipline removes the possibility of circular waits and eliminates deadlocks entirely.

Java

```

class Resource {
    private final String name;

    public Resource(String name) {
        this.name = name;
    }

    void use(Resource other) {
        Resource first = this.name
            .compareTo(other.name) < 0
            ? this : other;

        Resource second = this.name
            .compareTo(other.name) < 0
            ? other : this;

        // Always lock lower-named resource first
        synchronized (first) {
            System.out.println(
                Thread.currentThread().getName()
                + " locked " + first.name);

            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }

            synchronized (second) { // Lock 2nd resource
                System.out.println(
                    Thread.currentThread().getName()
                    + " locked " + second.name);
            }
        }
    }
}

public class DeadlockSolution {
    public static void main(String[] args) {
        Resource resourceA = new Resource("Resource A");
        Resource resourceB = new Resource("Resource B");

        // Both threads now lock in the same order: A B
        Thread t1 = new Thread(
            () -> resourceA.use(resourceB),
            "Thread-1");

        Thread t2 = new Thread(
            () -> resourceB.use(resourceA),
            "Thread-2");

        t1.start();
        t2.start();
    }
}

```

Java

```

class Resource {
    private final String name;

    public Resource(String name) {
        this.name = name;
    }

    void use(Resource other) {
        Resource first = this.name
            .compareTo(other.name) < 0
            ? this : other;

        Resource second = this.name
            .compareTo(other.name) < 0
            ? other : this;

        // Always lock lower-named resource first
        synchronized (first) {
            System.out.println(
                Thread.currentThread().getName()
                + " locked " + first.name);

            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }

            synchronized (second) { // Lock 2nd resource
                System.out.println(
                    Thread.currentThread().getName()
                    + " locked " + second.name);
            }
        }
    }
}

public class DeadlockSolution {
    public static void main(String[] args) {
        Resource resourceA = new Resource("Resource A");
        Resource resourceB = new Resource("Resource B");

        // Both threads now lock in the same order: A B
        Thread t1 = new Thread(
            () -> resourceA.use(resourceB),
            "Thread-1");

        Thread t2 = new Thread(
            () -> resourceB.use(resourceA),
            "Thread-2");

        t1.start();
        t2.start();
    }
}

```

Q20

ExecutorService vs. ForkJoinPool: When to use each?

Example: Sum an array using parallelism

ForkJoinPool

- When? Best for divide-and-conquer tasks like sorting, searching.
- Why? Efficiently splits tasks across multiple CPU cores.

Comparison

Feature	ExecutorService
Best For?	Independent tasks
Example Use Case	Running 100 different tasks in parallel

Java

```
import java.util.concurrent.*;

class SumTask extends RecursiveTask<Integer> {
    private int[] arr;
    private int start, end;

    SumTask(int[] arr, int start, int end) {
        this.arr = arr;
        this.start = start;
        this.end = end;
    }

    protected Integer compute() {
        if (end - start <= 2) {
            int sum = 0;
            for (int i = start; i < end; i++) sum += arr[i];
            return sum;
        }

        int mid = (start + end) / 2;
        SumTask left = new SumTask(arr, start, mid);
        SumTask right = new SumTask(arr, mid, end);

        left.fork();
        return right.compute() + left.join();
    }
}

public class Main {
    public static void main(String[] args) {
        int[] numbers = {1, 2, 3, 4, 5, 6, 7, 8};
        ForkJoinPool pool = new ForkJoinPool();
        int sum = pool.invoke(
            new SumTask(numbers, 0, numbers.length));
        System.out.println("Sum: " + sum);
    }
}
```

Q21

How can you enhance parallelization of code using Streams?

- Multi-core CPUs: Modern processors have multiple cores for better performance
- Parallelization: Run Multiple Tasks at Same Time (one in each core)
- Functional code can easily be parallelized:

`parallel()` or `parallelStream()`: Enables parallel execution

`.parallel()` is useful if you start with sequential stream, then decide to switch to parallel.

`.parallelStream()` is cleaner and more direct for starting in parallel.

How does it work?

Stream is split into multiple parts

Each part runs on a different core

Finally, the results are combined

Code

```
List<Integer> numbers =
    IntStream.rangeClosed(1, 1_000_000)
        .boxed()
        .collect(Collectors.toList());

long count1 = numbers.stream()
    .parallel()
    .filter(n -> n % 2 == 0)
    .count();

long count2 = numbers.parallelStream()
    .filter(n -> n % 2 == 0)
    .count();
```

CHAPTER 9

Collections in Java

22 Questions & Answers

Q1

How did the Collections Framework Evolve in Java?

Early Collections

- Early classes for grouping objects included Dictionary, Vector, Stack, and Properties.
- These classes had an ad-hoc design.
- There was no consistent set of interfaces.

Collections Framework

- The Collections Framework was introduced in Java 1.2.
- It had two main goals:
 - Unify collection handling through standard interfaces and implementations
 - Improve performance, flexibility, and ease of use

Evolution

- Over time, the framework evolved with new features:
 - Java 5: Generics, enhanced for-each loop, concurrent collections (e.g., ConcurrentHashMap)
 - Java 8: Stream API, forEach, lambda expressions
 - Java 9: Factory methods for immutable collections (List.of(), Set.of(), Map.of())
 - Java 21: Sequenced collections for consistent ordering across types

Q2

Why are Collections needed in Java?

Collections in Java provide a flexible and efficient way to store, retrieve, and manipulate data.

Limitations of Arrays:

- Fixed Size – Cannot grow or shrink dynamically.
- Manual Operations – Searching, adding, or removing elements requires extra logic
- Lack of Built-in Methods – No built-in sorting, searching, or advanced data structures.

How Collections Solve These Problems:

- Dynamic Resizing – Can grow or shrink as needed.
- Predefined Methods – Supports adding, removing, sorting, and searching elements easily.
- Rich Data Structures – Includes Lists, Sets, Maps, and Queues for different needs.

Example – Array vs. Collection

Code

```
String[] names = new String[3];
names[0] = "Alice";
names[1] = "Bob";
names[2] = "Charlie";

// Adding a new name?
// Requires creating a new array.

List<String> names = new ArrayList<>();
names.add("Alice");
names.add("Bob");
names.add("Charlie");
names.add("David"); // No need to resize manually
```

Q3

Can you give a 10,000 feet overview of interfaces in collection framework?

- Collection is at the root of the hierarchy (add element, remove element, ..)
 - SequencedCollection provides well-defined order (add first, add last, ..)
 - List stores ordered elements, allows duplicates, supports index (add element e at index 3)
 - Set holds unique elements; no duplicates allowed
 - SortedSet maintains elements in sorted order
 - NavigableSet adds search/navigation methods
 - Queue is used for FIFO processing of elements
 - Deque is a double-ended queue (SequencedCollection)
- Map stores key-value pairs; keys must be unique (Does not inherit from Collection interface)
 - SortedMap maintains elements in sorted order
 - NavigableMap adds search/navigation methods

Q4

What are the main interfaces in the Collections Framework?

NOTE: This is provided as a quick reference. We will not have a video discussing this question. We will discuss all the interfaces in videos for subsequent questions

Name	Description	Important Operations
Collection	Root interface for collections hierarchy.	add(e), remove(e), size(), iterator(), contains(e), ..
Sequenced Collection	A collection that has a well-defined encounter order, that supports operations at both ends, and that is reversible.	Inherited + addFirst(E e), addLast(E e), getFirst(), getLast(), removeFirst(), removeLast(), reversed(), ..
List	Ordered collection allowing duplicates. Access using index (positioned).	Inherited + get(int index), set(int index, E element), void add(int index, E element), remove(int index), indexOf(Object element),...
Set	Collection of unique elements (NO duplicates)	Inherited Methods
Sequenced Set	A collection that is both a Sequenced Collection and a Set	`Inherited Methods
SortedSet	Set maintaining elements in sorted order	first(), last(), subSet(), headSet(), tailSet()
NavigableSet	Sorted set with navigation methods (find closest match elements)	lower(e), higher(e), floor(e), ceiling(e)
Queue	Collection that follows FIFO order.	offer(e), poll(), peek()
Deque	Double-ended queue allowing insertions/removals at both ends	Queue + SequencedCollection
Blocking Queue	Thread-safe queue that blocks when full (on put()) or empty (on take())	put(e), take(), offer(e, timeout), poll(timeout)
Map	Stores key-value pairs	put(k,v), get(k), remove(k)

Sequenced Map	A Map that has a well-defined encounter order, that supports operations at both ends, and that is reversible.	reversed(), firstEntry(), lastEntry(), pollFirstEntry(), pollLastEntry(), putFirst(k, v), putLast(k, v), sequencedKeySet(), sequencedValues(), sequencedEntrySet()
SortedMap	Maintains keys in sorted order	firstKey(), lastKey(), subMap(), headMap(), tailMap()
NavigableMap	SortedMap with navigation methods	lowerKey(k), higherKey(k), lowerEntry(k), higherEntry(k)

Q5

What are the key methods in the Collection interface?

Collection is root interface of the collection framework!

Key Methods

- `add(E element)`: Adds an element to the collection.
- `remove(Object element)`: Removes an element from the collection.
- `size()`: Returns the number of elements in the collection.
- `isEmpty()`: Checks if the collection is empty.
- `clear()`: Removes all elements from the collection.
- `contains(Object element)`: Checks if an element exists in the collection.
- `containsAll(Collection<?> c)`: Checks if all elements in another collection exist.
- `addAll(Collection<? extends E> c)`: Adds all elements from another collection.
- `removeAll(Collection<?> c)`: Removes all matching elements.
- `retainAll(Collection<?> c)`: Keeps only elements present in another collection.
- `iterator()`: Returns an iterator to traverse elements.
- `stream()`: Returns a stream using the elements in the collection
- `parallelStream()`: Returns a parallel stream using the elements in the collection

[Code](#)

```

interface Collection<E> extends Iterable<E> {
    boolean add(E element);
    boolean remove(Object element);
    int size();
    boolean isEmpty();
    void clear();
    boolean contains(Object element);

    boolean containsAll(Collection<?> c);
    boolean addAll(Collection<? extends E> c);
    boolean removeAll(Collection<?> c);
    boolean retainAll(Collection<?> c);

    Iterator<E> iterator();

    Stream<E> stream();
    Stream<E> parallelStream();
    //...
    //...
}

```

Q6

What are the key methods in the SequencedCollection interface?

Introduced in JDK 21, SequencedCollection extends Collection Interface. It represents a collection that has a well-defined encounter order, that supports operations at both ends, and that is reversible.

Key Methods

- `addFirst(E e)`: Adds an element at the beginning of the collection.
- `addLast(E e)`: Adds an element at the end of the collection.
- `getFirst()`: Retrieves the first element of the collection.
- `getLast()`: Retrieves the last element of the collection.
- `removeFirst()`: Removes and returns the first element of the collection.
- `removeLast()`: Removes and returns the last element of the collection.
- `reversed()`: Returns a view of the collection with elements in reverse order.

Code

```

public interface SequencedCollection<E> extends Collection<E> {

    void addFirst(E e);
    void addLast(E e);
    E getFirst();
    E getLast();
    E removeFirst();
    E removeLast();

    SequencedCollection<E> reversed();

}

```

- **Extension of SequencedCollection**
List extends SequencedCollection, inheriting all its methods.
Provides additional methods for position-based operations.
A list may have duplicate elements.
- **Insertion Order is Maintained**
Maintains the insertion order of elements.
Elements are stored in the order they are added.
- **Inserting Elements**
Use add(int position, E element) to insert at a specific position.
Elements added without position go to the end.

Example:

Java

```
List<Integer> numbers = new ArrayList<>();

// Insertion Order is Maintained
numbers.add(10); // index 0
numbers.add(20); // index 1
numbers.add(30); // index 2

System.out.println(numbers); // [10, 20, 30]

// Inserting Elements - position-based operation
numbers.add(1, 15); // insert 15 at index 1
System.out.println(numbers); // [10, 15, 20, 30]

// Duplicates allowed
numbers.add(20); // add duplicate
System.out.println(numbers); // [10, 15, 20, 30, 20]

// Fast lookup using index (O(1))
System.out.println(numbers.get(2)); // 20
```

Code

```
interface List<E> extends SequencedCollection<E> {
    boolean addAll(int index, Collection<? extends E> c);
    E get(int index);
    E set(int index, E element);
    void add(int index, E element);
    E remove(int index);
    int indexOf(Object element);
    int lastIndexOf(Object element);
    ListIterator<E> listIterator();
    ListIterator<E> listIterator(int index);
    List<E> subList(int fromIndex, int toIndex);
    static <E> List<E> of(); //Static methods added in JDK 9
}
```

1. ArrayList – Best for Fast Random Access

Key Characteristics:

- Uses a dynamic array internally
- Maintains insertion order of elements
- Supports indexed access to elements

When to Use:

- Fast element access via `get(index)`
- Insertions/removals mostly at the end
- You don't need thread safety

When to Avoid:

- Frequent insertions or deletions in the middle
- Large shifts in elements (reallocation is expensive)

2. LinkedList – Best for Frequent Insertions/Deletions

Key Characteristics:

- Uses a doubly linked list internally
- Each element has references to the next and previous elements

When to Use:

- Frequent insertions or deletions in the middle

When to Avoid:

- Fast random access (`get(index)`) is needed
- Memory overhead is a concern (extra references per element)

3. Vector – Best for Thread-Safe Lists

Key Characteristics:

- Uses a dynamic array like ArrayList
- All methods are synchronized (thread-safe)
- Slower than ArrayList due to synchronization overhead

When to Use:

- Multiple threads access the list simultaneously
- Working with legacy code that already uses Vector

When to Avoid:

- Thread safety is not required (ArrayList is faster)
- Performance is important (synchronization adds overhead)

4. CopyOnWriteArrayList – Best for Read-Mostly, Thread-Safe Scenarios

Key Characteristics:

- Thread-safe variant of ArrayList
- Creates a new copy of the array on each write (add/remove)
- Safe to iterate even during concurrent modifications

When to Use:

- Concurrent, read-heavy scenarios

When to Avoid:

- Frequent writes or large lists (copying is expensive)

Q9

How do you choose the right approach to sort a List?

1. Using Collections.sort() – Best for Simple Sorting

- Uses natural ordering (e.g., alphabetical for Strings, ascending for numbers).
- Modifies the original list in place.

When to Use:

- Sorting a list of Strings or primitive types.
- Sorting objects that implement Comparable.

Example:

2. Using Comparable – Best for Natural Ordering

- Implement Comparable<T> in the class.
- Used when objects have a default sorting order (e.g., sorting players by runs).

When to Use:

- When an object has a natural ordering (e.g., sorting Cricketers by runs).
- If the class controls its sorting logic.

Example:

3. Using Comparator – Best for Custom Sorting

- Implement Comparator<T> separately from the class.
- Allows sorting by different criteria (e.g., descending order).

When to Use:

- When multiple sorting orders are needed (e.g., sorting by name, then by runs).
- When modifying the original class is not possible.

Example:

4. Comparison of Sorting Approaches

Approach	When to Use
Collections.sort()	Simple sorting of Strings/numbers
Comparable	When objects have a default sorting order
Comparator	For custom sorting orders

Java

```
List<String> numbers = new ArrayList<>();
numbers.add("one");
numbers.add("two");
numbers.add("three");
numbers.add("four");

Collections.sort(numbers);
System.out.println(numbers); // [four, one, three, two]
```

Java

```
class Cricketer implements Comparable<Cricketer> {
    int runs;
    String name;

    public Cricketer(String name, int runs) {
        this.name = name;
        this.runs = runs;
    }

    @Override
    public int compareTo(Cricketer that) {
        return Integer.compare(this.runs, that.runs);
    }

    @Override
    public String toString() {
        return name + " " + runs;
    }
}
```

Code

```
List<Cricketer> players = Arrays.asList(
    new Cricketer("Virat", 12200),
    new Cricketer("Rohit", 11600),
    new Cricketer("Gill", 2400),
    new Cricketer("Dhoni", 10700)
);

// Sort using Comparable (ascending by runs)
Collections.sort(players);
```

Java

```
import java.util.Comparator;

class DescendingSorter implements Comparator<Cricketer> {
    @Override
    public int compare(Cricketer c1, Cricketer c2) {
        return Integer.compare(c2.runs, c1.runs);
    }
}
```

Code

```
List<Cricketer> players = Arrays.asList(  
    new Cricketer("Virat", 12200),  
    new Cricketer("Rohit", 11600),  
    new Cricketer("Gill", 2400),  
    new Cricketer("Dhoni", 10700)  
);  
  
// Sort using Comparator (descending by runs)  
Collections.sort(players, new DescendingSorter());
```

Q10

What are the key interfaces associated with Set operations?

- Set – Extends Collection interface. Does not allow duplicates.
- SequencedSet - A collection that is both a Sequenced Collection and a Set. Represents a collection of unique elements with a well-defined order.
 - SortedSet – Extends SequencedSet and maintains sorted order. Provides methods like first(), last(), subSet(), headSet(), tailSet()
 - NavigableSet – Extends SortedSet and provides navigation methods like lower(), higher(), floor(), ceiling().

Java

```

//Set - NO DUPLICATES
Set<String> simpleSet = new HashSet<>();
simpleSet.add("Apple");
simpleSet.add("Banana");
simpleSet.add("Apple"); // Ignored
// Set: [Banana, Apple]
System.out.println("Set: " + simpleSet);

// SequencedSet: SET + ORDER
SequencedSet<String> sequencedSet = new LinkedHashSet<>();
sequencedSet.add("One");
sequencedSet.add("Two");
sequencedSet.add("Three");
// No output, modifies set
sequencedSet.addFirst("Zero"); // Java 21+
// No output, modifies set
sequencedSet.addLast("Four"); // Java 21+
// SeqSet: [Zero, One, Two, Three, Four]
System.out.println("SeqSet: " + sequencedSet);
// First: Zero
System.out.println("First: " + sequencedSet.getFirst());
// Last: Four
System.out.println("Last: " + sequencedSet.getLast());

// SortedSet: SEQUENCED SET + SORTED ORDER
SortedSet<Integer> sortedSet = new TreeSet<>();
sortedSet.add(30);
sortedSet.add(10);
sortedSet.add(20);
// Sorted: [10, 20, 30]
System.out.println("Sorted: " + sortedSet);
// First: 10
System.out.println("First: " + sortedSet.first());
System.out.println("First: " + sortedSet.getFirst());
// Last: 30
System.out.println("Last: " + sortedSet.last());
System.out.println("Last: " + sortedSet.getLast());
// HeadSet: [10]
System.out.println("HeadSet: " + sortedSet.headSet(20));
// TailSet: [20, 30]
System.out.println("TailSet: " + sortedSet.tailSet(20));
// SubSet: [10, 20]
System.out.println("SubSet: " + sortedSet.subSet(10, 30));

//NavigableSet - SORTED SET + NAVIGATION
NavigableSet<Integer> navSet = new TreeSet<>();
navSet.add(10);
navSet.add(20);
navSet.add(30);
// NavSet: [10, 20, 30]
System.out.println("NavSet: " + navSet);
// Lower: 10
System.out.println("Lower: " + navSet.lower(20));
// Floor: 20
System.out.println("Floor: " + navSet.floor(20));
// Higher: 30
System.out.println("Higher: " + navSet.higher(20));
// Ceiling: 20
System.out.println("Ceiling: " + navSet.ceiling(20));

```

Code

```

interface Set<E> extends Collection<E> {
    // No duplicate elements allowed
    // No ordering guaranteed
}

interface SequencedSet<E> extends Set<E>, SequencedCollection<E> {
    SequencedSet<E> reversed();
    void addFirst(E e); //From SequencedCollection
    void addLast(E e); //From SequencedCollection
    E getFirst(); //From SequencedCollection
    E getLast(); //From SequencedCollection
    E removeFirst(); //From SequencedCollection
    E removeLast(); //From SequencedCollection
}

interface SortedSet<E> extends SequencedSet<E> {
    SortedSet<E> subSet(E start, E end); // start to end-1
    SortedSet<E> headSet(E end); // < end
    SortedSet<E> tailSet(E start); // >= start
    E first();
    E last();
}

interface NavigableSet<E> extends SortedSet<E> {
    E lower(E e); //largest x such that x < e
    E floor(E e); //largest x such that x <= e
    E ceiling(E e); //smallest x such that x >= e
    E higher(E e); //smallest x such that x > e
    E pollFirst(); //return and remove first element
    E pollLast(); //return and remove last element
}

```

Q11

How do you decide which Set implementation to use?

1. HashSet – Best for Fast Lookups

Key Characteristics:

- Unordered collection
- Does not allow duplicate elements

When to Use:

- Fast lookups and uniqueness are priorities
- You don't care about the order of elements

When to Avoid:

- You need to preserve insertion or sorted order
- You need thread safety

2. LinkedHashSet – Best for Maintaining Insertion Order

Key Characteristics:

- Extends HashSet
- Maintains insertion order
- Slightly slower than HashSet

When to Use:

- You want predictable iteration order
- Need fast operations + ordering

When to Avoid:

- Order doesn't matter (HashSet is faster)
- You need sorting (TreeSet is more appropriate)
- You need thread safety

3. TreeSet – Best for Sorted Elements

Key Characteristics:

- Implements NavigableSet
- Maintains elements in natural or custom sort order
- Backed by a Red-Black Tree

When to Use:

- You need sorted elements
- You need Range-based queries (subSet, headSet, tailSet)

When to Avoid:

- Sorting is unnecessary (HashSet is faster)
- You need thread safety

4. CopyOnWriteArraySet – Best for Read-Mostly, Thread-Safe Scenarios

Key Characteristics:

- Thread-safe Set implementation
- All write operations (add/remove) create a new copy

When to Use:

- Many reads, few writes
- You need thread-safety

When to Avoid:

- Frequent writes or large data sets (copying is expensive)

5. ConcurrentSkipListSet – Best for Concurrent & Sorted Access

Key Characteristics:

- Thread-safe version of TreeSet
- Backed by a concurrent skip list
- Maintains sorted order

When to Use:

- Multi-threaded applications needing sorted data

When to Avoid:

- Single-threaded scenarios (overhead is unnecessary)

Java

```
TreeSet<Integer> set = new TreeSet<>();
set.add(55); set.add(25); set.add(35);
set.add(5); set.add(45);
System.out.println(set);           // [5, 25, 35, 45, 55]
System.out.println(set.first());    // 5
System.out.println(set.last());     // 55
System.out.println(set.subSet(10, 50)); // [25, 35, 45]
System.out.println(set.headSet(35)); // [5, 25]
System.out.println(set.tailSet(35)); // [35, 45, 55]
System.out.println(set.lower(25));  // 5
System.out.println(set.floor(25));   // 25
System.out.println(set.higher(25));  // 35
System.out.println(set.ceiling(25)); // 25
```

Interface	Description	Key Methods
Queue	Collection that holds elements for processing in FIFO order	offer(e), poll(), peek()
Deque	Double-ended queue allowing insertion/removal from both ends	addFirst(e), removeLast(), peekFirst()
BlockingQueue	Thread-safe queue that blocks when full/empty	put(e), take(), poll(timeout)

Queue Interface

- Extends the Collection interface.
- Used for holding elements in order for processing.
- Key Methods:
 - peek(): Retrieves the head without removal.
 - poll(): Retrieves and removes the head.

Deque Interface

- Extends Queue & SequencedCollection interfaces
- Supports insertion and removal from both ends.

BlockingQueue Interface

- A thread-safe queue designed for use in producer-consumer scenarios
- Blocks:
 - On put(e) if the queue is full
 - On take() if the queue is empty

Java

```
// Queue example using LinkedList
Queue<String> queue = new LinkedList<>();
queue.offer("A");
queue.offer("B");
// A (peek without remove)
System.out.println(queue.peek());
// A (remove and return)
System.out.println(queue.poll());
// B (now at head)
System.out.println(queue.peek());
```

Code

```
interface Queue<E> extends Collection<E> {  
    // inserts element, exception if full  
    boolean add(E e);  
  
    // inserts element, returns false if full  
    boolean offer(E e);  
  
    // removes head, exception if empty  
    E remove();  
  
    // removes head, null if empty  
    E poll();  
  
    // gets head, exception if empty  
    E element();  
  
    // gets head, null if empty  
    E peek();  
}
```

Java

```
// Deque example using ArrayDeque  
Deque<String> deque = new ArrayDeque<>();  
deque.addFirst("B");  
deque.addLast("C");  
deque.addFirst("A");  
// A  
System.out.println(deque.peekFirst());  
// C  
System.out.println(deque.peekLast());  
// A (remove front)  
System.out.println(deque.removeFirst());  
// C (remove end)  
System.out.println(deque.removeLast());
```

Code


```

public interface Deque<E>
    extends Queue<E>, SeencedCollection<E> {

    // add to front, exception if full
    void addFirst(E e);

    // add to end, exception if full
    void addLast(E e);

    // add to front, false if full
    boolean offerFirst(E e);

    // add to end, false if full
    boolean offerLast(E e);

    // remove from front, exception if empty
    E removeFirst();

    // remove from end, exception if empty
    E removeLast();

    // remove from front, null if empty
    E pollFirst();

    // remove from end, null if empty
    E pollLast();

    // get front, exception if empty
    E getFirst();

    // get end, exception if empty
    E getLast();

    // get front, null if empty
    E peekFirst();

    // get end, null if empty
    E peekLast();

    // remove first match
    boolean removeFirstOccurrence(Object o);

    // remove last match
    boolean removeLastOccurrence(Object o);
}

```

Java

```

// BlockingQueue example with ArrayBlockingQueue

// Queue with capacity 2
BlockingQueue<String> bq =
    new ArrayBlockingQueue<>(2);
//Produce elements
bq.put("A");
bq.put("B");
// The queue is now full
// The next line will block if uncommented
// bq.put("C");
// Consume one element
System.out.println(bq.take()); // A
// Add another element (now space is available)
bq.put("C");
// Consume remaining elements
System.out.println(bq.take()); // B
System.out.println(bq.take()); // C

```

Code

```
interface BlockingQueue<E> extends Queue<E> {
    // inserts, waits if full
    void put(E e) throws InterruptedException;

    // inserts with timeout, waits if full
    boolean offer(E e, long timeout, TimeUnit unit)
        throws InterruptedException;

    // removes, waits if empty
    E take() throws InterruptedException;

    // removes with timeout, waits if empty
    E poll(long timeout, TimeUnit unit)
        throws InterruptedException;

    // remaining capacity in queue
    int remainingCapacity();

    // removes all into given collection
    int drainTo(Collection<? super E> c);

    // removes up to maxElements into given collection
    int drainTo(Collection<? super E> c, int maxElements);
}
```

Q13

What are the important implementations of Queue?

Name	Description	Extends
PriorityQueue	Queue where elements are sorted by priority	AbstractQueue
ArrayDeque	Resizable array-based Deque	AbstractCollection
ArrayBlockingQueue	Array-backed blocking queue (Fixed Size)	AbstractQueue
LinkedBlockingQueue	Linked-list blocking queue	AbstractQueue
ConcurrentLinkedQueue	Lock-free thread-safe queue	AbstractQueue
ConcurrentLinkedDeque	Lock-free thread-safe deque	AbstractCollection

Q14

What are the key interfaces related to Map?

Here are the Map-related interfaces in Java:

1. Map Interface

- What? A collection that stores key-value pairs.
- Why? Allows quick lookup and modification of values using keys.
- Key Characteristics:
 - Does not extend Collection.
 - Keys are unique, but values can be duplicate.
 - Uses Entry objects to store key-value pairs.

Important Methods:

Method

put(K key, V value)

get(Object key)

remove(Object key)

containsKey(Object key)

containsValue(Object value)

keySet()

values()

entrySet()

2. SequencedMap Interface

- What? A Map that maintains a well-defined sequence of entries
- Why? Allows consistent iteration order and reverse traversal

Key Characteristics:

- Extends Map
- Maintains the insertion or access order (depending on implementation)
- Adds methods to get/remove first and last entries
- Useful when both ordering and key-based access are important

Important Methods:

Method
firstEntry()
lastEntry()
pollFirstEntry()
pollLastEntry()
reversed()
putFirst(K key, V value)
putLast(K key, V value)

3. SortedMap Interface

- What? A Map that maintains keys in sorted order.
- Why? Useful when sorting keys automatically.
- Key Characteristics:
 - Extends SequencedMap.
 - Keys are sorted in natural order or by a Comparator.

Important Methods:

Method
firstKey()
lastKey()
subMap(fromKey, toKey)
headMap(toKey)
tailMap(fromKey)

4. NavigableMap Interface

- What? An extension of SortedMap with navigation methods.
- Why? Provides methods to find the closest matches for keys.
- Key Characteristics:
 - Allows floor, ceiling, higher, and lower key lookups.
 - Supports reverse order traversal.

Important Methods:

Method
floorKey(K key)
ceilingKey(K key)
lowerKey(K key)
higherKey(K key)
floorEntry(K key)
ceilingEntry(K key)
lowerEntry(K key)
higherEntry(K key)
descendingMap()

Java

```
Map<String, Integer> map = new HashMap<>();
map.put("Alice", 25);
map.put("Bob", 30);
System.out.println(map.get("Alice")); // Output: 25
```

Java

```
SequencedMap<String, Integer> seqMap = new LinkedHashMap<>();
seqMap.put("A", 1);
seqMap.put("B", 2);
seqMap.put("C", 3);

System.out.println(seqMap.firstEntry()); // A=1
System.out.println(seqMap.lastEntry());  // C=3
```

Java

```
SortedMap<Integer, String> sortedMap = new TreeMap<>();
sortedMap.put(3, "C");
sortedMap.put(1, "A");
sortedMap.put(2, "B");
System.out.println(sortedMap); // Output: {1=A, 2=B, 3=C}
```

Java

```

NavigableMap<Integer, String> navMap = new TreeMap<>();
navMap.put(1, "A");
navMap.put(3, "C");
navMap.put(5, "E");

```

```

System.out.println(navMap.floorKey(4)); // Output: 3
System.out.println(navMap.ceilingKey(4)); // Output: 5

```

Q15

What are the key implementations of Map Interface?

Name	Description	Extends
HashMap	Unordered key-value store (NOT thread safe)	AbstractMap
LinkedHashMap	Map with insertion order (NOT thread safe)	HashMap
TreeMap	Sorted key-value store (Red-Black tree)(NOT thread safe)	AbstractMap
ConcurrentHashMap	Thread-safe map with high concurrency	AbstractMap
ConcurrentSkipListMap	Thread-safe, sorted map (Keys are sorted)	AbstractMap

What is the purpose of the Collections class?

The Collections class provides utility methods for working with collections:

Feature	Method
Sorting	sort(List)
Searching	binarySearch(List, key)
Thread Safety	synchronizedList(List)
Unmodifiable Collections	unmodifiableList(List)
Finding Min/Max	min(Collection), max(Collection)
Shuffling/Reversing	shuffle(List), reverse(List)

Java

```

import java.util.*;

public class CollectionsExample {
    public static void main(String[] args) {
        // Create a list of numbers
        List<Integer> numbers
            = new ArrayList<>(Arrays.asList(10, 5, 20, 15));

        // Sort the list
        Collections.sort(numbers);

        // Output: [5, 10, 15, 20]
        System.out.println("Sorted List: " + numbers);

        // Find the minimum and maximum values
        int min = Collections.min(numbers);
        int max = Collections.max(numbers);

        // Output: Min: 5, Max: 20
        System.out.println("Min: " + min + ", Max: " + max);

        // Reverse the list
        Collections.reverse(numbers);

        // Output: [20, 15, 10, 5]
        System.out.println("Reversed List: " + numbers);

        // Shuffle the list
        Collections.shuffle(numbers);

        // Output: Random order
        System.out.println("Shuffled List: " + numbers);

        // Perform binary search (List must be sorted first)
        Collections.sort(numbers);

        int index = Collections
            .binarySearch(numbers, 15);

        // Output: Index position of 15
        System.out.println("Index of 15: " + index);

        // Make the list unmodifiable
        List<Integer> unmodifiableList
            = Collections.unmodifiableList(numbers);

        System.out.println(
            "Unmodifiable List: " + unmodifiableList);
        // Uncommenting below line
        // will throw UnsupportedOperationException
        // unmodifiableList.add(25);
    }
}

```

Q16

How do Synchronized and Concurrent Collections differ?

Feature	Synchronized Collections
Synchronization Method	Uses synchronized methods & blocks
Performance	Slower due to full synchronization
Thread Safety	Only one thread can access the collection at a time
Examples	Vector, Hashtable, Collections. synchronizedList()
Best Use Case	When multiple threads rarely modify the collection

Q17

What is the CopyOnWrite approach in Concurrent Collections?

- What? A technique where modifications create a new copy of the collection instead of modifying the existing one.
- Why? Ensures thread safety without synchronization during read operations.
- How?
 - Stores values in an immutable internal array.
 - When modified, a new array is created with the updated data.
 - Read operations are fast and do not block, only write operations are synchronized.
- Best Use Case: When reads outnumber writes
- Example Implementations: CopyOnWriteArrayList, CopyOnWriteArraySet.

Q18

What is the Compare-And-Swap (CAS) approach?

- What? A low-level, non-blocking synchronization technique used for thread-safe updates
- Why? Avoids locking, improving performance in high-concurrency scenarios
- How it works:
 - Read the current value (expected value)
 - Compute the new value
 - Compare the current value with the expected value
 - If they match, update to the new value
 - If not, retries until it succeeds or exits (in very rare cases)
- Used In: ConcurrentLinkedQueue, AtomicInteger etc..

Q19

In what scenarios does a Java Collection throw UnsupportedOperationException?

- What? Thrown when an operation is not supported by a collection.
- Fixed Size Lists

Example: `Arrays.asList()`

Operations that change the size of a list, like `add()`/`remove()`, throw

`UnsupportedOperationException`

Operations that do not change the size of the list, like `set()`, are allowed

- Unmodifiable Collections

Example: `List.of()`, `Set.of()`, `Collections.unmodifiableList()`

All mutation operations (like `add()`, `remove()`, `set()`, `clear()`) throw `UnsupportedOperationException`

Example Code:

Code

```
List<String> list = Arrays.asList("A", "B");
list.add("C"); // Throws UnsupportedOperationException

List<String> immutable = List.of("X", "Y");
immutable.remove(0); // Throws UnsupportedOperationException
```

Q20

What is the difference between Fail-Fast and Fail-Safe Iterators?

1. What is a Fail-Fast Iterator?

- What? Throws `ConcurrentModificationException` if the collection is modified during iteration.
- Where? Used in non-thread-safe collections like `ArrayList`, `HashMap`, etc.
- When? Happens when a structural modification (`add/remove`) occurs outside the iterator.

2. What is a Fail-Safe Iterator?

- What? Does not throw exceptions on modification.
- Where? Used in concurrent collections (`ConcurrentHashMap`, `CopyOnWriteArrayList`).
- Why? Works on a separate copy of the data, ensuring safe iteration.

3. Fail-Fast vs Fail-Safe

Feature	Fail-Fast
Modification	Throws Concurrent Modification Exception
Thread Safety	Not safe
Works On	Direct collection
Examples	ArrayList, HashMap

Java

```
import java.util.HashMap;
import java.util.Iterator;
import java.util.Map;

public class FailFast {
    public static void main(String[] args) {
        Map<String, String> map = new HashMap<>();

        map.put("key1", "value1");
        map.put("key2", "value2");
        map.put("key3", "value3");

        Iterator<String> iterator = map.keySet().iterator();
        while (iterator.hasNext()) {
            System.out.println(map.get(iterator.next()));

            // Throws ConcurrentModificationException
            map.put("key4", "value4");
        }
    }
}
```

Java

```
import java.util.Iterator;
import java.util.concurrent.ConcurrentHashMap;

public class FailSafe {
    public static void main(String[] args) {
        ConcurrentHashMap<String, String> map
            = new ConcurrentHashMap<>();

        map.put("key1", "value1");
        map.put("key2", "value2");
        map.put("key3", "value3");

        Iterator<String> iterator = map.keySet().iterator();

        while (iterator.hasNext()) {
            System.out.println(map.get(iterator.next()));
            map.put("key4", "value4"); // No Exception
        }
    }
}
```

1. Choose the Right Collection Type

- Invest time to decide the right List, Map, Queue, Set or Map to use

2. Set Initial Capacity to Reduce Resizing

- Prevents frequent resizing and improves performance.

3. Use the Right Data Structure for Thread Safety

- Use ConcurrentHashMap instead of HashMap in multi-threaded environments.
- Use CopyOnWriteArrayList for read-heavy operations with rare writes.

4. Use Immutable Collections When Possible

- Prevents accidental modification and improves safety.
- Use Collections.unmodifiableList() or List.of().

5. Prefer Iterators or Streams Over Manual Loops

- Improves readability and avoids IndexOutOfBoundsException.

Code

```
// Avoids unnecessary resizing
Map<String, Integer> map = new HashMap<>(100);
```

Code

```
// Immutable List
List<String> names =
    List.of("Alice", "Bob", "Charlie");
```

Java

```
// More readable than a for-loop
list.forEach(System.out::println);
```

- Problem: Getting/Modifying first/last elements is tricky
- Solution: Java 21 added SequencedCollection interface

Sequenced Collection Interfaces

Example: ArrayList

Example: LinkedHashSet

Example: LinkedHashMap

Code

```
interface SequencedCollection<E> extends Collection<E> {
    void addFirst(E element);
    void addLast(E element);
    E getFirst();
    E getLast();
    E removeFirst();
    E removeLast();
    SequencedCollection<E> reversed();
}

interface SequencedSet<E>
    extends Set<E>, SequencedCollection<E> {
    SequencedSet<E> reversed();
}

interface SequencedMap<K, V> extends Map<K, V> {
    SequencedMap<K, V> reversed();
    SequencedSet<K> sequencedKeySet();
    SequencedCollection<V> sequencedValues();
    SequencedSet<Entry<K, V>> sequencedEntrySet();
    V putFirst(K key, V value);
    V putLast(K key, V value);
    Entry<K, V> firstEntry();
    Entry<K, V> lastEntry();
    Entry<K, V> pollFirstEntry();
    Entry<K, V> pollLastEntry();
}
```

Java

```

var courses = new ArrayList<>();
courses.add("Java");
courses.addFirst("Python");
courses.addLast("JavaScript");

System.out.println(
    courses.getFirst()); // Python

System.out.println(
    courses.getLast()); // JavaScript

System.out.println(
    courses.reversed()); // [JavaScript, Java, Python]

```

Java

```

var courses = new LinkedHashSet<>(
    List.of("Spring", "AWS", "Azure"));

courses.addFirst("Java");
courses.addLast("Kotlin");

System.out.println(
    courses.getFirst()); // Java

System.out.println(
    courses.getLast()); // Kotlin

System.out.println(
    courses.reversed()); // [Kotlin, Azure, AWS, Spring, Java]

```

Java

```

var courses = new LinkedHashMap<>();
courses.put(1, "Spring");
courses.put(2, "Spring Boot");
courses.put(3, "Spring AI");

courses.putFirst(0, "Java");
courses.putLast(4, "Kotlin");

//{4=Kotlin, 3=Spring AI, 2=Spring Boot, 1=Spring, 0=Java}
System.out.println(courses.reversed());

//[0, 1, 2, 3, 4]
System.out.println(courses.sequencedKeySet());

//[Java, Spring, Spring Boot, Spring AI, Kotlin]
System.out.println(courses.sequencedValues());

//0=Java
System.out.println(courses.firstEntry());

//4=Kotlin
System.out.println(courses.lastEntry());

//0=Java (Removes and returns the first key-value pair)
System.out.println(courses.pollFirstEntry());

//4=Kotlin
System.out.println(courses.pollLastEntry());

```

CHAPTER 10

Generics in Java

5 Questions & Answers

Q1

Why Do We Need Generics in Java?

What Are Generics?

- What? A way to create type-safe and reusable code in Java.
- Why? Eliminates the need for multiple versions of the same class for different data types.
- How? Instead of hardcoding a specific type, we use a type parameter (ex: T).

Problems without Generics - 1 - Too Rigid

- Works for One Type Only – Code below can only store String values
- No Reusability – Need to rewrite class if you want Integer, Double, etc.
- Lots of Duplicate Code – Repeating logic for each data type gets messy
- Hard to Maintain – More classes to write, test, and manage for every type

Problems without Generics - 2 - OR Too Much Flexibility

- No Type Safety – Any object type can be added without warnings
- Mixing Types – Strings, Integers, Objects can all go into one list
- Manual Casting – Must cast values when reading from the list
- Runtime Errors – Wrong casts cause `ClassCastException` at runtime
- Hard to Read – No clear info on what the list is supposed to store
- Code Example:

Solution: Using Generics

- Use Type Parameter T – Replace hardcoded type like String.
- Flexible and Reusable – One class works for all data types.
- Type Safe – Compiler checks type at compile time.
- No Casting Needed – You get the right type automatically.

Code

```

class MyList {
    private List values = new ArrayList();

    void add(String value) {
        values.add(value);
    }

    void remove(String value) {
        values.remove(value);
    }
}

MyList myList = new MyList();
myList.add("Value 1");
myList.add("Value 2");

```

Python

```

import java.util.ArrayList;
import java.util.List;

class MyList {
    private List values = new ArrayList(); // Raw list, no generics

    void add(Object value) {
        values.add(value);
    }

    void remove(Object value) {
        values.remove(value);
    }

    Object get(int index) {
        return values.get(index);
    }
}

//Usage
MyList myList = new MyList();
myList.add("Value 1");
myList.add("Value 2");
myList.add(123); // Allowed - no type safety

// Optional: safe cast if you *know* the type
String first = (String) myList.get(0); // okay
// Integer wrong = (Integer) myList.get(0); // runtime error

```

Code


```

class MyListGeneric<T> {
    private List<T> values = new ArrayList<>();

    void add(T value) {
        values.add(value);
    }

    void remove(T value) {
        values.remove(value);
    }

    T get(int index) {
        return values.get(index);
    }
}

MyListGeneric<String> myListString
    = new MyListGeneric<>();
myListString.add("Value 1"); // Type safe
myListString.add("Value 2");
// myListString.add(10); // Compile-time error

// No casting required
String strValue = myListString.get(0);

MyListGeneric<Integer> myListInteger
    = new MyListGeneric<>();
myListInteger.add(1);
myListInteger.add(2);

Integer num = myListInteger.get(0);

```

Q2

Generic Class vs Generic Method – What's the Real Difference?

What is a Generic Class?

- Uses Type Parameter T in class definition – Works with different data types.
- Code Reusability – One class, many data type options.
- Type Safe – Compiler checks the type during compilation.
- No Need for Casting – You get the right type automatically.

What Is a Generic Method?

- Method Defines Its Own <T> – Not tied to class-level generic types.
- Flexible and Reusable – Method works with any data type.
- Common in Utility Classes – Like sorting, printing, comparing.

Differences Between Generic Class and Generic Method

Feature	Generic Class
Scope	Applies to all methods in the class
Type Parameter Definition	Defined at class level (class Box<T>)
Use Case	Used when multiple methods in a class need the same type
Example	Box<T> for storing generic values

Do Not Forget About Generic Interfaces

- What? Interfaces can define type parameters just like classes
- Why? Makes them reusable with any type
- Used In? Collections, functional interfaces, comparators, etc.
- Code Example:

Java

```

class Box<T> {
    private T value;

    public void set(T value) {
        this.value = value;
    }

    public T get() {
        return value;
    }
}

// Using Generic Class
Box<String> stringBox = new Box<>();
stringBox.set("Hello");
System.out.println(stringBox.get()); // Output: Hello

Box<Integer> intBox = new Box<>();
intBox.set(10);
System.out.println(intBox.get()); // Output: 10

//Example Generic Class From JDK
public class ArrayList<E> extends AbstractList<E> {
    public E set(int index, E element) { /*Code*/ }
    public boolean add(E e) { /*Code*/ }
    public void addFirst(E e) { /*Code*/ }
    public void addLast(E e) { /*Code*/ }
    public Iterator<E> iterator() { /*Code*/ }
}

```

Java

```

class Utility {
    // Generic Method (T can be any type)
    public static <T> void printArray(T[] array) {
        for (T element : array) {
            System.out.print(element + " ");
        }
        System.out.println();
    }
}

// Using Generic Method
String[] words = {"Hello", "World"};
Integer[] numbers = {1, 2, 3};

Utility.printArray(words); // Output: Hello World
Utility.printArray(numbers); // Output: 1 2 3

//Example Generic Methods From JDK
public class Collections {
    public static <T> void sort(List<T> list) { /*Code*/ }
    public static <T> boolean replaceAll(
        List<T> list, T oldVal, T newVal) { /*Code*/ }
}

```

Java

```

//Generic Interface
interface Printer<T> {
    void print(T value);
}

// Generic implementation
class GenericPrinter<T> implements Printer<T> {
    public void print(T value) {
        System.out.println("Printing: " + value);
    }
}

// Examples from JDK
public interface Comparable<T> {
    int compareTo(T o);
}

public interface Iterator<E> {
    boolean hasNext();
    E next();
}

public interface Supplier<T> {
    T get();
}

```

Q3

How Are Generics Actually Used in the JDK?

1. Collections Framework (List, Set, Map, ...)

- Built Using Generics – Core of Java's data structures.
- Type Safe – Prevents adding wrong data types.
- No Typecasting Needed – Compiler handles types automatically.
- Flexible and Reusable – Use same class with different types.

2. Comparable and Comparator Interfaces

- Comparable – Define natural order for instances of a Class
- Comparator – Define custom sort order for specific cases
- Example:

3. Optional in Java 8+

- Avoids Null Checks – No more if (obj != null) madness.
- Type Safe Wrapper – Wraps values that may or may not be present.
- Cleaner Code – Handles defaults with orElse, maps with map.
- No NullPointerException – Safer than returning null.

4. Stream API (Java 8+)

- Type-Safe Processing – Works with the type you stream.
- Generic All the Way – filter, map, collect keep type info.
- No Casting Required – Results are the right type automatically.
- Functional Style – Chain operations in a clean, readable way.

Java

```
//public class ArrayList<E>
List<String> names = new ArrayList<>();
names.add("Java");
// names.add(123); // Compile error
String first = names.get(0); // No casting

//public class HashMap<K,V>
Map<String, Integer> map = new HashMap<>();
map.put("Alice", 25);
map.put("Bob", 30);
```

Java

```
//public interface Comparable<T> {
// public int compareTo(T o);
//}
class Person implements Comparable<Person> {
    private String name;
    private int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public int getAge() { return age; }
    public String getName() { return name; }

    @Override
    public int compareTo(Person other) {
        return Integer.compare(this.age, other.age);
    }
}

//USAGE
// Natural sorting (Comparable)
List<Person> people = new ArrayList<>();
people.add(new Person("Alice", 25));
people.add(new Person("Bob", 30));

Collections.sort(people);

// Custom sorting (Comparator)
// public interface Comparator<T> {
// int compare(T o1, T o2);
// }
class NameSorter implements Comparator<Person> {
    public int compare(Person p1, Person p2) {
        return p1.getName().compareTo(p2.getName());
    }
}

//USAGE
Collections.sort(people, new NameSorter());
```

Java

```
//public final class Optional<T> {
// public static <T> Optional<T> of(T value) {}
//}
Optional<String> optional = Optional.of("Hello");
System.out.println(
    optional.orElse("Default")); // Output: Hello

//public static<T> Optional<T> empty() {
Optional<String> emptyOptional = Optional.empty();
System.out.println(
    emptyOptional.orElse("Default")); // Output: Default
```

Java

```

List<String> names = List.of("Alice", "Bob", "Charlie");

//public interface List<E>
//    public Stream<E> stream()

//public interface Stream<T>
//    Stream<T> filter(Predicate<? super T> predicate);

List<String> filtered = names.stream()
    .filter(name -> name.startsWith("A"))
    .collect(Collectors.toList());

System.out.println(filtered); // Output: [Alice]

```

Q4

What Is Type Erasure and Why Should You Care?

- Replaces Type Parameters – T becomes its bound or Object.
- Compile Time – Done at compile time, not runtime.
- No Runtime Type Info – Reflection can't access actual type arguments.
- Raw Types at Runtime – JVM sees List, not List<String>.
- Example: Type Erasure in Action
 - Before Type Erasure (What Compiler Sees):
 - After Type Erasure (What JVM Sees):
- Effects of Erasure
 - No Type Info at Runtime – JVM forgets all generic details.
 - instanceof Doesn't Work – Can't check for Box<String>.
 - Type Checks Are Limited – All generics look the same to JVM.
- Why Does Java Use Erasure?
 - Backward Compatibility – Supports old Java code without generics.
 - Simpler Bytecode – No extra bytecode for every generic type.
 - No Runtime Overhead – Type info removed, so faster execution.
 - Easier JVM Implementation – JVM doesn't need to handle generics.

Code

```

class Box<T> { // Generic class
    private T value;

    public void set(T value) {
        this.value = value;
    }
    public T get() {
        return value;
    }
}

```

Code

```
class Box { // No generics at runtime
    private Object value;

    public void set(Object value) {
        this.value = value;
    }
    public Object get() { return value; }
}
```

Code

```
class Box<T> { // Generic class
    private T value;

    public void set(T value) {
        this.value = value;
    }
    public T get() {
        return value;
    }
}
```

Code

```
class Box { // No generics at runtime
    private Object value;

    public void set(Object value) {
        this.value = value;
    }
    public Object get() { return value; }
}
```

Code

```
Box<String> strBox = new Box<>();

// Compilation error
// if (strBox instanceof Box<String>) { }
```

Code

```
Box<String> strBox = new Box<>();

// Compilation error
// if (strBox instanceof Box<String>) { }
```

Q5

How Can You Restrict Types Using Generics in Java?

1. Bounded Type (T extends SomeClass)

- What? Restricts T to subclasses of a specific class
- Why? Allows safe access to superclass methods
- Example? Accepts Integer, Double, Float – all extend Number
- Prevents? Using unrelated types like String or Boolean

2. Multiple Bounds (T extends Class & Interface)

- What? Restricts T to extend a class and implement one or more interfaces
- Why? Ensures T has both class behavior and interface methods
- Example? T must extend Document and implement Printable
- Prevents? Using types that only meet one condition (just class or just interface)

3. Upper Bound Wildcard in Methods (<? extends T>)

- What? Accepts any type that extends T (or is T itself)
- Example? You can pass Integer, Double, Float to a method expecting ? extends Number
- Safe For? Reading only, because all elements are guaranteed to be at least Number
- Prevents? Adding elements (compiler says: "I don't know what exact type to accept!")
- How It Works

The method says: "Give me a list of something that's a Number"

But I don't know what that "something" is (could be Integer, Double, etc.)

So I'll just read it — I can safely treat all items as Number

But I won't write to it — I might mess up the actual subtype stored inside

4. Lower Bound Wildcard in Methods (? super T)

- What? Accepts T or any of its supertypes
- Example? Accepts Integer, Number, or Object for ? super Integer
- Why? Allows safely adding elements of type T
- Prevents? Reading elements as Integer – type is too generic to trust
- How It Works

The method says: "Give me a list that can hold an Integer"

That could be a List<Integer>, List<Number>, or List<Object>

You can safely write an Integer to any of those

But reading? Nope. The compiler doesn't know what exact type is inside

5. Generic Method with Both Wildcards

- What? A method that uses both Upper Bound Wildcard (? extends T) and Lower Bound Wildcard (? super T)
- Why? Offers maximum flexibility for utility methods
- Example? Copy elements from one list to another
- Use Case? Utility methods in libraries like Collections.copy()
- How It Works
 - ? extends T read from source (safe for reading)
 - ? super T write to destination (safe for writing)
- “Producer Extends, Consumer Super” (PECS rule) (collection's point of view)
 - If you are only reading items from a generic collection, it is a producer => use extends
 - If you are only writing items in, it is a consumer => use super.
 - If you do both with the same collection, you shouldn't use either extends or super.
- Prevents? Copying incompatible types (like String into Integer list)

Java

```
class MathUtils<T extends Number> {
    public double square(T num) {
        //NOTE: Calling a Number specific method!
        return num.doubleValue() * num.doubleValue();
    }
}

MathUtils<Integer> intMath = new MathUtils<>();
System.out.println(intMath.square(5)); // Output: 25.0

// Compilation error: String is not a subclass of Number
// MathUtils<String> strMath = new MathUtils<>();
```

Java

```

interface Printable {
    void print();
}

class Document {}

class Report<T extends Document & Printable> {
    private T content;

    public Report(T content) {
        this.content = content;
    }

    public void printReport() {
        content.print();
    }
}

class MyReport extends Document implements Printable {
    public void print() {
        System.out.println("Printing report...");
    }
}

Report<MyReport> report = new Report<>(new MyReport());
report.printReport(); // Output: Printing report...

```

Java

```

public class WildcardExample {
    public static void printNumbers(
        List<? extends Number> numbers) {
        for (Number n : numbers) {
            System.out.println(n);
        }

        // Not safe to add
        // type could be Integer, Double, etc.
        // numbers.add(42);
    }

    public static void main(String[] args) {
        List<Integer> intList = List.of(1, 2, 3);
        List<Double> dblList = List.of(1.1, 2.2);

        printNumbers(intList); // Allowed
        printNumbers(dblList); // Allowed
    }
}

```

Java

```

public class SuperExample {
    public static void addNumbers(
        List<? super Integer> numbers) {
        numbers.add(10);
        numbers.add(20); // safe

        // Not safe to read as Integer

        // Compilation Error: Type Mismatch
        // numbers.get(0) is assumed of type Object
        // Integer num = numbers.get(0);

        //Needs Casting
        //Integer num = (Integer) numbers.get(0);

    }

    public static void main(String[] args) {
        List<Number> nums = new ArrayList<>();
        // Number is a supertype of Integer
        addNumbers(nums);

        List<Object> objs = new ArrayList<>();
        // Object is also a supertype
        addNumbers(objs);

        System.out.println(nums); // Output: [10, 20]
    }
}

//Collections class
//Replaces all of the elements of the
//specified list with the specified element.
//public static <T> void fill(
//    List<? super T> list, T obj) {

```

Java

```

public class GenericUtils {
    public static <T> void copyElements(
        List<? super T> destination,
        List<? extends T> source) {

        for (T item : source) {
            destination.add(item);
        }
    }

    public static void main(String[] args) {
        List<Integer> source = List.of(1, 2, 3);
        List<Number> destination = new ArrayList<>();

        // Integer Number
        copyElements(destination, source);

        // Output: [1, 2, 3]
        System.out.println(destination);
    }
}

//Collections
//public static <T> void copy(
//    List<? super T> dest, List<? extends T> src) {

```

CHAPTER 11

Modifiers in Java

5 Questions & Answers

Q1

What Are the Different Access Modifiers in Java and When Should You Use Them?

Access modifiers control visibility of classes, methods, and variables.

Modifier	Class	Package	Subclass	World
public				
protected				
default				
private				

Best Practices:

- **Use private** – Best for encapsulating fields and internal logic
 In `java.lang.String`, the value array is private final:
 Ensures immutability and prevent external access or modification.
 In `java.lang.Integer`, the value field is private:
 Internal representation stays hidden while exposing only necessary methods.
- **Use protected** – Only when subclasses genuinely need access
 The `java.util.ArrayList` class, the default constructor `protected ArrayList()` is protected:
 Prevents direct instantiation.
 Ensures only subclasses like `ArrayList` or `LinkedList` can extend and initialize it.
 In `java.util.ArrayList` class `protected void removeRange (int fromIndex, int toIndex)`:
 Allows subclasses to efficiently remove elements in bulk.
 Not exposed publicly to avoid accidental misuse.
- **Use public** – Expose only what external code truly needs
 In `java.lang.String`, methods like `length()` and `charAt()` are public:
 Other classes can safely read string content without touching internals
 In `java.util.ArrayList`, methods like `add()` and `get()` are public:
 Other classes can use the collection methods to perform operations.
- **Avoid default (package-private)** – Visibility leaks across unrelated classes in the same package

1. Use static final for Constants

- What? Use static final for constant values.
- Why? Saves memory by sharing a single copy across all instances (static)
Also the value cannot be changed (final)
- JDK Example: `Integer.MAX_VALUE`, `Math.PI`
- Example:

2. Use static for Utility Methods

- What? Methods that don't need instance variables.
- Why? Avoids unnecessary object creation.
- JDK Example: `Collections.sort()`, `Math.abs()`
- Example:

3. Use static Blocks for Initialization

- What? Code that runs once when the class loads.
- Why? Useful for setting up static variables.
- Example:
- Source Code (from Integer class)

4. Be Careful with static Variables in Multithreading

- Why? Shared across threads, leading to concurrency issues.
- Solution: Use synchronization (or other concurrency alternatives - `AtomicInteger`, ..) if needed.
- Example:

5. Use static Factory Methods for Object Creation

- Why? Improves readability and flexibility.
Constructors are great for simple initialization of values
Factory methods improve readability with meaningful names, allow caching, can return subtypes, and give better control over object creation (like returning `Optional`, a singleton, or even reusing existing instances)
- JDK Example: `List.of()`, `Optional.of()`, `LocalDate.now()`
- Example:

6. For Implementing Singleton Pattern

- What? One instance shared across app
- JDK Example: `Runtime.getRuntime()` (Gives access to the JVM's runtime environment to run system commands (`exec()`), manage memory (`totalMemory()`, `freeMemory()`), trigger garbage collection (`gc()`), or shut down the JVM (`exit()`))

7. Use static Imports Carefully

- What? Lets you access static members without class name
- Why? Overuse can make code less readable.
- Avoid? Wildcard imports – they reduce clarity
- Example:

8. Use static Methods in Interfaces

- What? Interfaces can define static methods with implementation
- Why? Useful for utility methods specific to the interface
 - Not inherited by implementing classes
 - Belongs only to the interface
- JDK Example: `Comparator.comparing()`, `Map.entry()`, `Predicate.isEqual()`
- Example:

9. Use static Nested Classes

- What? A nested class that doesn't require an instance of the outer class
- Why? Ideal when inner class doesn't need access to instance variables
- Can Access? Only static members of the outer class
- When? When you need a helper class that groups related functionality, like a `Database.Connection` class managing database connections without needing an outer class instance.
- JDK-Like Use Case: `Integer.IntegerCache` (cache -128 to 127 - inclusive)
- Example:

Code

```
class MathUtil {  
    public static final double PI = 3.14159;  
}
```

Code

```
class MathUtil {
    public static int square(int num) {
        return num * num;
    }
}
```

Java

```
class Config {
    public static String appName;

    static {
        appName = "MyApp";
        //Other initialization of static variables
        System.out.println("Static block executed");
    }
}
```

Code

```
private static class IntegerCache {
    static final Integer cache[];

    static {
        // Cache range: -128 to 127
        cache = new Integer[-(-128) + 127 + 1];

        for (int i = 0; i < cache.length; i++)
            cache[i] = new Integer(i - 128);
    }
}
```

Code

```
class Counter {
    private static int count = 0;

    // Use synchronized
    public static synchronized void increment() {
        count++;
    }
}
```

Code


```

class User {
    private String name;

    private User(String name) {
        this.name = name;
    }

    public static User createUser(String name) {
        return new User(name);
    }
}

```

Code

```

class Singleton {
    private static Singleton instance = new Singleton();
    public static Singleton getInstance() {
        return instance;
    }
}

```

Python

```

import static java.lang.Math.sqrt;
//import static java.lang.Math.*; AVOID

class Example {
    double result = sqrt(25);
}

```

Java

```

interface MathOperations {
    static int square(int n) {
        return n * n;
    }
}

class Demo {
    public static void main(String[] args) {
        int result = MathOperations.square(5);
        System.out.println(result); // Output: 25
    }
}

```

Java

```

class Database {
    private static String url = "jdbc:mysql://localhost:3306/mydb";

    static class Connection {
        void connect() {
            System.out.println("Connecting to: " + url);
        }
    }
}

class Main {
    public static void main(String[] args) {
        Database.Connection connection = new Database.Connection();
        connection.connect(); // Output: Connecting to:
        jdbc:mysql://localhost:3306/mydb
    }
}

```

Q3

What Are the Best Practices for Using the final Modifier in Java?

1. Use final for Constants

- What? Declare constants using final and static.
- Why? Prevents modification and improves code readability.
- JDK Example: Integer.MAX_VALUE, Math.PI
- Example:

2. Use final for Member Variables

- What? Makes a field constant after it's set
- Why? Ensures the variable is assigned once and never changed
- When? Great for setting values in constructors
- Example:

3. Use final with Classes to Prevent Inheritance

- What? Prevents other classes from extending it.
- Why? Useful for utility or immutable classes.
- Example:

4. Use final with Methods to Prevent Overriding

- What? Stops subclasses from modifying behavior.
- Why? Ensures security and correctness.
- Example:

5. Be Cautious with final Reference Variables

- What? A final reference variable cannot change its reference, but the object's data can be modified.
- Why? Prevents reassignment but allows state changes.
- Example:

Code

```
class MathUtil {
    public static final double PI = 3.14159;
}

//Examples From JDK
public final class Integer {
    public static final int    MAX_VALUE = 0x7fffffff;
}

public final class Math {
    public static final double PI = 3.141592653589793;
}
```

Code

```
class User {
    private final String name;

    public User(String name) {
        this.name = name; // Must assign in constructor
    }

    public String getName() {
        return name;
    }
}

//Example from JDK
public final class String {
    private final byte[] value;
}
```

Java

```
final class Utility {
    public static void log(String message) {
        System.out.println(message);
    }
}

//Examples From JDK
public final class String // Immutable text
public final class Integer // Immutable wrapper
public final class Math // Utility methods only
```

Java

```

class Parent {
    public final void show() {
        System.out.println(
            "This method cannot be overridden.");
    }
}

//Examples from JDK
//These methods are marked `final` to prevent subclasses
//from changing core thread behavior. Overriding them
//could break thread safety, consistency,
//or violate JVM expectations during execution.
public class Thread implements Runnable {
    public final String getName() {}
    public final boolean isAlive() {}
    public final void setPriority(
        int newPriority) {}
    //And a lot of others!
}

```

Code

```

class Example {
    public static void main(String[] args) {
        final StringBuilder sb =
            new StringBuilder("Hello");

        sb.append(" World"); // Allowed

        // Compilation Error
        // sb = new StringBuilder("New");
    }
}

```

Q4

What Are the Best Practices for Using the abstract Modifier in Java?

1. Best Use Case For abstract class: Template Method Pattern

- What? Defines an algorithm structure, letting subclasses implement specific steps.
- Why? Ensures a controlled workflow while allowing flexibility.
- Example:

2. abstract class: Use Proper Naming

- What? Use meaningful names that indicate a base class.
- Why? Makes the purpose of the class clear.
- Example:

3. Prefer Interfaces if Abstract Class has No Implementation

- What? Use an interface when a class only defines method signatures.
- Why? Provides more flexibility than abstract classes.
- Example:

4. Avoid Full Implementation in an abstract Class

- Why? If all methods are implemented, there is no need for abstract.
- Bad Example:

5. Default in Interfaces – No Need to Specify abstract

- What? Methods in interfaces are implicitly abstract unless marked default or static.
- Why? Reduces redundancy in code.
- Bad Example:
- Good Example:

Code

```
abstract class AbstractGame {
    abstract void initialize();
    abstract void startPlay();
    abstract void endPlay();

    public final void play() { // Defines the sequence
        initialize();
        startPlay();
        endPlay();
    }
}
```

Code

```
abstract class AbstractLogger {
    abstract void log(String message);
}

//Examples from JDK
public abstract class AbstractList<E>
public abstract class AbstractMap<K,V> implements Map<K,V>
```

Code

```
interface Flyable {
    void fly();
}
```

Java

```
abstract class Utility { // No need for abstract
    public void log(String message) {
        System.out.println(message);
    }
}
```

Code

```
interface Flyable {
    void fly();
    // abstract void fly(); // 'abstract' is redundant
    // public abstract void fly(); // also redundant
}
```

Code

```
interface Flyable {
    void fly(); // Abstract by default
}
```

Q5

Can You Explain Other Useful Modifiers in Java With Simple Examples?

Modifier	Purpose	Example
volatile	Each thread can cache variables in its own CPU core or registers. So, there might be a delay in visibility of updates from other threads. volatile ensures changes to a variable are visible to all threads immediately.	private volatile int count;
synchronized	Ensures thread safety by allowing only one thread to access a method/block at a time.	synchronized void method() { }
transient	Prevents a field from being serialized.	private transient String password;
native	Declares a method that is implemented in another language (e.g., C/C++).	native void nativeMethod();
strictfp	Ensures consistent floating-point calculations across platforms.	strictfp class Calculator { }
sealed	Restricts which classes can extend a class.	sealed class Vehicle permits Car, Bike { }
non-sealed	Allows a subclass of a sealed class to be extended further.	non-sealed class Car extends Vehicle { }

CHAPTER 12

Exception Handling in Java

10 Questions & Answers

Q1

Why is Exception Handling Important?

Exceptions Happen Regularly

- **Why?** Complex code, changing environments, and human errors
 - Defects Are Inevitable – Even great developers miss edge cases
 - Runtime Surprises – Files missing, servers down, data corrupted
 - External Systems Fail – APIs, databases, and networks aren't always reliable
 - Environment Changes – OS updates, memory limits, config issues, etc.
- **Moral of the Story?** Expect exceptions. Handle them gracefully.

Need to Handle Them Properly

- **What?** Handling exceptions isn't just about catching them
 - It's about solving them gracefully without scaring your end users!
- **1. User Communication**
 - Show friendly, non-technical error messages
 - Avoid app crashes or ugly stack traces
 - Display a unique error ID the user can report
 - Guide users on how to contact support
- **2. Debugging Support**
 - Log the full stack trace and context
 - Include the same error ID shown to the user
 - Capture input data, user actions, or environment details
 - Helps support/dev teams trace and fix the issue fast

Q2

Which Design Pattern Powers Java's Exception Flow?

- Chain of Responsibility: A behavioral pattern where a request moves through a chain of handlers
 - Why?: Decouples sender and receiver — each handler decides to process or pass along
 - How It Works:
 - Exception travels up the call stack method by method
 - Each method can catch or pass the exception higher
 - Stops when an appropriate handler is found
 - Real-World Analogy:
 - Like a leave request going from employee → manager → HR
 - Each can approve or escalate the request

Code

```
public static void main(String[] args) {
    method1();
}
private static void method1() {
    method2();
}
private static void method2() {
    String str = null;
    str.toString(); // Throws NullPointerException
}

//Exception in thread "main"
//java.lang.NullPointerException at
//method2(ExceptionHandlingExample1.java:15)
//at method1(ExceptionHandlingExample1.java:10)
//at main(ExceptionHandlingExample1.java:6)
```

Q3

Can You Walk Through a Practical Example of Java Exception Handling?

- PaymentApp Tries to make a payment of \$600 (more than balance) using PaymentService
 - Custom exception InsufficientFundsException extends Exception
 - PaymentService throws InsufficientFundsException if not enough money
 - PaymentApp Uses try-catch-finally to handle exceptions cleanly
 - Logs and prints appropriate error or success messages
 - Always prints "Transaction attempt completed" using finally

1. Try-Catch Block (Handling Exceptions Gracefully)

- What? Used to catch and handle exceptions to prevent program crashes.
- Where? In `main()`, different catch blocks handle specific exceptions.
- Key Points:

Multiple catch Blocks: Each block handles a different exception type.

Order of Exceptions -> Specific to General: `InsufficientFundsException` (custom exception) comes before `Exception` (general).

2. Custom Exception (`InsufficientFundsException`)

- What? A user-defined exception for handling specific errors.
- Why? Provides meaningful error messages instead of generic exceptions.
- Extends `Exception` Makes it a checked exception.

3. Throw and Throws (Propagating Exceptions)

• What? The `throw` statement generates exceptions, and `throws` declares them in the method signature.

- Where? Inside `PaymentService.makePayment()`

- Key Points: `throw` vs `throws`

`throw new InsufficientFundsException(...)` Generates the exception.

`throws InsufficientFundsException` Informs the caller (`main()`) to handle it.

4. Finally Block (Ensuring Cleanup Code Executes)

- What? The `finally` block (almost) always executes whether an exception occurs or not.
- Why? Used for cleanup tasks like closing connections or logging actions.
- Key Points:

Runs even if an exception occurs.

Used for releasing resources, or performing essential operations.

5. Catch-All Exception Handling (Unexpected Errors)

- What? The last catch block catches any unexpected exceptions.
- Why? Ensures the program doesn't crash due to an unhandled error.
- Key Points:

Catches any other runtime exceptions that are not explicitly handled.

Helps in debugging unexpected errors.

```

public class PaymentApp {
    public static void main(String[] args) {
        PaymentService paymentService = new PaymentService();

        try {
            paymentService.makePayment(600);
        } catch (InsufficientFundsException e) {
            System.err.println(
                "Error: " + e.getMessage());
        } catch (IllegalArgumentException e) {
            System.err.println(
                "Invalid Input: " + e.getMessage());
        } catch (Exception e) {
            System.err.println(
                "Unexpected error: " + e.getMessage());
        } finally {
            System.out.println(
                "Transaction attempt completed.");
        }
    }
}

class PaymentService {
    private static final Logger logger
        = Logger.getLogger(PaymentService.class.getName());

    private double balance = 500.0; // Initial balance

    public void makePayment(double amount)
        throws InsufficientFundsException {
        logger.info("Processing payment of $" + amount);

        if (amount <= 0) {
            throw new IllegalArgumentException(
                "Payment amount must be greater than zero.");
        }
        if (amount > balance) {
            throw new InsufficientFundsException(
                "Available balance: $"
                + balance);
        }

        balance -= amount;
        System.out.println(
            "Payment of $" + amount
            + " successful. Remaining balance: $"
            + balance);
    }
}

class InsufficientFundsException extends Exception {
    public InsufficientFundsException(String message) {
        super(message);
    }
}

```

Code

```

try {
    paymentService.makePayment(600);
} catch (InsufficientFundsException e) {
    System.err.println("Error: "
        + e.getMessage());
} catch (IllegalArgumentException e) {
    System.err.println("Invalid Input: "
        + e.getMessage());
} catch (Exception e) {
    System.err.println("Unexpected error: "
        + e.getMessage());
}

```

Code

```
class InsufficientFundsException extends Exception {  
    public InsufficientFundsException(String message) {  
        super(message);  
    }  
}
```

Code

```
public void makePayment(double amount)  
    throws InsufficientFundsException {  
  
    if (amount > balance) {  
        throw new InsufficientFundsException  
            ("Available balance: $" + balance);  
    }  
  
}
```

Java

```
finally {  
    // Almost Always executes  
    // Clean up!  
    System.out.println("Transaction attempt completed.");  
}
```

Code

```
catch (Exception e) {  
    System.err.println(  
        "Unexpected error: " + e.getMessage());  
}
```

Q4

What Happens When You Swallow Exceptions — And Why Is It Bad?

- What? An exception is caught but silently ignored
- Why It's Bad? Looks like everything is fine... but it's not
- No Logging = No Clues – Developer's can't trace what went wrong
- Program Continues – But state might be broken or inconsistent
- Result? Bugs hide, grow, and jump out later in weirder places
- Example of Exception Swallowing (Bad Practice)
- Why Is This a Problem?
 - No Clue Something Broke – Developer can't fix what they can't see
 - Silent Failures = Sneaky Bugs – App keeps running in a bad state
 - Delayed Explosions – Errors surface much later in unrelated places
 - Wasted Time – Debugging without logs is like finding a needle in a dark room
 - Always log, handle, or rethrow exceptions
 - Your future self (and your team) will thank you!

Java

```
try {
    int[] numbers = {1, 2};
    // ArrayIndexOutOfBoundsException
    System.out.println(numbers[3]);
} catch (Exception e) {
    // Exception is caught but ignored
}
System.out.println("Program continues...");
```

Q5

In What Scenarios is Code in finally Not Executed?

- Example Without finally (Problem Case)
 - Problem: If an exception occurs, opened connections may remain unclosed.
- Example With finally (Proper Resource Cleanup)
- finally is NOT executed in these two cases
 - Exception Inside finally – If something goes wrong inside finally, it may skip the rest
 - JVM Shutdown – If the JVM dies (e.g., System.exit() or crash), finally won't get a chance

Java

```
private static void method2() {
    Connection connection = new Connection();
    connection.open();
    try {
        // LOGIC
        String str = null;
        str.toString();
    } catch (Exception e) {
        System.out.println("Exception Handled - Method 2");
    }
    // Connection is never closed if exception occurs
}
```

Java

```
private static void method2() {
    Connection connection = new Connection();
    connection.open();
    try {
        String str = null;
        str.toString();
    } catch (Exception e) {
        System.out.println("Exception Handled - Method 2");
    } finally {
        // Ensures connection is always closed
        connection.close();
    }
}
```

Q6

What Combinations of try, catch, and finally Are Allowed?

- try + catch – Allowed
Handles exceptions, no need for finally
- try + finally – Allowed
Cleanup logic runs, even without handling exceptions
- try + catch + finally – Allowed
The full combo: handle errors + cleanup
- try Alone – Not allowed
Compilation error: must have at least a catch or finally

Code

```
try {
    riskyThing();
} catch (Exception e) {
    handleIt(e);
}
```

Code

```
try {
    riskyThing();
} finally {
    cleanup();
}
```

Code

```
try {
    riskyThing();
} catch (Exception e) {
    handleIt(e);
} finally {
    cleanup();
}
```

Code

```
try {
    riskyThing();
}
// Error: Missing catch or finally
```

Q7

What Is the Class Hierarchy Behind Java Exceptions?

Class Name	Category (Parent)	Checked?
Throwable	—	—
Error	Throwable	No
OutOfMemoryError	VirtualMachineError (Error)	No
StackOverflowError	VirtualMachineError (Error)	No
Exception	Throwable	Yes
IOException	Exception	Yes
SQLException	Exception	Yes
Class Not Found Exception	Exception	Yes
RuntimeException	Exception	No
NullPointerException	RuntimeException	No
ArrayIndex OutOfBounds Exception	RuntimeException	No
Illegal Argument Exception	RuntimeException	No
Arithmetic Exception	RuntimeException	No

Error vs Exception

Error

Used for critical system failures.

Cannot be handled by the programmer.

Examples: StackOverflowError, OutOfMemoryError.

Q8

How Do Checked and Unchecked Exceptions Differ in Java?

1. What Are Checked Exceptions?

- What? Subclasses of Exception excluding RuntimeException and its subclasses
- When? Compiler checks them at compile time
- Must Handle? Yes — use try-catch or declare with throws
- Why? These represent recoverable problems (like file not found)
- Common Checked Exceptions:
 - IOException – File or network access error
 - ClassNotFoundException – Class loading issue
- Compiler Says: "Handle it, or tell me you're throwing it!"

2. What are Unchecked (Runtime) Exceptions?

- What? Subclasses of RuntimeException
- When? Compiler doesn't force you to handle them
- Checked? Nope — they're unchecked at compile-time
- Why? These represent programming bugs or logic errors
- Common Unchecked Exceptions:
 - NullPointerException – Operation on a null variable
 - ArithmeticException – Divide by zero exception
 - ArrayIndexOutOfBoundsException – Bad array index
 - IllegalArgumentException – Invalid method argument

How to Choose Between Checked and Unchecked Exceptions?

- Use Checked – When the caller can and should handle the issue
 - Example: Missing file, invalid input, network timeout
 - Forces the developer to acknowledge the risk
- Use Unchecked – When the issue is a bug or misuse of API
 - Example: NullPointerException, IndexOutOfBoundsException
 - Caller usually can't recover

```
import java.io.*;

class FileReaderExample {
    public static void readFile()
        throws IOException {
        // Method must declare or handle IOException

        FileReader file = new FileReader("file.txt");
        BufferedReader br = new BufferedReader(file);
        System.out.println(br.readLine());
        br.close();
    }

    public static void main(String[] args) {
        try {
            readFile(); // Handling the checked exception
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Java

```
class UncheckedExample {
    public static void main(String[] args) {
        String text = null;

        // NullPointerException
        System.out.println(text.length());
    }
}
```

Q9

What Are Chained Exceptions?

- What? Chained exceptions allow you to link one exception to another
- Why? To show the original cause of an exception
- How? Use `initCause()` method
- Helps With? Debugging and tracing multi-layered errors
- Common Use Case:
 - You catch a low-level exception and wrap it inside a higher-level one
 - This way, you don't lose the original stack trace
- Example:

What Are the Best Practices for Handling Exceptions in Java?

- 1. Never Hide Exceptions – Always log them, even if you re-throw or recover
Why? Silent failures are hard to debug
Bad Example:
Good Example:
- 2. User-Friendly Messages – Show clear, non-technical messages to users
Why? Users shouldn't see scary stack traces
Example (Spring Controller):
- 3. Provide Debugging Info – Log stack traces and context for developers
Why? Helps devs trace root cause
Example:
- 4. Global Exception Handling – Use centralized mechanisms (like @ControllerAdvice in Spring)
Why? Avoids duplicate try-catch blocks everywhere
Example:
- 5. Avoid Misusing Exceptions – Don't use exceptions for expected logic like looping or validation
Why? Bad for performance and readability
Bad Example:
Good Example:

Java

```
public class ChainedExceptionDemo {
    public static void main(String[] args) {
        try {
            validate(null);
        } catch (Exception e) {
            System.out.println("Exception: " + e);
            System.out.println("Caused by: "
                               + e.getCause());
        }
    }

    static void validate(String name) throws Exception {
        try {
            if (name == null) {
                throw new NullPointerException(
                    "Username is null");
            }
        } catch (NullPointerException e) {
            Exception wrapped
                = new Exception(
                    "User validation failed");

            // Linking original cause
            wrapped.initCause(e);

            throw wrapped;
        }
    }
}
```

Code

```
try {
    riskyOperation();
} catch (Exception e) {
    // Nothing logged, issue disappears
}
```

Code

```
try {
    riskyOperation();
} catch (Exception e) {
    // Log the exception
    logger.error("Something went wrong", e);

    throw e; // Optional: rethrow if needed
}
```

Code

```
try {
    riskyOperation();
} catch (Exception e) {
    // Nothing logged, issue disappears
}
```

Code

```
try {
    riskyOperation();
} catch (Exception e) {
    // Log the exception
    logger.error("Something went wrong", e);

    throw e; // Optional: rethrow if needed
}
```

Code

```
@GetMapping("/item")
public ResponseEntity<?> getItem() {
    try {
        return ResponseEntity.ok(service.getItem());
    } catch (ItemNotFoundException e) {
        return ResponseEntity
            .status(404)
            .body("Item not found." +
                " Please check the ID.");
    }
}
```

Code

```

@GetMapping("/item")
public ResponseEntity<?> getItem() {
    try {
        return ResponseEntity.ok(service.getItem());
    } catch (ItemNotFoundException e) {
        return ResponseEntity
            .status(404)
            .body("Item not found." +
                " Please check the ID.");
    }
}

```

Code

```

try {
    processData();
} catch (Exception e) {
    logger.error(
        "Failed to process data for user: "
        + userId, e);
}

```

Code

```

try {
    processData();
} catch (Exception e) {
    logger.error(
        "Failed to process data for user: "
        + userId, e);
}

```

Java

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<String> handleBadInput(
        Exception e) {
        return ResponseEntity
            .badRequest()
            .body(
                "Invalid input. Please try again.");
    }
}

```

Java

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<String> handleBadInput(
        Exception e) {

        return ResponseEntity
            .badRequest()
            .body(
                "Invalid input. Please try again.");
    }
}
```

Code

```
while (true) {
    try {
        return Integer.parseInt(input);
    } catch (NumberFormatException e) {
        // Using exception to control loop
        input = getNewInput();
    }
}
```

Code

```
// Validate before parsing
while (!isNumeric(input)) {
    input = getNewInput();
}
return Integer.parseInt(input);
```

Code

```
while (true) {
    try {
        return Integer.parseInt(input);
    } catch (NumberFormatException e) {
        // Using exception to control loop
        input = getNewInput();
    }
}
```

Code

```
// Validate before parsing
while (!isNumeric(input)) {
    input = getNewInput();
}
return Integer.parseInt(input);
```

Q10

What Are the Newer Java Features That Help with Exception Handling?

1. Try-With-Resources (Automatic Resource Management)

- What? A try block that automatically closes resources
- Why? No need for manual finally cleanup
- When? Use with classes that implement AutoCloseable
- Benefit? Less boilerplate, fewer resource leaks
- Common Resources:
BufferedReader, FileInputStream, Connection, Scanner

2. Multi-Catch Block

- What? Handle multiple exception types in one catch block
- How? Use | to separate exception classes
- Why? Avoids repeating the same logic for different exceptions
- When? When multiple exceptions need the same handling

3. Optional.orElseThrow()

- What? Throws an exception if the value is absent
- Why? Avoids null checks and improves readability
- When? Use when a value is required and absence is exceptional
- Benefit? Cleaner, expressive, and avoids if (optional.isPresent())

Java

```
try (BufferedReader br =
    new BufferedReader(
        new FileReader("file.txt"))) {
    System.out.println(br.readLine());
} catch (IOException e) {
    e.printStackTrace();
}

//Use with classes that implement AutoCloseable
public interface AutoCloseable {
    void close() throws Exception;
}
```

Code


```
try {
    methodThatThrowsException();

    // Multi-catch block
} catch (IOException | SQLException ex) {
    System.err.println(
        "Exception occurred: "
        + ex.getMessage());
}
```

Code

```
Optional<String> name = Optional.ofNullable(null);
String result = name.orElseThrow(
    () -> new IllegalArgumentException(
        "Value is missing"));
```

CHAPTER 13

Serialization in Java

7 Questions & Answers

Q1

What is Serialization?

- What Is Serialization?
 - Serialization – Converts an object's state into a byte stream
 - Deserialization – Rebuilds the object from that byte stream
- Why Is Serialization Needed?
 - Saving Objects – Write to files or databases
 - Transferring Objects – Send across networks (e.g., sockets, APIs)
 - Caching – Store objects for faster reuse
 - Distributed Systems – Transfer data between JVMs
- Think of It Like:
 - Packing your object into a suitcase (byte stream) so it can travel!
 - Then unpacking it at the destination to bring it back to life!

Q2

How to Serialize an Object?

- Step 1: Class must implement Serializable
 - Marker Interface – Serializable has no methods
 - Tells the JVM that this class can be serialized
- Step 2: Use ObjectOutputStream to write the object
- Step 3: Save it to a .ser file
- Example: Serializing a Rectangle Object

Java

```

import java.io.FileOutputStream;
import java.io.ObjectOutputStream;
import java.io.Serializable;

class Rectangle implements Serializable {
    int length;
    int breadth;
    int area;

    public Rectangle(int length, int breadth) {
        this.length = length;
        this.breadth = breadth;
        this.area = length * breadth;
    }
}

public class SerializationExample {
    public static void main(String[] args)
        throws Exception {

        FileOutputStream fileStream
            = new FileOutputStream("Rectangle.ser");

        ObjectOutputStream objectStream
            = new ObjectOutputStream(fileStream);

        // Serializing object
        Rectangle rect = new Rectangle(5, 6);
        objectStream.writeObject(rect);

        objectStream.close();

        System.out.println(
            "Rectangle object serialized.");
    }
}

```

Q3

How to Deserialize an Object in Java?

- What Is Deserialization?
Deserialization – Reconstructs an object from a byte stream
Used For – Reading back previously saved or transmitted objects
- How Does It Work?
Reads the .ser file using ObjectInputStream
Converts the byte stream back into a Java object
Requires the class to be present on the classpath
- Example: Deserialization of a Rectangle Object

Java

```

import java.io.FileInputStream;
import java.io.ObjectInputStream;

public class DeserializationExample {
    public static void main(String[] args)
        throws Exception {

        FileInputStream fileInputStream =
            new FileInputStream("Rectangle.ser");

        ObjectInputStream objectInputStream =
            new ObjectInputStream(fileInputStream);

        Rectangle rectangle =
            (Rectangle) objectInputStream.readObject();

        objectInputStream.close();

        System.out.println(
            "Length: " + rectangle.length);    // 5
        System.out.println(
            "Breadth: " + rectangle.breadth); // 6
        System.out.println(
            "Area: " + rectangle.area);        // 30
    }
}

```

Q4

How to Exclude Specific Parts of an Object from Serialization?

- Why Use transient?
 - What? Marks a field to be skipped during serialization
 - Why? Some data shouldn't or doesn't need to be saved
- Common Use Cases:
 - Sensitive Data – Passwords, secrets, API keys
 - Derived Fields – Values that can be recomputed
 - Non-Serializable Fields – Like Thread, Socket, Logger, etc.
- Use transient when saving that field is unnecessary, risky, or problematic.
- Example: Marking area as transient
- Deserialization Output:
- Remember: Static variables belong to the class, not an instance.
 - Since serialization works on objects, static variables are ignored.

Python

```
import java.io.Serializable;

class Rectangle implements Serializable {
    int length;
    int breadth;
    transient int area; // Not serialized

    public Rectangle(int length, int breadth) {
        this.length = length;
        this.breadth = breadth;
        this.area = length * breadth;
    }
}
```

Code

```
Length: 5
Breadth: 6
Area: 0 // Area is lost because it was marked as transient
```

Q5

Are Constructors Invoked During Deserialization?

- No. When an object is deserialized, constructors are NOT called.
- The object is restored directly from the byte stream with its previous state.

Example: Checking if Constructor is Called

Output:

During Deserialization: No constructor message appears, confirming constructors are not executed during Deserialization.

Java

```
import java.io.*;

class Demo implements Serializable {
    public Demo() {
        System.out.println("Constructor called!");
    }
}

public class ConstructorCheck {
    public static void main(String[] args) throws Exception {
        // Serialization
        System.out.println("Serialization Started");
        FileOutputStream fos = new FileOutputStream("Demo.ser");
        ObjectOutputStream oos = new ObjectOutputStream(fos);
        // Constructor will be called here
        oos.writeObject(new Demo());
        oos.close();
        System.out.println("Serialization Done");

        // Deserialization
        System.out.println("Deserialization Started");
        FileInputStream fis = new FileInputStream("Demo.ser");
        ObjectInputStream ois = new ObjectInputStream(fis);
        Demo d = (Demo) ois.readObject();
        ois.close();
        System.out.println("Deserialization Done");
    }
}
```

Code

```
Serialization Started
Constructor called!
Serialization Done
Deserialization Started
Deserialization Done
```

Q6

What is serialVersionUID and Why Is It Important?

- serialVersionUID: Unique identifier used by Java to verify object compatibility during deserialization

- No serialVersionUID? – JVM generates it automatically based on class structure
 - Structure Changes? – Even a small change creates a different UID
 - Breaks Compatibility – Old serialized objects can't be deserialized with the new class
 - Throws Exception – Causes InvalidClassException at runtime
- Fix? – Define serialVersionUID manually to keep versioning under control

Why Is serialVersionUID Important?

- Controls Class Versioning Allows developers to explicitly define compatibility rules.
- Ensures Backward Compatibility Helps deserialize older versions of an object even if the class has changed slightly.

Example Without serialVersionUID If a class is modified after serialization without a serialVersionUID, Java generates a new UID automatically, breaking compatibility.

Step 1: Serialize the Object

Step 2: Modify the Class (Add a New Field)

Step 3: Try to Deserialize (Fails)

Fix: Explicitly Define serialVersionUID By manually specifying serialVersionUID, Java ensures that even after modifying the class, deserialization works as expected (if changes are compatible).

What should you do?

Change

Add a new non-transient field

Remove an existing field

Change field data type

Change method logic

Key Takeaway: Update serialVersionUID only if changes break compatibility (e.g., removing a field or changing its type).

Code

```
private static final long serialVersionUID = 123456789L;
```

Python

```
import java.io.*;

class Rectangle implements Serializable {
    int length;
    int breadth;

    public Rectangle(int length, int breadth) {
        this.length = length;
        this.breadth = breadth;
    }
}
```


Code

```
FileOutputStream fileStream
    = new FileOutputStream("Rectangle.ser");

ObjectOutputStream objectStream
    = new ObjectOutputStream(fileStream);

objectStream.writeObject(new Rectangle(5, 6));

objectStream.close();
```

Code

```
class Rectangle implements Serializable {
    int length;
    int breadth;
    int area; // New field added
}
```

Code

```
FileInputStream fileInputStream
    = new FileInputStream("Rectangle.ser");

ObjectInputStream objectInputStream
    = new ObjectInputStream(fileInputStream);

Rectangle rectangle // Throws InvalidClassException
    = (Rectangle) objectInputStream.readObject();

objectInputStream.close();
```

Code

```
class Rectangle implements Serializable {
    private static final long serialVersionUID = 123456789L;

    int length;
    int breadth;
    int area; // New field added
}
```

Q7

What is Externalizable and Its Advantages Over Serializable?

- What? Externalizable gives you full control over what gets serialized and how
- How? You must implement both writeExternal() and readExternal() methods
- Why? To customize, compress, or skip unnecessary fields
- Difference from Serializable? Java auto-handles Serializable; you write logic manually with Externalizable
- When to Use? For selective serialization

Key Differences: Serializable vs. Externalizable

Feature	Serializable
Serialization Control	Java handles automatically
Performance	Slower (writes all fields)
Customization	No control over fields being serialized
Constructor Invocation	Skips constructor during deserialization
Methods to Implement	No methods (marker interface)
Use Case	Simple serialization

Example: Using Externalizable for Custom Serialization

Serialization & Deserialization Using Externalizable

Why Use Externalizable?

- More efficient Avoids serializing unnecessary fields.
- More control Manually control encryption, compression, and transient fields.
- Custom initialization Allows recalculating transient values like area.

Java

```

import java.io.*;

class Rectangle implements Externalizable {
    int length;
    int breadth;
    transient int area;

    // Default constructor required for Externalizable
    public Rectangle() {}

    public Rectangle(int length, int breadth) {
        this.length = length;
        this.breadth = breadth;
        this.area = length * breadth;
    }

    @Override
    public void writeExternal(ObjectOutput out)
        throws IOException {
        out.writeInt(length);
        out.writeInt(breadth);
    }

    @Override
    public void readExternal(ObjectInput in)
        throws IOException {
        length = in.readInt();
        breadth = in.readInt();

        // Recalculate area
        area = length * breadth;
    }
}

```

Java

```

public class ExternalizableExample {
    public static void main(String[] args)
        throws Exception {

        Rectangle rect = new Rectangle(5, 6);

        // Serialization
        FileOutputStream fos
            = new FileOutputStream("RectangleExternal.ser");
        ObjectOutputStream oos
            = new ObjectOutputStream(fos);
        oos.writeObject(rect);
        oos.close();

        // Deserialization
        FileInputStream fis
            = new FileInputStream("RectangleExternal.ser");
        ObjectInputStream ois
            = new ObjectInputStream(fis);
        Rectangle deserializedRect
            = (Rectangle) ois.readObject();
        ois.close();

        System.out.println(
            "Length: " + deserializedRect.length);
        System.out.println(
            "Breadth: " + deserializedRect.breadth);
        System.out.println(
            "Area: " + deserializedRect.area);
    }
}

```


CHAPTER 14

Built-in Classes in Java

9 Questions & Answers

Q1 **Object**

What? Object is the root class for all Java classes.

Why? Ensures that all Java objects have basic functionality, such as comparison (`equals()`), hashing (`hashCode()`), and string representation (`toString()`). In addition, there are several methods related to synchronizing threads (`wait()`, `notify()`, `notifyAll()`).

An Example:

Common Methods in `java.lang.Object`

Method	Type
<code>hashCode()</code>	Instance
<code>toString()</code>	Instance
<code>getClass()</code>	Instance
<code>clone()</code>	Instance
<code>finalize()</code>	Instance
<code>wait()</code>	Instance
<code>wait(long timeoutMillis)</code>	Instance
<code>wait(long timeoutMillis, int nanos)</code>	Instance
<code>notify()</code>	Instance
<code>notifyAll()</code>	Instance

Java

```

class Example {
    int id;

    Example(int id) {
        this.id = id;
    }

    @Override
    public String toString(){
        return "Example ID: " + id;
    }
}

public class ObjectExample {
    public static void main(String[] args) {
        Example obj = new Example(10);
        System.out.println(obj.toString());
        // Output: Example ID: 10
    }
}

```

Q2

Void

What? Void is a special class that represents the absence of a return value.

Example Usecase Used in generics where a method does not return any value.

Example: Future<Void>

Example: Using Void with Generics

Java

```

import java.util.concurrent.*;

public class FutureVoidExample {
    public static void main(String[] args)
        throws Exception {
        ExecutorService executor =
            Executors.newSingleThreadExecutor();

        Future<Void> future = executor.submit(() -> {
            System.out.println("Running task...");
            // Do something, but return nothing
            return null;
        });

        // Wait for task to complete
        future.get(); // Returns null
        System.out.println("Task done!");

        executor.shutdown();
    }
}

```

Java

```

class Task<I, R> {
    private I input;
    private R result;

    public Task(I input, R result) {
        this.input = input;
        this.result = result;
    }

    public R process() {
        System.out.println("Processing input: " + input);

        if (result != null) {
            System.out.println("Returning result: " + result);
        } else {
            System.out.println("Returning no result (Void)");
        }

        return result;
    }
}

Task<String, Integer> wordLength = new Task<>("Hello", 5);
Integer length = wordLength.process();
// Output:
// Processing input: Hello
// Returning result: 5

Task<String, Void> logTask = new Task<>("Log this", null);
Void result = logTask.process();
// Output:
// Processing input: Log this
// Returning no result (Void)

```

Q3

Runtime

What? Runtime allows interaction with the JVM, such as executing system commands and managing memory.

Why? Used for memory optimization and running external system processes.

An Example:

Java


```

public class RuntimeExample {
    public static void main(String[] args) throws Exception {
        Runtime runtime = Runtime.getRuntime();

        // 1 Get Available Processors (CPU Cores)
        System.out.println("Available Processors: "
            + runtime.availableProcessors());

        // 2 Get Free Memory in JVM
        System.out.println("Free Memory: "
            + runtime.freeMemory() + " bytes");

        // 3 Get Total Memory in JVM
        System.out.println("Total Memory: "
            + runtime.totalMemory() + " bytes");

        // 4 Get Maximum Memory JVM Can Use
        System.out.println("Max Memory: "
            + runtime.maxMemory() + " bytes");

        // 5 Run an External System Command
        Process process = runtime.exec(
            "echo Hello, World!");
        // Wait for command to finish
        process.waitFor();

        // 6 Request Garbage Collection
        runtime.gc();

        // 7 Shutdown the JVM
        // 0 (Normal Exit)
        // Non Zero (abnormal termination)
        // runtime.exit(0);
    }
}

```

Q4

System

What? System provides system-level functionalities like console I/O, environment variables, and system properties.

Why? Used for printing output, reading input, and retrieving environment settings.

An Example:

Java

```

public class SystemExample {
    public static void main(String[] args) {
        // 1 Retrieve Java Version
        System.out.println("Java Version: "
            + System.getProperty("java.version"));

        // 2 Retrieve Operating System Name
        System.out.println("OS Name: "
            + System.getProperty("os.name"));

        // 3 Retrieve User's Home Directory
        System.out.println("User Home: "
            + System.getProperty("user.home"));

        // 4 Retrieve an Environment Variable (e.g., PATH)
        System.out.println("System PATH: "
            + System.getenv("PATH"));

        // 5 Measure Execution Time of a Task
        long startTime = System.nanoTime();

        // Simulated workload
        for (int i = 0; i < 1000000; i++) {}

        long endTime = System.nanoTime();

        System.out.println("Execution Time (ns): "
            + (endTime - startTime));

        // 6 Print an Error Message
        System.err.println("This is an error message.");

        // 7 Exit the Program with Status Code
        // 0 (Normal Exit)
        // Non Zero (abnormal termination)
        // System.exit(0);

    }
}

```

Q5

ThreadLocal

What? ThreadLocal allows each thread to have its own copy of a variable.

Why? Used in multi-threaded applications to maintain thread-specific data.

An Example:

Java

```

class ThreadLocalExample {
    private static ThreadLocal<Integer> threadLocalCount =
        ThreadLocal.withInitial(() -> 0);

    public static void main(String[] args) {
        Runnable task = () -> {

            int count = threadLocalCount.get();
            threadLocalCount.set(count + 1);

            System.out.println(
                Thread.currentThread().getName() +
                " - " + threadLocalCount.get());
        };

        Thread t1 = new Thread(task);
        Thread t2 = new Thread(task);

        t1.start();
        t2.start();
    }
}

```

Q6

UUID

What? UUID (Universally Unique Identifier) generates unique 128-bit identifiers.

Why?

- Ensures uniqueness without needing centralized ID management.
- Useful for database primary keys, transaction IDs, and session tracking.
- Prevents collisions better than traditional sequential IDs.

An Example:

Java

```

import java.util.UUID;

public class UUIDExample {
    public static void main(String[] args) {

        UUID uniqueID = UUID.randomUUID();

        System.out.println("Generated UUID: " + uniqueID);
    }
}

```

Q7

Files

What? Files is a utility class for reading, writing, and managing files.

Why? Simplifies file operations without requiring `FileInputStream` or `BufferedReader`.

An Example:

Another Example

Common Methods in `java.nio.file.Files`

Method	Type	Purpose
<code>exists(Path)</code>	Static	Checks if a file or directory exists
<code>notExists(Path)</code>	Static	Checks if a file does not exist
<code>createFile(Path)</code>	Static	Creates a new empty file
<code>createDirectory(Path)</code>	Static	Creates a single directory
<code>createDirectories(Path)</code>	Static	Creates directory + all missing parents
<code>delete(Path)</code>	Static	Deletes file or empty directory
<code>copy(Path, Path)</code>	Static	Copies file to a new location
<code>move(Path, Path)</code>	Static	Moves or renames a file
<code>readAllLines(Path)</code>	Static	Reads all lines from a file (as List)
<code>write(Path, byte[] or List)</code>	Static	Writes bytes or lines to file
<code>size(Path)</code>	Static	Returns file size in bytes
<code>isDirectory(Path)</code>	Static	Checks if the path is a directory
<code>isRegularFile(Path)</code>	Static	Checks if the path is a regular file
<code>lines(Path)</code>	Static	Stream lines from a file (lazy reading)

Java

```
import java.nio.file.Files;
import java.nio.file.Path;

public class FilesExample {
    public static void main(String[] args) throws Exception {

        String content
            = Files.readString(Path.of("example.txt"));

        System.out.println("File Content: " + content);

    }
}
```

Java

```
import java.nio.file.Files;
import java.nio.file.Path;

public class AdvancedFilesExample {
    public static void main(String[] args) throws Exception {
        Path filePath = Path.of("example.txt");

        // 1 Write to a file
        Files.writeString(filePath, "Hello, Java!");

        // 2 Check if file exists
        System.out.println("File Exists: "
            + Files.exists(filePath)); // Output: true

        // 3 Get file size
        System.out.println("File Size: "
            + Files.size(filePath) + " bytes");

        // 4 Copy file to a new location
        Files.copy(filePath, Path.of("copy_example.txt"));

        // 5 Delete the copied file
        Files.delete(Path.of("copy_example.txt"));

    }
}
```

Q8

Timer

What? Timer schedules tasks for execution at a fixed rate or delay.

Why? Useful for running periodic background tasks.

An Example: The Timer starts after 2 seconds (2000ms) and repeats every 3 seconds (3000ms).

Java

```
import java.util.Timer;
import java.util.TimerTask;

public class TimerExample {
    public static void main(String[] args) {

        Timer timer = new Timer();

        timer.schedule(new TimerTask() {
            @Override
            public void run() {
                System.out.println("Task executed!");
            }
        }, 2000, 3000);
    }
}
```

Q9

Locale

What? Locale represents geographical, cultural, and political regions for internationalization.

Why? Used to format numbers, dates, and messages based on the user's location.

An Example:

Code

```
//Example Locales
public static final byte ENGLISH = 0,
    FRENCH = 1,
    GERMAN = 2,
    ITALIAN = 3,
    JAPANESE = 4,
    KOREAN = 5,
    CHINESE = 6,
    SIMPLIFIED_CHINESE = 7,
    TRADITIONAL_CHINESE = 8,
    FRANCE = 9,
    GERMANY = 10,
    ITALY = 11,
    JAPAN = 12,
    KOREA = 13,
    UK = 14,
    US = 15,
    CANADA = 16,
    CANADA_FRENCH = 17;
```

Java

```
import java.text.NumberFormat;
import java.util.Locale;

public class LocaleExample {
    public static void main(String[] args) {
        Locale locale = Locale.FRANCE;

        System.out.println("Country: "
            + locale.getDisplayCountry()); // Output: France

        System.out.println("Language: "
            + locale.getDisplayLanguage()); // Output: French

        double amount = 1234.56;

        NumberFormat currencyFormatter
            = NumberFormat.getCurrencyInstance(Locale.US);

        System.out.println("Formatted Currency: "
            + currencyFormatter.format(amount)); //$1,234.56

        //Locale.UK - £1,234.56
    }
}
```

CHAPTER 15

Miscellaneous Topics in Java

11 Questions & Answers

Q1

What are some best practices for using enums in Java?

- How does Enum help?

Fixed Set of Constants – Define all allowed values in one place

Type Safe – Only valid enum values can be used

Readable Code – Improves clarity over magic strings or numbers

Prevents Errors – No typos or invalid assignments

Switch Friendly – Works great in switch statements

- JDK Examples

`java.time.DayOfWeek` (enum representing the 7 days of the week)

`java.util.concurrent.TimeUnit` (Represents time durations - NANoseconds, MICROseconds, MILLISECONDS, SECONDS, MINUTES, HOURS, DAYS)

`java.lang.Thread.State`

- Simple Example:

- Using switch with Enums

- Enum with Fields, Constructor, and Methods

- Enum Best Practices

Uppercase Names – Use all caps for enum constants (MONDAY, HIGH, STARTED)

Add Fields & Methods – Enums can have constructors, fields, and behavior

Avoid ordinal() for Logic – Enum order can change, leading to bugs

Use switch or switch expressions – Clean and readable for branching logic

Use Enums Instead of Strings – Prevents typos and invalid values

Don't Subclass Enums – Enums are implicitly final

Override toString() if Needed – For user-friendly display values

Use EnumSet / EnumMap – Optimized for enum keys

Code

```
public enum State {  
    NEW,  
    RUNNABLE,  
    BLOCKED,  
    WAITING,  
    TIMED_WAITING,  
    TERMINATED;  
}
```

Code

```

public enum State {
    NEW,
    RUNNABLE,
    BLOCKED,
    WAITING,
    TIMED_WAITING,
    TERMINATED;
}

```

Code

```

enum Status {
    PENDING, APPROVED, REJECTED;
}

Status status = Status.APPROVED;

```

Code

```

enum Season {
    WINTER, SPRING, SUMMER, FALL
}

class Weather {
    public int getMaxTemperature(
        Season season) {
        return switch (season) {
            case WINTER -> 5;
            case SPRING, FALL -> 15;
            case SUMMER -> 30;
        };
    }
}

```

Java

```

enum Direction {
    NORTH("Up"),
    SOUTH("Down"),
    EAST("Right"),
    WEST("Left");

    private final String movement;

    Direction(String movement) {
        this.movement = movement;
    }

    public String getMovement() {
        return movement;
    }
}

public class EnumExample {
    public static void main(String[] args) {
        System.out.println(
            Direction.NORTH.getMovement());
    }
}

```

- What? – EnumSet is a specialized Set for enum types
- Why? – Much faster than HashSet when working with enums
- How? – Internally uses a bit vector for storage
- Null Allowed? – No null values allowed
- All Constants? – Use EnumSet.allOf(EnumType.class)
- Empty Set? – Use EnumSet.noneOf(EnumType.class)
- Range of Values? – Use EnumSet.range(A, C)
- Example: Using EnumSet
- Best Practices for EnumSet
 - Use EnumSet for Enums – Better than HashSet in speed and memory
 - Faster Operations – add(), remove(), contains() are constant-time
 - No Nulls Allowed – Inserting null throws NullPointerException
 - Cleaner Code – Methods like of(), allOf(), range() make code more readable

Java

```
import java.util.EnumSet;

enum Day {
    MONDAY, TUESDAY, WEDNESDAY,
    THURSDAY, FRIDAY, SATURDAY, SUNDAY
}

public class EnumSetExample {
    public static void main(String[] args) {

        // Use Case 1: Weekend Days
        EnumSet<Day> weekend =
            EnumSet.of(Day.SATURDAY, Day.SUNDAY);
        System.out.println("Weekend: " + weekend);

        // Use Case 2: Working Days
        EnumSet<Day> weekdays =
            EnumSet.range(Day.MONDAY, Day.FRIDAY);
        System.out.println("Weekdays: " + weekdays);

        // Use Case 3: All Days
        EnumSet<Day> allDays =
            EnumSet.allOf(Day.class);
        System.out.println("All Days: " + allDays);

        // Use Case 4: Empty Set (will add later)
        EnumSet<Day> holidays =
            EnumSet.noneOf(Day.class);
        holidays.add(Day.FRIDAY); // dynamically add
        System.out.println("Holidays: " + holidays);

        // Use Case 5: Removing from a set
        weekdays.remove(Day.WEDNESDAY);
        System.out.println(
            "Working Days (No Wednesday): " + weekdays);
    }
}
```

Q3

What is the purpose of using assertions in Java?

- What? – Assertions check conditions that must be true during development
- How? – Use `assert condition;` or `assert condition : "message";`
- Fails? – Throws `AssertionError` if the condition is false
- When? – Only works if assertions are enabled (`-ea` flag)
- Not for Production – Meant for testing, not for handling runtime errors
- Example: Using `assert`
- Running with assertions enabled:
- Asserts - Best Practices
 - Validate Assumptions – Use assertions to catch logic errors during development
 - Pre/Post Conditions – Check method input/output expectations in private/internal code
 - Never for User Input Validation – Don't validate user input with assertions
 - Never for Business Logic – Business rules must throw proper exceptions
 - Don't Use for Security – Never rely on assertions for security-sensitive checks
 - Add Messages – Use `assert condition : "Helpful message"` for clarity

Code

```
public void processFile(String fileNameTxt) {  
    assert fileNameTxt.endsWith(".txt")  
        : "Invalid file type: " + fileNameTxt;  
    //Other code to process the file!  
}
```

Code

```
java -ea AssertionExample
```

Q4

When should you use variable arguments (varargs) in a method?

- What? – Varargs (...) let a method take zero or more arguments
- Syntax? – Use type... name as the last parameter in the method
- Internally? – Treated as an array inside the method
- Why? – Makes method calls flexible without overloading
- JDK Examples?
 - String.format(String format, Object... args)
 - Collections.addAll(Collection<? super T> c, T... elements)
 - List.of(E... elements)
- Example: Using Varargs
- Best Practices for Varargs
 - Use When Necessary – Avoid overusing varargs; prefer specific parameters if possible
 - Only One Vararg – A method can't have more than one varargs parameter
 - Always Last – Varargs must be the last parameter in the method signature
 - Avoid Ambiguity – Overloaded methods with varargs can confuse the compiler
 - Handle Nulls Safely – Varargs can be null; check before using
 - Combine with Streams – Use functional style (Arrays.stream(...)) for clean code

Java

```
public class VarargsExample {
    public static int sum(int... numbers) {
        return Arrays.stream(numbers).sum();
    }

    public static void main(String[] args) {
        System.out.println(sum(1, 2, 3));
        System.out.println(sum(5, 10, 15, 20));
        System.out.println(sum());
    }
}
```

Q5

What are the best practices for working with date and time APIs in Java?

Evolution of Date and Time APIs

API	Introduced In	Issues
java.util.Date	Java 1.0	Mutable, time zone issues, bad API design
java.util.Calendar	Java 1.1	Complex, still mutable, verbose
java.time (LocalDate, LocalTime, ZonedDateTime)	Java 8	Immutable, easy to use, thread-safe

Example: `java.util.Date` (Not Recommended)

Example: `Calendar` (Better, but still not ideal)

Best Practice: Use `java.time` API (Java 8+)

Advantages of `java.time` API:

- Immutable and thread-safe.
- Clear and concise API.
- Better formatting support (`DateTimeFormatter`).

Best Practices for Date and Time Handling

- Prefer `java.time` over `Date` and `Calendar`.
- Use `DateTimeFormatter` for proper formatting instead of `SimpleDateFormat`.

Java

```
import java.util.Date;

public class DateExample {
    public static void main(String[] args) {
        Date date = new Date();

        // Output: Mon Mar 04 12:30:45 IST 2024
        System.out.println("Current Date: " + date);
    }
}
```

Java

```
import java.util.Calendar;

public class CalendarExample {
    public static void main(String[] args) {
        Calendar calendar = Calendar.getInstance();
        calendar.set(2024, Calendar.MARCH, 4);
        System.out.println("Date: " + calendar.getTime());
    }
}
```

Java

```

import java.time.*;
import java.time.format.DateTimeFormatter;
import java.time.temporal.ChronoUnit;

public class JavaTimeExample {
    public static void main(String[] args) {

        // 1 LocalDate - Immutable, thread-safe (unlike Date)
        LocalDate today = LocalDate.now();
        System.out.println("Today's Date: " + today);

        // 2 LocalDateTime + Formatting (no SimpleDateFormat)
        LocalDateTime dateTime = LocalDateTime.now();
        DateTimeFormatter formatter =
            DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");
        System.out.println("Formatted: " +
            dateTime.format(formatter));

        // 3 Specific Date - Easier than Calendar
        // No month-0 confusion (Jan = 1, not 0)
        LocalDate independenceDay =
            LocalDate.of(1947, 8, 15);
        System.out.println("Independence Day: " +
            independenceDay);

        // 4 Add Days/Months - Fluent API, clear syntax
        LocalDate nextMonth = today.plusMonths(1);
        System.out.println("Next Month: " + nextMonth);

        LocalDate tenDaysLater = today.plusDays(10);
        System.out.println("10 Days Later: " +
            tenDaysLater);

        // 5 Subtract Time - No confusing Calendar math
        LocalDate lastYear = today.minusYears(1);
        System.out.println("Last Year: " + lastYear);

        // 6 Change Fields
        LocalDate changedYear = today.withYear(2020);
        System.out.println("Changed Year: " +
            changedYear);

        LocalDate firstDay = today.withDayOfMonth(1);
        System.out.println("First Day: " + firstDay);

        // 7 Compare Dates - Easier than Date.before/after
        boolean isBefore = independenceDay.isBefore(today);
        System.out.println("Is Before Today? " + isBefore);

        boolean isAfter = today.isAfter(independenceDay);
        System.out.println("Is After Independence? " +
            isAfter);

        // 8 Days Between - Use ChronoUnit directly
        long days = ChronoUnit.DAYS.between(
            independenceDay, today);
        System.out.println("Days Since 1947: " + days);
    }
}

```

- **What? – Reflection lets you inspect and interact with code at runtime**

Used For? – Dynamic loading, calling methods, accessing fields, working with annotations

Common Uses

Dependency injection

Test frameworks

Annotation processing

- **JDK API Classes – Class, Method, Field, Constructor (from java.lang.reflect)**

- **Example: Getting Class Information Using Reflection**

- **Example: Creating Objects Using Reflection**

- **Best Practices with Reflection:**

Use When Needed – Prefer reflection only for special use cases (frameworks, testing)

Avoid in Hot Paths – Reflection is slower than direct access — not for performance-critical code

Handle Exceptions Properly – Catch and log `ClassNotFoundException`, `NoSuchMethodException`, etc.

Respect Security – Accessing private fields/methods may break encapsulation — use with caution

Document Usage – Clearly explain why reflection is used in code comments

Avoid Reflection for Business Logic – Keep it in utilities or framework-level code only

Test Thoroughly – Reflective code is harder to debug — write strong tests to verify behavior

Java

```
import java.lang.reflect.Method;

public class ReflectionExample {
    public static void main(String[] args) {
        Class<String> clazz = String.class;

        // 1 Print declared methods of String class
        for (Method method :
            clazz.getDeclaredMethods()) {
            System.out.println(
                "Method: " + method.getName());
        }
    }
}
```

Java


```
import java.lang.reflect.Constructor;

class Example {
    public void display() {
        System.out.println("Reflection Example");
    }
}

public class ReflectionConstructorExample {
    public static void main(String[] args)
        throws Exception {

        Class<Example> clazz = Example.class;

        // Create instance dynamically
        Constructor<Example> constructor
            = clazz.getDeclaredConstructor();

        Example obj = constructor.newInstance();

        obj.display(); // Output: Reflection Example
    }
}
```

Q7

Why are annotations used in Java?

- What? – Annotations are special markers that add metadata to code
- JDK Examples:
 - @Override – Tells the compiler you're overriding a super-class method or implementing an interface method.
 - Catches typos or mismatches early.
 - @Deprecated – Marks a method or class as outdated.
 - Warns developers to avoid using it.
 - @FunctionalInterface – Ensures the interface has only one abstract method.
 - Required for lambda expressions.
 - Fails at compile-time if multiple abstract methods exist in an interface.
- Marker Annotations: Annotations without parameters
- Framework Usage? – Widely used in Java, Spring, JUnit, Jakarta EE
- Examples of Using Annotations

Java

```

class Parent {
    void show() {
        System.out.println("Parent class method");
    }
}

class Child extends Parent {
    @Override
    void show() {
        System.out.println("Child class method");
    }

    @Deprecated
    void oldMethod() {
        System.out.println("This method is deprecated");
    }

    @SuppressWarnings("unchecked")
    void warningFree() {
        // No warning due to SuppressWarnings
        List rawList = new ArrayList();
    }
}

@FunctionalInterface
interface Greeter {
    void greet(String name);

    // Uncommenting the below method
    // would cause a compile error

    // void sayBye();
}

```

Q8

How do you define and use a custom annotation in Java?

- What? – Custom annotations are user-defined markers that add metadata to code
- @Retention – Controls how long the annotation is retained
 - SOURCE – Discarded by compiler, used only in code (e.g., @Generated for code gen tools)
 - CLASS – Kept in bytecode, ignored at runtime (e.g., @Deprecated)
 - RUNTIME – Available at runtime via reflection (e.g., @Override, custom annotations)
- @Target – Controls where the annotation can be applied
 - METHOD – Used on methods
 - FIELD – Used on fields or variables
 - TYPE – Used on classes, interfaces, enums
 - PARAMETER – Used on method parameters
 - CONSTRUCTOR, etc. – More specialized use cases
- Creating a Custom Annotation:
- Applying the Custom Annotation
- Using Reflection to Process the Annotation
- Creating a Simple Unit Test Framework with Annotations
 - The @Test annotation is applied to test methods.
 - TestRunner automatically finds and runs methods marked with @Test.
 - The test description is displayed before executing each test.
 - Example Output

Python

```
import java.lang.annotation.*;

// Define the custom annotation
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface TestAnnotation {
    String value(); // Annotation element
}
```

Java

```
class Example {
    @TestAnnotation("This is a test method")
    public void myMethod() {
        System.out.println("Method executed");
    }
}
```

Java

```

import java.lang.reflect.Method;

public class AnnotationProcessor {
    public static void main(String[] args)
        throws Exception {

        Example example = new Example();

        // Get the method
        Method method
            = Example.class.getMethod("myMethod");

        // Check if annotation is present
        if (method.isAnnotationPresent(
            TestAnnotation.class)) {

            // Retrieve the annotation
            TestAnnotation annotation
                = method.getAnnotation(TestAnnotation.class);

            System.out.println(
                "Annotation Value: " + annotation.value());

            // Execute the method dynamically
            method.invoke(example);
        }
    }
}

```

Java

```

import java.lang.annotation.*;
import java.lang.reflect.Method;

// Define a test annotation
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Test {
    String description() default "No description";
}

// Example test class
class TestSuite {
    @Test(description = "Test for addition")
    public void testAddition() {
        System.out.println("Addition Test Passed!");
    }

    @Test(description = "Test for subtraction")
    public void testSubtraction() {
        System.out.println("Subtraction Test Passed!");
    }
}

// Annotation Processor
//Runs all methods with @Test annotation
public class TestRunner {
    public static void main(String[] args)
        throws Exception {

        TestSuite testSuite = new TestSuite();

        for (Method method :
            TestSuite.class.getDeclaredMethods()) {

            if (method.isAnnotationPresent(Test.class)) {

                Test test
                    = method.getAnnotation(Test.class);

                System.out.println(
                    "Running Test: " +
                    test.description());

                // Execute the test method
                method.invoke(testSuite);
            }
        }
    }
}

```

Code

```

Running Test: Test for addition
Addition Test Passed!
Running Test: Test for subtraction
Subtraction Test Passed!

```

Java

```

import java.lang.annotation.*;
import java.lang.reflect.Method;

// Define a test annotation
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Test {
    String description() default "No description";
}

// Example test class
class TestSuite {
    @Test(description = "Test for addition")
    public void testAddition() {
        System.out.println("Addition Test Passed!");
    }

    @Test(description = "Test for subtraction")
    public void testSubtraction() {
        System.out.println("Subtraction Test Passed!");
    }
}

// Annotation Processor
//Runs all methods with @Test annotation
public class TestRunner {
    public static void main(String[] args)
        throws Exception {

        TestSuite testSuite = new TestSuite();

        for (Method method :
            TestSuite.class.getDeclaredMethods()) {

            if (method.isAnnotationPresent(Test.class)) {

                Test test
                    = method.getAnnotation(Test.class);

                System.out.println(
                    "Running Test: " +
                    test.description());

                // Execute the test method
                method.invoke(testSuite);
            }
        }
    }
}

```

Code

```

Running Test: Test for addition
Addition Test Passed!
Running Test: Test for subtraction
Subtraction Test Passed!

```

Q9

What are unnamed variables and patterns, and why are they useful?

Unnamed variables (`_`) and unnamed patterns (`catch (NumberFormatException _)`) help reduce clutter when a variable or exception is declared but not used.

Example 1: Using Unnamed Variable in Try-Catch

Example 2: Unnamed Pattern in Switch Expressions

Example 3: Unnamed Variable in For-Each Loops

Example 4: Unnamed Variable in Record Patterns

Example 5: Ignoring Unused Return Values

Advantages

- Cleaner Code – Removes unnecessary variable names when they are not used.
- Improved Readability – Makes the intent clearer by avoiding unused variables.
- Less Clutter – No warnings for unused variables, reducing distractions in IDEs.

Code

```
try {  
    // Invalid integer  
    int n = Integer.parseInt("abc");  
} catch (NumberFormatException _) { // No name needed  
    //} catch (NumberFormatException ex) { // 'ex' is unused  
    System.err.println("Invalid number format!");  
}
```

Java

```
static void checkObject(Object obj) {  
    switch (obj) {  
        case String _  
            -> System.out.println("It's a string.");  
        case Integer _  
            -> System.out.println("It's an integer.");  
        default  
            -> System.out.println("Unknown type.");  
    }  
}
```

Java

```
String[] names = {"Alice", "Bob", "Charlie"};  
  
//Since we don't need to use the variable, `_` replaces it.  
for (String _ : names) {  
    System.out.println("Processing...");  
}
```

Java

```
record Person(String name, int age) {}

static void printPerson(Object obj) {
    //Here, `name` is ignored, as we only need `age`.
    if (obj instanceof Person(_, int age)) {
        System.out.println("Person's age: " + age);
    }
}
```

Java

```
static int calculate(int a, int b) {
    return a + b;
}

public static void main(String[] args) {
    _ = calculate(10, 20); // We ignore the return value
    System.out.println("Calculation done.");
}
```

Q10

How has pattern matching evolved across recent Java versions?

Before Java 14 Example

- Requires explicit casting ((String) obj).
- Redundant code for type checking.

Java 14: Simplified instanceof Check

- Eliminates explicit casting.
- Declares variable inline.
- Cleaner and safer.

Java 21: Record Patterns

- Java 21 allows automatic deconstruction of records.
- Automatically extracts fields (sender, receiver, amount).

Java 21: Nested Record Patterns

- Nested pattern matching extracts values directly.
- Eliminates manual field access (order.customer().name()).
- More concise and readable.

Java


```

public void process(Object obj) {
    if (obj instanceof String) {
        // Explicit casting needed
        String s = (String) obj;
        System.out.println("Message: " + s);
    }
}

```

Java

```

public void process(Object obj) {
    if (obj instanceof String s) {
        // No explicit cast needed
        System.out.println("Message: " + s);
    }
}

```

Java

```

record Transaction(
    String sender, String receiver
    , double amount) {}

public void processTransaction(Object obj) {
    if (obj instanceof Transaction(
        String sender, String receiver,
        double amount)) {
        System.out.println("Processing transaction: "
            + sender + " -> "
            + receiver + " : " + amount);
    }
}

```

Java

```

package com.in28minutes;

record Customer(String name, String email) {}

record Product(String name, double price) {}

record Order(Customer customer, Product product) {}

public class CustomerSupport {
    public static void processOrder(Object obj) {
        if (obj instanceof Order(
            Customer(String name, String email),
            Product(String prodName, double price))) {

            System.out.println("Customer " + name +
                " ordered " + prodName +
                " for " + price);

        }
    }

    public static void main(String[] args) {
        processOrder(new Order(
            new Customer("Ranga", "Email"),
            new Product("Cricket Bat", 100)));
    }
}

```

Q11

How have switch expressions improved in modern Java?

Java 14: Arrow Syntax (->) in switch

- No need for break statements.
- More concise and readable.
- Uses -> instead of case : syntax.

Java 14: Switch Expression with Return Values

- switch as an expression (returns a value).

Java 17: Switch Expression with Enums

- Eliminates repetitive if-else logic.

Java 21: Switch Expression with Record Patterns

- Pattern Matching in switch allows direct record deconstruction.
- More concise and readable code.

Java 21: Nested Record Patterns in Switch Expressions

- Switch expressions work seamlessly with nested records.
- Automatic deconstruction of records inside switch.

Java 22: when Guards in Switch Expressions

- What? Add extra conditions inside switch cases
- Why? Filter patterns without nesting if statements
- Benefit? Cleaner and more powerful control flow

Example: Using when for Additional Filtering

Java

```
switch (day) {
    case "Monday" -> System.out.println("Start of the week");
    case "Friday" -> System.out.println("Weekend is near");
    case "Sunday" -> System.out.println("Weekend");
    default -> System.out.println("Regular day");
}
```

Java

```
public class CustomerSupportSwitchExpression {
    public static void main(String[] args) {
        Object message = "Need help with my order";

        String response = switch (message) {
            case String text ->
                "Processing text message: "
                    + text;
            case Integer ticketNumber ->
                "Processing support ticket: #"
                    + ticketNumber;
            case Double rating ->
                "Processing feedback rating: "
                    + rating + " stars";
            case byte[] attachment ->
                "Processing attachment of size: "
                    + attachment.length;
            default -> "Unknown message type.";
        };

        System.out.println(response);
    }
}
```

Java

```

enum Priority {
    LOW, MEDIUM, HIGH, CRITICAL
}

public class CustomerSupportJava17 {
    public static void main(String[] args) {

        // Example: High-priority ticket
        Priority priority = Priority.HIGH;

        String response = switch (priority) {
            case LOW
                -> "Response within 48 hours.";
            case MEDIUM
                -> "Response within 24 hours.";
            case HIGH
                -> "Immediate response required!";
            case CRITICAL
                -> "Escalating to the highest level!";
        };

        System.out.println(response);
    }
}

```

Java

```

sealed interface CustomerMessage
    permits TextMessage, Ticket, Feedback, Attachment {}

record TextMessage(String text) implements CustomerMessage {}
record Ticket(int ticketNumber) implements CustomerMessage {}
record Feedback(double rating) implements CustomerMessage {}
record Attachment(byte[] file) implements CustomerMessage {}

public class CustomerSupportJava21 {
    public static void main(String[] args) {
        CustomerMessage message = new Ticket(1050);

        String response = switch (message) {
            case TextMessage(String text)
                -> "Processing text message: "
                    + text;
            case Ticket(int ticketNumber)
                -> "Processing support ticket: #"
                    + ticketNumber;
            case Feedback(double rating)
                -> "Processing feedback rating: "
                    + rating + " stars";
            case Attachment(byte[] file)
                -> "Processing attachment of size: "
                    + file.length;
        };

        System.out.println(response);
    }
}

```

Java

```

package com.in28minutes;

sealed interface CustomerMessage
    permits TextMessage, Ticket,
           Feedback, Attachment {}

record TextMessage(String text)
    implements CustomerMessage {}

record Ticket(int ticketNumber)
    implements CustomerMessage {}

record Feedback(double rating)
    implements CustomerMessage {}

record Attachment(byte[] file)
    implements CustomerMessage {}

record SupportRequest(String user,
                     CustomerMessage message) {}

public class CustomerSupport {
    public static void main(String[] args) {
        SupportRequest request
            = new SupportRequest("Alice",
                               new Feedback(4.8));

        String response = switch (request) {
            case SupportRequest(String user,
                               Ticket(int ticketNumber)) ->
                "User " + user +
                " has a support ticket: #" + ticketNumber;

            case SupportRequest(String user,
                               Feedback(double rating)) ->
                "User " + user +
                " left a rating: " + rating + " stars";

            case SupportRequest(String user,
                               TextMessage(String text)) ->
                "User " + user +
                " sent a message: " + text;

            case SupportRequest(String user,
                               Attachment(byte[] file)) ->
                "User " + user +
                " uploaded a file of size: " + file.length;
        };

        System.out.println(response);
    }
}

```

Java

```
record Feedback(double rating) implements CustomerMessage {}

public class SupportWithWhen {
    public static void main(String[] args) {
        CustomerMessage message = new Feedback(2.5);

        String response = switch (message) {
            case Feedback(double rating) when rating < 3.0 ->
                "We're sorry to hear that. Support will contact you.";
            case Feedback(double rating) ->
                "Thanks for your positive feedback!";
            default ->
                "Unhandled message type.";
        };

        System.out.println(response);
    }
}
```

CHAPTER 16

New Features in Java

3 Questions & Answers

Q1

How is Java Marching Forward?

Breaking Myth

- Myth: "Java is old and unpopular"
- Truth: Java remains one of the most popular programming languages worldwide!

Java is one of the popular choices for building:

Modern cloud applications (Spring Boot, Quarkus, Micronaut)

IMHO Java is evolving faster than ever, staying modern, powerful, and future-proof!

Continuous Innovation in Java Eco System

- Java 8 – Embraced functional programming with Lambda expressions and Streams.
- Java 9 – Introduced modularization to the Java Platform - enabling faster startup and less memory footprint for Java applications
- Java 10 to NOW – Started the six-month release cycle for faster updates and quick feedback.
- Java 11 to NOW – Reduced code verbosity
 - Switch Expressions – More concise conditional logic.
 - Text Blocks – Simplifies multi-line strings.
 - Records – Reduces boilerplate for data objects.
 - Pattern Matching – Enhances code readability and conciseness.
- Java 21 to NOW:
 - Project Loom – Virtual threads for high-performance concurrency.
 - Cloud & Container-Ready – Optimized for modern cloud-native applications.

Q2

What are the primary goals behind recent Java releases?

- **Make it concise – Reduce boilerplate and improve readability.**
Records – No need for boilerplate code in data classes.
Switch Expressions – More readable and concise switch statements.
- **Cloud and container support – Improve compatibility with cloud-native environments.**
Optimized JDK Container Images – Optimized JDK images for containers with small memory footprint and fast start up times.
GraalVM – Enables efficient native image compilation for lightweight, fast-running applications.
- **Improve concurrency – Introduce Virtual Threads for scalable parallelism.**
Virtual Threads – Handles millions of lightweight threads efficiently.
- **Developer productivity – Provide better debugging tools and runtime efficiency.**
JShell – Interactive REPL for quick testing.
Simpler Main Without Class - Write just the method, skip the boilerplate
Easy Launch without Compilation - Run source files directly
- **Reduce startup time – Optimize performance for faster application launch.**
- **Better memory management – Optimize Garbage Collection (GC) for lower latency.**
- **Backward compatibility – Ensure smooth migration from older versions.**

Code

```
record Person(String name, int age) {}
```

Code

```
String result = switch (day) {
    case MONDAY, FRIDAY -> "Weekend is near!";
    default -> "Regular day.";
};
```

Code

```
record Person(String name, int age) {}
```

Code

```
String result = switch (day) {
    case MONDAY, FRIDAY -> "Weekend is near!";
    default -> "Regular day.";
};
```

Java

```
Thread.startVirtualThread(
    () -> System.out.println(
        "Hello from a virtual thread!"));
```

Java

```
Thread.startVirtualThread(  
    () -> System.out.println(  
        "Hello from a virtual thread!"));
```

Java

```
jshell> System.out.println("Hello, Java!");
```

Java

```
//HelloWorld.java  
void main() {  
    System.out.println("Hello, World!");  
}
```

Code

```
$ java HelloWorld.java
```

Java

```
jshell> System.out.println("Hello, Java!");
```

Java

```
//HelloWorld.java  
void main() {  
    System.out.println("Hello, World!");  
}
```

Code

```
$ java HelloWorld.java
```

Q3

What are some major Java versions and their most important features?

J2SE 5.0

- Enhanced For Loop – Simplifies iteration over collections and arrays.
- Generics – Enables type safety in collections and methods.
- Enums – Provides a type-safe way to define constants.
- Autoboxing – Automatically converts primitives to wrapper classes.

Java SE 7

- Try-With-Resources: Automates resource management in try-catch blocks.
- Strings in Switch: Allows using String values in switch statements.

Java SE 8

- Lambda Expressions – Allows writing concise, functional-style code.
- Streams API – Provides a powerful way to process collections.
- Functional Programming – Introduces higher-order functions and immutability.

Java SE 9

- Modularization – Splits JDK into modules for better scalability.
- JShell (REPL) – Interactive command-line tool for testing Java code.

Java SE 10

- Local Variable Type Inference (var) – Allows type inference for local variables.

Java SE 14

- Switch Expressions – Enables switch to return values, reducing boilerplate code.

Java SE 15

- Text Blocks – Simplifies working with multi-line strings.

Java SE 16

- Pattern Matching for instanceof - Simplify code making use of instanceof
- Record Classes – Provides a compact syntax for immutable data objects.

Java SE 17

- Sealed Classes - Controlled inheritance of your class hierarchies.

Java SE 21

- Virtual Threads – Improves concurrency with lightweight threads.
- Sequenced Collections – Adds operations for ordered collections.
- Record Patterns - Supports deconstruction, including nested patterns
- Pattern Matching for Switch - More expressive and type-safe switch statements

Java SE 22

- Unnamed Variables & Patterns – Allows ignoring unused variables in code.

Java SE 23

- Markdown Documentation Comments – Enhances Javadoc with Markdown support.

Java SE 24

- Stream Gatherers – Introduces new ways to aggregate stream data.

Ongoing Improvements

- Performance Optimizations – Enhances start up speed, memory footprint and efficiency of Java applications.
- Garbage Collection Enhancements – Improves memory management and GC algorithms.
- API and Syntactic Improvements – Continuously refines Java syntax and library functions.

Code

```
for (int num : numbers) {  
    //Do something  
}
```

Java

```
public class HashMap<K,V> extends AbstractMap<K,V>
```

Code

```
enum Season {  
    WINTER, SPRING, SUMMER, FALL  
}
```

Code

```
Integer integer = 5;  
int i = integer;
```

Code

```
try (BufferedReader br =  
    new BufferedReader(  
        new FileReader("file.txt"))) {
```

Code

```
switch (day) {  
    case "Monday": //Do Something;  
    case "Friday": //Do Something;  
    case "Sunday": //Do Something;  
    default: //Do Something;  
}
```

Code

```
public static Predicate<Course>  
    createPredicate(int cutoff) {  
    return course ->  
        course.getReviewScore() > cutoff;  
}
```

Code

```
List<String> transformedNumbers = numbers.stream()  
    .map(x -> x * x) // Step 1: Square each number  
    .map(x -> x + 10) // Step 2: Add 10  
    .map(x -> "Value: " + x) // Step 3: Convert  
    .toList(); // Collect the result
```

Java

```
courses.stream()  
    .sorted((c1, c2) -> c1.length() - c2.length())  
    .forEach(System.out::println);
```

Java

```
jshell> System.out.println("Hello, Java!");
```

Code

```
// ArrayList<String> numbers = new ArrayList<>();

// No need to repeat ArrayList<String>
var numbers = new ArrayList<String>();

// Inferred as HashMap<Integer, String>
var map = new HashMap<Integer, String>();
```

Code

```
String result = switch (day) {
    case MONDAY, FRIDAY -> "Weekend is near!";
    default -> "Regular day.";
};
```

JavaScript

```
String json = ""
{
  "name": "John",
  "age": 30
}
"";
//json ==> "{\n  \"name\": \"John\", \n  \"age\": 30\n}\n"
```

Java

```
public void process(Object obj) {
    if (obj instanceof String s) {
        // No explicit cast needed
        System.out.println("Message: " + s);
    }
}
```

Code

```
public record Person(
    String name, String email,
    String phoneNumber) {}
```

Code

```
sealed class Vehicle permits Car, Truck, Bike {}

// No further subclassing
final class Car extends Vehicle {}

// Can be extended freely
non-sealed class Truck extends Vehicle {}

sealed class Bike extends Vehicle
    permits ElectricBike {}

// No further subclassing
final class ElectricBike extends Bike {}
```

Java

```
Thread.startVirtualThread(
    () -> System.out.println(
        "Hello from a virtual thread!"));
```

Code

```
interface SequencedCollection<E> extends Collection<E> {
    void addFirst(E element);
    void addLast(E element);
    E getFirst();
    E getLast();
    E removeFirst();
    E removeLast();
    SequencedCollection<E> reversed();
}
```

Java

```
record Transaction(
    String sender, String receiver, double amount) {}

public void processTransaction(Object obj) {
    if (obj instanceof Transaction(
        String sender, String receiver, double amount)) {
        System.out.println("Processing transaction: "
            + sender + " -> " + receiver + " : " + amount);
    }
}
```

Java

```

interface CustomerMessage{}
record TextMessage(String text) implements CustomerMessage {}
record Ticket(int ticketNumber) implements CustomerMessage {}

CustomerMessage message = new Ticket(1050);

String response = switch (message) {
    case TextMessage(String text)
        -> "Processing text message: "
            + text;
    case Ticket(int ticketNumber)
        -> "Processing support ticket: #"
            + ticketNumber;
};

System.out.println(response);

```

Java

```

switch (obj) { //I will not use the value!
    case String _ -> System.out.println("String");
    case Integer _ -> System.out.println("Integer");
    default -> System.out.println("Unknown type.");
}

```

Code

```

/// This is the traditional
/// Hello World! program.

/// This is the traditional
/// Hello World! program.

```

Code

```

//A new way to group,
//transform, and aggregate stream elements
List<List<Integer>> windows = numbers.stream()
    .gather(Gatherers.windowFixedSize(3))
    .toList();

//[1, 2, 3]
//[4, 5, 6]
//[7, 8, 9]
//[10]

```


CHAPTER 17

Java Interview - Diagrams

7 Questions & Answers

Q1

Evolution to Java

Code

```
graph TD;
  A[Writing Machine Code] --> B[Using Assembly Language]
  B --> C[Structured Programming with C Language]
  C --> D[OOPS with C++]
  D --> E[OOPS with Java]
  E --> F[Constant Java Updates]
```

Q2

Java Platform Independence

Code

```
graph TD
  A[Java Code] -->|javac Compiled| C[bytecode]
  C --> D[Windows JVM]
  D --> K[Windows Machine Code]
  C --> E[Unix JVM]
  E --> L[Unix Machine Code]
  C --> F[Linux JVM]
  F --> M[Linux Machine Code]
  C --> G[Other platform JVM]
  G --> N[Other Platform Machine Code]
```

Q3

Compiled Languages

Code

```
graph TD
  A[Code] -->|Windows Compiler| B[Windows Machine Code]
  A[Code] -->|Linux Compiler| C[Linux Machine Code]
  A[Code] -->|Mac Compiler| D[Mac Machine Code]
```

Q4

JDK vs JVM vs JRE

Code

```
graph TD;
JDK --> |contains| JRE;
JRE --> |contains| JVM;
```

Q5

JVM is Reused by Other Programming Languages

Code

```
graph TD
A[Java Code] -->|Compiled| C(bytecode)
AA[Kotlin Code] -->|Compiled| C(bytecode)
AAA[Scala Code] -->|Compiled| C(bytecode)
AAAA[Groovy Code] -->|Compiled| C(bytecode)
C --> D[Windows JVM]
D --> K[Windows Machine Code]
C --> E[Unix JVM]
E --> L[Unix Machine Code]
C --> F[Linux JVM]
F --> M[Linux Machine Code]
C --> G[Other platform JVM]
G --> N[Other Platform Machine Code]
```

Q6

HashCode usage in Buckets

Code

```
graph LR
%% Hash Code Calculation Outside HashMap %%
A1["Key: 'John' hashCode(35123) 35123 % 3 Index 1"] -->|Stored at Index 1| B1
A2["Key: 'Emma' hashCode(67234) 67234 % 3 Index 2"] -->|Stored at Index 2| B2
A3["Key: 'Alice' hashCode(89125) 89125 % 3 Index 0"] -->|Stored at Index 0| B3
A4["Key: 'Bob' hashCode(11234) 11234 % 3 Index 1"] -->|Stored at Index 1| B1
A5["Key: 'Charlie' hashCode(98341) 98341 % 3 Index 2"] -->|Stored at Index 2| B2
A6["Key: 'David' hashCode(54189) 54189 % 3 Index 0"] -->|Stored at Index 0| B3

%% HashMap Buckets %%
subgraph HashMap
B1["Bucket[1]: ('John', 1001) ('Bob', 4004)"]
B2["Bucket[2]: ('Emma', 2002) ('Charlie', 5005)"]
B3["Bucket[0]: ('Alice', 3003) ('David', 6006)"]
end
```

Q7

Parallelism and Concurrency

Sequential Execution

Concurrent Execution

Parallelism

Concurrency + Parallelism

OS Process and Threads

Platform Threads

Virtual Threads

Code

```
sequenceDiagram
    participant Main as Main Thread
    participant S1 as Step 1 (IO-Bound)
    participant S2 as Step 2 (IO-Bound)
    participant S3 as Step 3 (Processing)

    Main->>S1: Start IO Task 1
    S1-->>Main: Return Result 1 (after IO)
    Main->>S2: Start IO Task 2
    S2-->>Main: Return Result 2 (after IO)
    Main->>S3: Process Result 1 & 2
    S3-->>Main: Final Output
```

Code

```
sequenceDiagram
    participant Main as Main Thread
    participant S1 as Step 1 (IO-Bound)
    participant S2 as Step 2 (IO-Bound)
    participant S3 as Step 3 (Processing)

    Main->>S1: Start IO Task 1
    Main->>S2: Start IO Task 2
    S1-->>Main: Return Result 1 (after IO)
    S2-->>Main: Return Result 2 (after IO)
    Main->>S3: Process Result 1 & 2
    S3-->>Main: Final Output
```

Code

```
graph TD;
    A[Parallelism] -->|Needs Multi-Core| B[Multiple Cores];
    B --> C[Core 1];
    B --> D[Core 2];
    B --> E[Core 3];
    B --> F[Core 4];
    C -->|Runs Task A| G[Task A];
    D -->|Runs Task B| H[Task B];
    E -->|Runs Task C| I[Task C];
    F -->|Runs Task D| J[Task D];
```

Code

```
graph TD;
  A[Concurrency + Parallelism] -->|Efficient| B[Concurrency];
  A -->|Fast| C[Parallelism];
  B -->|Task Switching| D[One CPU, Multiple Tasks];
  C -->|Task Splitting| E[Multiple CPUs, Multiple Tasks];
```

Code

```
graph TD;
  A[OS Process] -->|Has| B[Multiple OS Threads];
  B --> C[OS Thread 1];
  B --> D[OS Thread 2];
  B --> E[OS Thread 3];
```

Code

```
graph TD;
  A[OS Process] -->|Has| B[Multiple OS Threads];
  B --> C[OS Thread 1];
  C --> CC[Platform Thread 1];
  B --> D[OS Thread 2];
  D --> DD[Platform Thread 2];
  B --> E[OS Thread 3];
  E --> EE[Platform Thread 3];
```

Code

```
graph TD;
  A[OS Thread 1] -->|Maps to| B[Multiple Virtual Threads];
  B --> C[Virtual Thread 11];
  B --> D[Virtual Thread 12];
  B --> E[Virtual Thread 13];
  B --> F[Virtual Thread 14];
  B --> G[...];
  B --> H[Virtual Thread n];
```